

ProSeller — Contrato do Sistema

- [ProSeller — Contrato do Sistema \(revisão completa\)](#)
 - [Índice](#)
- [System Contract — Índice](#)
 - [Domínios](#)
 - [Resumo por domínio](#)
- [Contrato — Arquitetura & Stack](#)
 - [Invariants](#)
 - [Business Rules](#)
 - [Arch Decisions](#)
- [Contrato — Catálogo de Edge Functions](#)
 - [business-rule](#)
 - [arch-decision](#)
- [Contrato — Clientes](#)
 - [Business rules](#)
 - [Invariants](#)
 - [Arch decisions](#)
- [Contrato — Vendas & Emissão ao Tiny \(ERP\)](#)
 - [Invariantes](#)
 - [Regras de negócio](#)
- [Contrato — Fiscal: Simples Nacional & Natureza de Operação](#)
 - [Business rules](#)
 - [Invariants](#)
 - [Arch decisions](#)
- [Contrato — Condições de Pagamento](#)
 - [Invariantes](#)
 - [Regras de Negócio](#)
 - [Decisões de Arquitetura](#)
- [Contrato — Comissões](#)
 - [Invariantes \(garantidas por constraint/schema\)](#)
 - [Business rules](#)
- [Contrato — Logística & Rastreo SSW](#)
 - [Invariantes](#)
 - [Regras de negócio](#)
- [Contrato — Permissões, Usuários & RLS](#)
 - [Business Rules](#)
 - [Invariants](#)
 - [Arch Decisions](#)
- [Contrato — Produtos & Listas de Preço](#)
 - [Invariantes](#)
 - [Regras de negócio](#)
 - [Decisões de arquitetura](#)
- [Contrato — Relatórios](#)
 - [Business rules](#)
 - [Architecture decisions](#)
- [Contrato — Conta Corrente](#)
 - [Invariants](#)
 - [Business rules](#)
 - [Arch decisions](#)
- [Contrato — Convenções de Dados & Padrões](#)
 - [Invariants](#)

- [Business Rules](#)
- [Arch Decisions](#)
- [Regression Checklist — Revisão de feature antes do merge](#)
 - [Clientes / persistência \(clientes.md\)](#)
 - [Vendas & emissão Tiny \(vendas-emissao-tiny.md\)](#)
 - [Condições de pagamento \(condicoes-pagamento.md\)](#)
 - [Fiscal / Simples / Natureza \(fiscal-simples-natureza.md\)](#)
 - [Logística SSW \(logistica-ssw.md\)](#)
 - [Permissões & RLS \(permissoes-rls.md\)](#)
 - [Edges & backend \(edges-catalog.md\)](#)
 - [Migrations \(arquitetura-stack.md, convencoes-dados.md\)](#)
 - [Convenções de dados \(convencoes-dados.md\)](#)
 - [Versionamento visível \(arquitetura-stack.md\)](#)
- [Known Incidents — Mapa incidente → invariante violada](#)
 - [INC — UPSERT com EXCLUDED apagava observação de contato](#)
 - [INC — Falso sucesso na emissão ao ERP](#)
 - [INC — Entrega rebaixada por evento administrativo posterior \(SSW\)](#)
 - [INC — Nome de condição parcelada mostrava só a última parcela](#)
 - [INC — Grupo/Rede não exibido no detalhe do cliente](#)
 - [INC/PITFALL — Número PV-2025-XXXX não é rastreável](#)
 - [INC — Optante Simples sumia no detalhe \(acento + omissão\)](#)
 - [INC — Emissão com regime Simples não confirmado](#)

ProSeller — Contrato do Sistema (revisão completa)

Consolidado da análise multi-agente (151 regras verificadas, 13 domínios, 48 refutadas descartadas na verificação adversarial). Gerado em 02/07/2026.

Índice

1. Visão geral (index) 2–14. Domínios
2. Checklist de regressão
3. Incidentes conhecidos

System Contract — Índice

Guarda anti-regressão do sistema. Cada arquivo deste diretório documenta as **invariantes, regras de negócio e decisões de arquitetura** verificadas no código de um domínio. O propósito é ser um contrato executável de leitura: **toda feature nova deve ser cotejada com estas invariantes antes de mergear**. Se uma mudança colide com um item aqui registrado, ou a invariante está errada (e o SPEC precisa ser corrigido primeiro), ou a feature está introduzindo uma regressão. Não editar um domínio sem cotejar o impacto cruzado. Ver também `regression-checklist.md` (revisão prática por área) e `known-incidents.md` (mapa incidente → invariante violada).

Domínios

Domínio	Arquivo	Escopo
Arquitetura & Stack	arquitetura-stack.md	Estrutura do app, roteamento manual, camadas de dados, versionamento visível
Catálogo de Edges	edges-catalog.md	Edge Functions -v2, verify_jwt, contratos de payload
Clientes	clientes.md	update_cliente_v2, UPSERT/COALESCE, get_cliente_completo_v2, mappers de detalhe
Vendas & Emissão Tiny	vendas-emissao-tiny.md	Envio ao ERP, tiny.pedido_id, sucesso real vs falso sucesso
Fiscal / Simples / Natureza	fiscal-simples-natureza.md	Optante Simples, tipoPessoa, bloqueio de emissão sem regime confirmado
Condições de Pagamento	condicoes-pagamento.md	intervalo_parcela como source-of-truth, nome com todas as parcelas
Comissões	comissoes.md	Cálculo, rateio, regras de vendedor
Logística SSW	logistica-ssw.md	Resolver de status, eventos sticky, cron ssw-sweep-hourly
Permissões & RLS	permissoes-rls.md	Permissionamento backoffice, allow_all pendente, update-user-v2
Produtos & Listas de Preço	produtos-listas-preco.md	Catálogo, listas, imagens
Relatórios	relatorios.md	Dashboards, positivação, agregações
Conta Corrente	conta-corrente.md	Lançamentos, saldo, visualização
Convenções de Dados	convencoes-dados.md	Normalização, *_nome via join, formatos BR

Resumo por domínio

Contagem de itens verificados por domínio. refutadas = hipóteses testadas contra o código e **descartadas** (não representam o comportamento real); ficam documentadas para evitar re-investigação.

Domínio	Itens	Refutadas
arquitetura-stack	9	4
edges-catalog	9	8
clientes	13	5
vendas-emissao-tiny	8	4
fiscal-simples-natureza	11	1
condicoes-pagamento	8	3
comissoes	22	2
logistica-ssw	14	2
permissoes-rls	12	8
produtos-listas-preco	13	1
relatorios	7	6
conta-corrente	17	3

Domínio	Itens Refutadas	
convencoes-dados	8	1
Total	151	44

Contrato — Arquitetura & Stack

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Invariants

Roteamento manual via `useState<Page>` em `App.tsx`

O roteamento entre páginas é exclusivamente controlado por `useState<Page>` em `src/App.tsx`. Não há React Router. As transições de página chamam `setCurrentPage` com tipos discriminados (Page union type). `handlePageChange` valida permissões antes de mudar página.

- **Tipo:** invariant
- **Evidência:**
 - `src/App.tsx:79,302` — `type Page union type; useState<Page>("dashboard")` inicializa sempre com dashboard
 - `src/App.tsx:541-582` — `handlePageChange` valida permissões antes de mudar página via `setCurrentPage`
- **Regressão se:** Adicionar React Router, Next.js, roteamento por URL hash/path, ou navegação sem passar por `setCurrentPage`.

Branch `main` é fonte de verdade única (git espelha produção)

O branch `main` é a fonte de verdade INTENCIONADA para frontend, Edge Functions e migrations, com suporte documental e procedural (ADR-005, runbooks, AGENTS.md §14–15). Deploy manual de backend (edge functions + migrations) é sempre a partir do `main`, APÓS commitar. Entretanto, o enforcement é apenas procedural, não técnico:

- a. master NÃO foi arquivado — ainda existe em `origin/master` desatualizado;
- b. CI valida apenas `main` (bloqueando merge), mas deploy manual (Edge Functions + Netlify) NÃO valida branch — aceitam qualquer ramo;
- c. Commitar antes de deployar é procedimento exigido (o supabase functions deploy lê o working tree), mas sem check técnico pré-deploy;
- d. Conversão `repo↔prod` é via git, sem barreira técnica caso o operador execute deploy de branch errado ou código não-commitado.
- **Tipo:** invariant
- **Evidência:**
 - `AGENTS.md:90-91` — `main` é fonte de verdade ÚNICA; master arquivada em 2026-06-02 (convergência)
 - `AGENTS.md:255` — Git ESPELHA produção; commitar ANTES de deployar; deploy manual sempre a partir do `main`
 - `docs/wiki/runbooks/deploy-edge-function.md:20-23` e `deploy-netlify.md:26-29` — scripts de deploy manual aceitam qualquer branch (`git pull` sem validação)

- **Regressão se:** Deployar edge function ou migration de branch não-main, deployar código não-commitado, ou usar master como branch de trabalho.

data_initialized localStorage sempre true (DataInitializer desabilitado)

O DataInitializer component existe mas está desabilitado (comentado).

localStorage.setItem('data_initialized', 'true') é feito no primeiro login. O sistema nunca apresenta tela de inicialização de dados. ENTRETANTO, há código morto: handleDataInitComplete chama setShowDataInit(false), mas a variável de estado showDataInit não foi declarada e não existe mais no componente AppContent.

- **Tipo:** invariant
- **Evidência:**
 - src/App.tsx:338,366–372 — dataInitialized sempre true; useEffect garante localStorage.setItem('data_initialized', 'true')
 - src/App.tsx:1215–1218 — render do DataInitializer comentado (desabilitado)
 - src/App.tsx:376 — setShowDataInit(false) chamado em handleDataInitComplete(), mas showDataInit nunca declarado (código morto)
- **Regressão se:** Renderizar DataInitializer em alguma rota, ou deixar dataInitialized=false bloquear acesso ao dashboard.

Business Rules

Stack: React 18 + Vite + TypeScript + Supabase + Netlify

O frontend é SPA React 18 + Vite compilada para build, servida via Netlify. Backend é Supabase (Postgres + Deno Edge Functions). A maioria dos fluxos de dados passa por Edge Functions, mas existem exceções:

1. Alguns campos de dados_vendedor (idtiny, Comissão) são lidos/escritos via PostgREST direto (/rest/v1/);
2. Fluxos de autenticação usam /auth/v1/ direto;
3. Upload de arquivos usa Storage direto.

A nomenclatura de Edge Functions não é uniformemente -v2: mantém-se versões v1 por contrato (tiny-enviar-pedido-venda-v1) e novos módulos (logística, SSW) usam v1, além de funções sem versionamento. Não há frameworks MVC nem mudanças no build. A stack é efetivamente o contrato de deploy, mas com exceções pontuais documentadas e não-documentadas.

- **Tipo:** business-rule
- **Evidência:**
 - vite.config.ts:1–10 — defineConfig com React plugin SWC, outDir=build para Netlify
 - AGENTS.md:14 — Stack: React 18 + Vite + TypeScript (SPA) · Supabase (Postgres com RPCs + Edge Functions Deno) · Netlify
 - docs/wiki/architecture.md:8–35 — diagrama de 3 camadas: navegador (React SPA), Edge Functions (Deno), Postgres
 - src/services/api.ts:256–311 (contraexemplo) — fetchDadosVendedorIdtiny(), updateDadosVendedorIdtiny(), updateDadosVendedorComissao() fazem GET/PATCH via /rest/v1/dados_vendedor direto ao Postgres, contornando Edge Functions
- **Regressão se:** Trocar build tool (Webpack/Turbopack), mudar SSR/SSG, migrar backend para Node.js, deixar de usar Deno nas Edge Functions, ou self-host fora do Netlify.

Permissões discriminam backoffice vs. vendedor + telas backoffice-only

Usuários têm tipo ('backoffice' | 'vendedor') + array de permissões string (ex: 'clientes.visualizar'). Menu filtra por canAccessPage(page). Telas equipe/metastas/configuracoes/logistica requerem permissão específica ('equipe.visualizar', 'metas.visualizar', 'configuracoes.visualizar') OU ter 'config.geral' nas permissões. O fallback legado funciona via 'config.geral' no array de permissões (injetado por getDefaultPermissoes para tipo=backoffice), NÃO via verificação de tipo. Usuários tipo=backoffice COM PERMISSÕES EXPLÍCITAS que excluem 'config.geral' ficariam bloqueados.

- **Tipo:** business-rule
- **Evidência:**
 - src/App.tsx:88–102 — menuItems array com flag backofficeOnly; canAccessPage filtra por tipo + permissão
 - src/App.tsx:511–539 — canAccessPage: perfil/dashboard sempre; equipe/metastas/config requerem config.geral ou permissão específica
 - src/services/api.ts:40–80 — getDefaultPermissoes(tipo): backoffice tem defaults com config.geral; vendedor tem seller defaults
 - src/App.tsx:529–531 (contraexemplo) — verifica has("equipe.visualizar") || isLegacyAdmin, onde isLegacyAdmin depende de 'config.geral' no array, não do tipo do usuário
- **Regressão se:** Permitir vendedor acessar telas backoffice-only, remover permissão sem migração para usuários legacy, ou misturar autorização de forma inconsistente entre frontend e Edge Functions.

Arch Decisions

Edge Functions: padrão -v2 aspiracional; deploy manual imposto

O padrão de nomenclatura -v2 em Edge Functions é aspiracional (maioria seguindo <recurso>-v2, ~38 funções), mas NÃO imposto: ~14 funções sem versão no nome permanecem ativas (Criar_Vendedor, ObterListas, ObterProdutos, TinyEmitirAPI, emitir-pedido-sem-vendedor, emitirpedido, enviar-email-comissao, enviar-email-comissoes, export-csv, fetch_clientesSP, get-price-list-detail, upsert_price_list, verificarquantosclientes, webhook-tiny-atualizacao), nenhuma justificada como contrato externo ativo. Somente 1 v1 é genuinamente nomeado por contrato externo (tiny-enviar-pedido-venda-v1 / Tiny ERP). A regra de nomenclatura vive em wiki (memória), sem ADR governando nem lint/CI validando. O deploy via Supabase CLI é SIM imposto (ADR-005), nunca via MCP Cursor.

- **Tipo:** arch-decision
- **Evidência:**
 - docs/wiki/architecture.md:48–52 — funções em padrão <recurso>-v2/index.ts; deploy somente via Supabase CLI (ADR-005); Cursor MCP deploy_edge_function proibido
 - AGENTS.md:72 — Deploy de Edge Function EXCLUSIVAMENTE via Supabase CLI local, nunca via MCP
 - AGENTS.md:254 — Deploy de Edge Function só via Supabase CLI (ADR-005); MCP Cursor proibido
 - supabase/functions/webhook-tiny-atualizacao/index.ts (contraexemplo) — sem versão, mantida em 2026, sem contrato externo declarado
 - supabase/functions/enviar-email-comissao/index.ts (contraexemplo) — sem versão, ativa, consumida em src/services/api.ts
- **Regressão se:** Deployar via Cursor MCP, ou usar Node.js em lugar de Deno.

Feature flags: dual-layer (frontend compile-time + Edge runtime)

Feature flags seguem padrão dual-layer:

- **(A) Frontend:** VITE_FEATURE_LOG_CRM, VITE_FEATURE_SIMPLES_* lidas em **compile-time** via `import.meta.env` (inlined pelo Vite) em `App.tsx`, `SalesPage.tsx`.
- **(B) Edge Functions:** FEATURE_SIMPLES_NACIONAL_LOOKUP, FEATURE_LOG_CRM, FEATURE_LOG_CRM_AUTO_FRETE lidas em **runtime** via `Deno.env.get()` a partir de Supabase secrets, permitindo toggle de 1 clique sem redeploy (ADR-001).

Ambas as camadas usam convenção SCREAMING_SNAKE_CASE. Flags nunca são expostas como toggles de UI — apenas renderização condicional de componentes.

- **Tipo:** arch-decision
- **Evidência:**
 - `src/App.tsx:81,98-100` — `const FEATURE_LOG_CRM_ENABLED = import.meta.env.VITE_FEATURE_LOG_CRM === 'true'`; condicional no menu
 - `docs/wiki/architecture.md:84-88` — FEATURE_SIMPLES_NACIONAL_LOOKUP, FEATURE_LOG_CRM env vars; convenção FEATURE_<SCREAMING_SNAKE_CASE> (ADR-001)
 - `AGENTS.md:250` — Feature flag para contrato público; mudança que altera payload Tiny/ProSeller passa por flag até rollout controlado
 - `docs/decisions/adr/ADR-001-feature-flag-via-env-var.md:23-29` — Edge Function flags definidas como env var lida em **runtime** (contraste com compile-time)
- **Regressão se:** Usar feature flags de UI (localStorage toggles), misturar env vars com runtime toggles no frontend, ou variar comportamento feature-flag-compátível sem compilação.

Mapeamento Page → permissão via PAGE_VIEW_PERMISSION

A arquitetura implementa DOIS padrões distintos de autorização:

1. **Verificações EXPLÍCITAS** (`App.tsx:526-531`): `has(permission) || isLegacyAdmin` para `tiny-erp`, `clientes-pendentes`, `logistica`, `equipe`, `metas`, `configuracoes`.
 2. **PAGE_VIEW_PERMISSION** (`App.tsx:533-536`): `has(permission)` APENAS, SEM fallback de `isLegacyAdmin`, para `vendas`, `clientes`, `produtos`, `comissoes`, `contacorrente`, `relatorios`.
 3. **Páginas sem entrada em nenhum lugar** (`dashboard`, `perfil`, `changelog`): acesso sempre permitido (fallback permissivo `return true`).
- **Tipo:** arch-decision
 - **Evidência:**
 - `src/App.tsx:104-111` — PAGE_VIEW_PERMISSION mapeamento: `vendas->vendas.visualizar`, `clientes->clientes.visualizar`, etc.
 - `src/App.tsx:533-536` — `const requiredPermission = PAGE_VIEW_PERMISSION[page]`; retorna `has(requiredPermission)` se existe, senão `return true` (permissivo)
 - `src/App.tsx:526-531` (contraste) — páginas explícitas usam `has(...)` `|| isLegacyAdmin`; PAGE_VIEW_PERMISSION não inclui `isLegacyAdmin`
 - **Regressão se:** Nova página sem entrada em PAGE_VIEW_PERMISSION + sem check explícito em `canAccessPage`, ou checar permissão de forma diferente em frontend vs. Edge Function RLS.

UI Components via shadcn + Radix UI + Tailwind (CVA)

Componentes usam Radix UI (acessibilidade), shadcn (componentes pré-estilizados), Tailwind CSS (utility-first), CVA (variantes), `tailwind-merge` (utility cn). Path alias `@` aponta para `src/`. EXCEÇÕES:

1. `PedidoPrintView.css` contém estilos customizados para impressão;

2. `globals.css` define CSS variables e tipografia base;
 3. Alguns componentes (`ABCCurveCard`, `CustomerWalletCard`, `SegmentSalesCard`, etc.) usam `style={{}}` inline para valores dinâmicos, violando `utility-first`;
 4. Tailwind configurado via `@theme` inline em `globals.css`, sem `tailwind.config.js`.
- **Tipo:** arch-decision
 - **Evidência:**
 - `docs/wiki/architecture.md:40` — Componentes em `src/components/` (Radix + shadcn + Tailwind, padrão CVA + tailwind-merge)
 - `vite.config.ts:52` — Path alias `@ -> src/`, `@shared -> packages/shared`
 - `src/components/PedidoPrintView.tsx:1` (contraexemplo) — importa `./PedidoPrintView.css` com classes customizadas `.pp-*`
 - `src/components/ABCCurveCard.tsx:172-173` (contraexemplo) — usa `style={{ cursor: 'pointer' }}`, `style={{ opacity }}`, `style={{ backgroundColor: COLORS[...] }}` em vez de classes Tailwind/CVA
 - **Regressão se:** Usar Material-UI, Chakra UI, ou CSS-in-JS customizado em lugar de Tailwind + CVA.

Contrato — Catálogo de Edge Functions

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Documento gerado a partir de regras verificadas contra o código-fonte das Edge Functions em `supabase/functions/`. Para regras com `verdict partial`, o enunciado abaixo já incorpora o `corrected_statement` (a forma precisa e fiel ao código real).

business-rule

JWT validation via Bearer token

Enunciado (corrigido): All protected Edge Functions (mutações de dados e leitura de dados sensíveis) must validate incoming Authorization header as Bearer {token}, extract the token, call `supabase.auth.getUser(token)`, and verify user exists in user table with `ativo=true` and `deleted_at IS NULL`. Public/webhook Edge Functions (`entrega-publica-v1`, `TinyEmitirAPI`, `webhook-tiny-atualizacao`) que não requerem JWT devem ter (a) um mecanismo alternativo de validação explícito (ex.: chave de acesso de 44 dígitos) ou (b) um comentário explícito documentando que são intencionalmente públicas e sem autenticação. Data-reading functions (`ref-situacao-v2`, `export-csv`) sem autenticação devem incluir comentário explícito declarando que são públicas.

Tipo: business-rule

Evidência: - `supabase/functions/create-user-v2/index.ts:18-58` — `validateJWT` pattern: checks Authorization header, extracts Bearer token, calls `supabase.auth.getUser(token)`, queries user table com filtros `ativo=true` e `deleted_at IS NULL`. - `supabase/functions/list-users-v2/index.ts:17-51` — padrão idêntico de validação JWT: Bearer extraction, `getUser()`, user table query com checks `active/deleted_at`. - `supabase/functions/pedido-venda-v2/index.ts:18-55` — mesmo padrão nas funções v2, com campo `permissoes` também carregado.

Contraexemplo: `supabase/functions/ref-situacao-v2/index.ts:19-21` — seleciona de `ref_situacao` sem validação JWT nem comentário de endpoint público. Violações similares em `export-csv` (linhas 20-22, usa `ANON_KEY` e acessa `cliente` sem auth), `ObterListas` (linhas 51-69) e `Criar_Vendedor`.

Regressão se: uma função protegida aceita requisições sem Authorization header, pula os checks active/deleted_at, ou aceita tokens inválidos/expirados sem a verificação via supabase.auth.getUser.

Public webhook functions accept requests without JWT

Enunciado (corrigido): Apenas webhook-tiny-atualizacao e entrega-publica-v1 implementam o padrão no-Authorization com respostas always-200. Porém: (1) webhook-tiny-atualizacao NÃO valida webhook timestamp (não existe parsing/validação de timestamp), validando apenas a estrutura do payload; (2) entrega-publica-v1 GET valida nfe_chave_acesso, mas POST aceita qualquer freteId sem revalidar contra a chave_acesso (lacuna de segurança); (3) outras funções Tiny (tiny-verificar-pedido-v1) e de logística (frete-logistica-v1) EXIGEM Authorization e retornam status 4xx, provando que o padrão NÃO é universal entre funções webhook/public-driver.

Tipo: business-rule

Evidência: - supabase/functions/webhook-tiny-atualizacao/index.ts:327-390 — sem validação JWT; handler Deno.serve aceita POST/GET direto; sempre retorna status 200 (linhas 376-390) para evitar retry do Tiny. - supabase/functions/entrega-publica-v1/index.ts:1-15, 44-49 — função pública para motoristas; sem checks de Authorization; usa chave_acesso (chave NFe de 44 dígitos) como controle de acesso via query param.

Contraexemplo: supabase/functions/tiny-verificar-pedido-v1/index.ts:26-51 — retorna 405 para não-POST e 401 para Authorization ausente, violando o padrão “always-200, no-auth”.
supabase/functions/frete-logistica-v1/index.ts:36-52 — EXIGE Authorization header (linha 39 retorna erro se ausente).

Regressão se: a função webhook passa a exigir Authorization header/JWT, ou retorna status não-200 em erro (causando retry loops do Tiny/sistema externo), ou a validação de chave_acesso é removida dos endpoints públicos.

Webhook functions validate payload structure, not cryptographic signature

Enunciado: O webhook Tiny (webhook-tiny-atualizacao) valida a estrutura JSON (payload.tipo, payload.dados) e as respostas da API Tiny (retorno.status === 'OK'), mas NÃO valida assinatura criptográfica do webhook. Depende da unicidade da URL (o Tiny não consegue adivinhar a URL do endpoint) e de ACL/rate limiting.

Tipo: business-rule

Evidência: - supabase/functions/webhook-tiny-atualizacao/index.ts:46-100, 370-376 — handlePedido/handleNfe checam estrutura JSON: dados.id ou idVendaTiny requerido (linha 47), campo tipo checado (linha 364), resposta da API Tiny validada retorno.status === 'OK' (linha 93), mas sem validação HMAC/signature.

Regressão se: o webhook passa a exigir validação de assinatura criptográfica sem documentar como obter o segredo do webhook Tiny, ou a validação de estrutura é removida.

User deletion is soft-delete with deleted_at timestamp

Enunciado (corrigido): A regra soft-delete é parcialmente imposta: delete-user-v2 e a reativação em create-user-v2 funcionam conforme descrito (soft-delete setando deleted_at; reativação setando deleted_at=null),

MAS a validação de JWT não é consistentemente aplicada. Funções críticas como `get-user-v2` e um padrão base compartilhado (`_shared_helpers.ts`) NÃO filtram `soft-deleted users`, permitindo acesso a usuários deletados. A proteção depende de qual edge function é invocada — não é uma garantia arquitetural uniforme.

Tipo: business-rule

Evidência: - `supabase/functions/create-user-v2/index.ts:365-411` — linhas 365-370: identifica `usuarioSoftDeletado` via `!!u.deleted_at`; linhas 374-410: reativação seta `deleted_at: null` em vez de criar novo usuário. - `supabase/functions/list-users-v2/index.ts:36` — validação JWT inclui filtro `is('deleted_at', null)` para excluir `soft-deleted users`.

Contraexemplo: `supabase/functions/get-user-v2/index.ts:48-53` — `validateJWT` não inclui `.is('deleted_at', null)`, permitindo `soft-deleted user` autenticar. `supabase/functions/_shared_helpers.ts:36-41` — helper base também sem filtro, replicado em 52+ funções.

Regressão se: `delete-user-v2` remove linhas em vez de `soft-delete`, ou a validação JWT pula o filtro `is('deleted_at', null)`, ou `create-user-v2` lança erro em vez de reativar `soft-deleted users`.

User reactivation sends password reset email, not duplicate auth records

Enunciado: Quando `create-user-v2` encontra um `soft-deleted user` com email correspondente, ele reativa setando `deleted_at=NULL` e `ativo=TRUE`, então chama `supabaseAdmin.auth.resetPasswordForEmail()` para enviar link de acesso. NÃO deve chamar `inviteUserByEmail()` novamente (falharia com ‘email already exists’), e deve tratar erros de `resetPasswordForEmail` de forma graciosa.

Tipo: business-rule

Evidência: - `supabase/functions/create-user-v2/index.ts:374-411` — linhas 384-390: `update` seta `deleted_at=NULL` (reativa); linhas 395-400: chama `resetPasswordForEmail`, não `inviteUserByEmail`; linhas 398-399 tratam o erro graciosamente.

Regressão se: a reativação chama `inviteUserByEmail` em vez de `resetPasswordForEmail`, ou deleta o registro `soft-deleted` em vez de limpar `deleted_at`, ou lança erro de `resetPasswordForEmail` em vez de tratamento gracioso.

Vendor permissions are enumerated whitelist

Enunciado: Usuários vendedores só podem receber permissões do conjunto `SUPPORTED_SELLER_PERMISSION_IDS` (`clientes.*`, `vendas.*`, `relatorios.*`, `contacorrente.*`, `produtos.*`, `comissoes.*`). `create-user-v2` deve rejeitar quaisquer permission IDs fora desse whitelist. Usuários backoffice ignoram essa restrição e podem ter quaisquer permissões.

Tipo: business-rule

Evidência: - `supabase/functions/create-user-v2/index.ts:7-28` — `SUPPORTED_SELLER_PERMISSION_IDS` Set com ~27 strings de permissão; linhas 230-238 `sanitizeAndValidatePermissionIds` checa permissões de vendedor contra o whitelist. - `supabase/functions/create-user-v2/index.ts:346` — permissões de vendedor validadas; backoffice pula validação de whitelist.

Regressão se: o whitelist `SUPPORTED_SELLER_PERMISSION_IDS` é removido ou modificado sem migration, vendedores podem receber permission IDs arbitrários, ou usuários backoffice ficam sujeitos à restrição de whitelist.

Webhook functions protected by x-sweep-secret header

Enunciado (corrigido): A regra é implementada corretamente para a ÚNICA função chamada por `pg_cron` (`ssw-sweep-v1`): valida `x-sweep-secret` contra `SSW_SWEEP_SECRET` e retorna 401 conforme especificado, deployada com `--no-verify-jwt` conforme documentado. A regra NÃO é enforçada por CI/RLS/linter. Não há contraexemplos por haver única função `pg_cron` no repo. Nota de escopo: a documentação menciona `--no-verify-jwt` para o webhook externo `webhook-tiny-atualizacao`, que NÃO é `pg_cron` e não valida esse secret.

Tipo: business-rule

Evidência: - `supabase/functions/ssw-sweep-v1/index.ts:1-27` — comentário indica ‘deploy with `--no-verify-jwt`’; linhas 23-26 checam header `x-sweep-secret` contra env var `SSW_SWEEP_SECRET`, retornam 401 se mismatch.

Regressão se: a função é deployada sem `--no-verify-jwt`, ou o check do header secret é removido, ou o valor do header não é comparado ao valor exato da env var, ou 401 não é retornado em mismatch.

Feature flags use strictly ‘true’ string value convention

Enunciado (corrigido): Feature flags `FEATURE_LOG_CRM`, `FEATURE_LOG_CRM_SSW` e `FEATURE_LOG_CRM_AUTO_FRETE` SÃO checadas pelos helpers em `log-crm-feature-flag.ts` usando `=== 'true'` exato. Porém, `FEATURE_SIMPLES_NACIONAL_LOOKUP` em `create-cliente-v2` e `tiny-enviar-pedido-venda-v1` usa `.toLowerCase() === 'true'`. Webhooks/cron jobs (`ssw-sweep-v1`) retornam 200 quando flags OFF (não 503), o que é intencional para idempotência. Operações best-effort (`FEATURE_SIMPLES_NACIONAL_LOOKUP`) não bloqueiam com 503 quando desligadas, apenas pulam a operação. A regra de “MUST return 503” não é aplicada universalmente — apenas em APIs de leitura/escrita críticas (`ssw-tracking-v1`, `frete-logistica-v1`, `origem-frete-v1`, `transportador-logistica-v1`, `regiao-destino-v1`).

Tipo: business-rule

Evidência: - `supabase/functions/_shared/log-crm-feature-flag.ts:8-18,20-26` — `isLogCrmFeatureEnabled`, `isLogCrmSswFeatureEnabled`: funções puras checando `value === 'true'` exato, convenção documentada em comentários. - `supabase/functions/ssw-sweep-v1/index.ts:29-34` — retorna status 200 com mensagem skipped quando flags OFF, não 503 (webhook deve retornar 200).

Contraexemplo: `supabase/functions/ssw-sweep-v1/index.ts:29-34` (retorna 200 em vez de 503); `supabase/functions/create-cliente-v2/index.ts:331` (usa `.toLowerCase()`, não `=== 'true'`); `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:403` (usa `.toLowerCase()`, não `=== 'true'`).

Regressão se: o check de feature flag usa comparação frouxa (truthy), trata ausência como true, checa a string ‘false’ como flag ON, ou retorna status HTTP errado (503 para user-facing, 200 para webhooks).

arch-decision

Business logic via RPC functions, not inline

Enunciado (corrigido): ALGUMAS Edge Functions delegam operações a RPC functions (`create_user_v2`, `delete_user_v2`, `list_users_v2`), mas muitas outras realizam modificações diretas de tabela (`.insert`, `.update`, `.delete`) sem delegação a RPC. O padrão é aplicado de forma inconsistente ao longo do codebase.

Tipo: arch-decision

Evidência: - supabase/functions/create-user-v2/index.ts:434-463 — chama supabase.rpc('create_user_v2', {...}) na linha 441, depois trata permissões separadamente na linha 472. - supabase/functions/delete-user-v2/index.ts:137-150 — chama supabase.rpc('delete_user_v2', {p_user_id, p_deleted_by}) na linha 142.

Contraexemplo: supabase/functions/condicoes-pagamento-v2/index.ts:325-340 (INSERT direto), 442-447 (UPDATE direto), 498-501 (DELETE direto) — todas as operações CRUD contornam o RPC.

Regressão se: a camada RPC é contornada e a lógica de negócio é codificada diretamente na Edge Function (updates diretos de tabela), ou a assinatura de uma RPC function muda sem atualizar todos os callers.

Contrato — Clientes

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Business rules

Nome obrigatório com tamanho mínimo

Enunciado: O nome (razão social) do cliente é obrigatório e deve conter pelo menos 2 caracteres. Nomes só com espaços são tratados como vazios (via TRIM/trim). A validação é imposta em dois níveis: frontend/API (clientes-v2/index.ts, if (!nome || String(nome).trim().length < 2)) e backend/banco (create_cliente_v2 e update_cliente_v2, IF ... LENGTH(TRIM(p_nome)) < 2 THEN RAISE EXCEPTION). Não há caminho de INSERT/UPDATE direto na tabela cliente fora dessas RPCs (RLS permite INSERT apenas via RPC SECURITY INVOKER).

Tipo: business-rule

Evidência: - supabase/migrations/030_fix_create_cliente_v2_ambiguous_nome.sql:46 — IF p_nome IS NULL OR LENGTH(TRIM(p_nome)) < 2 THEN RAISE EXCEPTION 'Nome deve ter pelo menos 2 caracteres' - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:106-107 — IF p_nome IS NOT NULL AND LENGTH(TRIM(p_nome)) < 2 THEN RAISE EXCEPTION ... - supabase/functions/clientes-v2/index.ts:332,449 — if (!nome || String(nome).trim().length < 2) throw new Error(...)

Regressão se: nomes vazios ou só com espaços são aceitos; nomes com menos de 2 caracteres são armazenados; a validação é removida da RPC ou do frontend.

Exclusão de cliente restrita a backoffice (soft delete)

Enunciado: delete_cliente_v2 faz soft delete (define deleted_at e ref_situacao_id = 'Excluído'). Apenas usuários com tipo = 'backoffice' podem invocar a RPC; usuários vendedor são explicitamente bloqueados.

Tipo: business-rule

Evidência: - supabase/migrations/007_rpc_clientes_v2.sql:422-425 — IF v_user_tipo != 'backoffice' THEN RAISE EXCEPTION 'Apenas usuários backoffice podem excluir clientes' -

supabase/migrations/007_rpc_clientes_v2.sql:428-433 — soft delete: UPDATE cliente SET deleted_at = NOW(), ref_situacao_id = (SELECT ... WHERE nome = 'Excluído')

Regressão se: usuários vendedor conseguem excluir clientes; usa-se hard delete no lugar de soft delete; a exclusão não atualiza ref_situacao_id para 'Excluído'; usuários backoffice não conseguem excluir.

Simples Nacional cacheado com timestamp de consulta

Enunciado: A tabela cliente armazena optante_simples_nacional (boolean NULL) e optante_simples_nacional_consultado_em (timestamp NULL). O valor vem de consulta CONSOPT na ReceitaWS. NULL significa nunca consultado, cliente PF, ou consulta inconclusiva. A revalidação ocorre a cada envio de pedido ao Tiny (ADR-004) **quando a empresa tem dual-mapping**; caso contrário é pulada. Se a revalidação falhar para empresa dual-mapped, o envio é **BLOQUEADO (D3)** em vez de cair no valor persistido. create-cliente-v2 faz lookup best-effort (não bloqueante). A coluna só é modificada por lookups ReceitaWS, nunca por update_cliente_v2.

Tipo: business-rule

Evidência: - supabase/migrations/108_simples_nacional_lookup.sql:10-11 — ADD COLUMN optante_simples_nacional boolean null, optante_simples_nacional_consultado_em timestamp null - supabase/migrations/108_simples_nacional_lookup.sql:13-17 — comentários: true = optante (CONSOPT); null = nunca consultado, PF ou inconclusivo; revalidação no envio Tiny - supabase/functions/clientes-v2/index.ts:185-186 — mapClienteCompleto inclui optanteSimplesNacional e optanteSimplesNacionalConsultadoEm - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:453-481 — empresa com dual-mapping (DP-006) e lookup falho (D2) → BLOQUEIA envio (D3) em vez de aceitar NULL

Regressão se: o valor de Simples Nacional não é cacheado ou o timestamp não é registrado; o lookup não é disparado no envio ao Tiny; NULL é tratado como false em vez de “ainda não verificado”; a coluna é removida.

Condições de pagamento substituídas por completo no update

Enunciado: Quando p_condicoes_pagamento_ids é passado a update_cliente_v2: (1) DELETE de todas as linhas existentes de condicoes_cliente do cliente, (2) INSERT das novas linhas do array. É substituição atômica all-or-nothing, não aditiva. **Ressalva de integridade:** existem dois caminhos que burlam o padrão e não devem ser usados/reintroduzidos — a edge function update_cliente_v2 chama a RPC sem passar o parâmetro (pulando o DELETE+INSERT), e add_condicoes_disponiveis apenas adiciona ao array sem deletar. O frontend em uso (clientes-v2) sempre passa p_condicoes_pagamento_ids.

Tipo: business-rule

Evidência: - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:213-221 — IF p_condicoes_pagamento_ids IS NOT NULL THEN DELETE all, then INSERT unnest(array) - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:175-184 — mesmo padrão: DELETE, depois INSERT das linhas - supabase/functions/clientes-v2/index.ts:501-517 — frontend extrai o array e passa como p_condicoes_pagamento_ids - Contraexemplo: supabase/functions/update_cliente_v2/index.ts:170-184 — chama a RPC sem p_condicoes_pagamento_ids, pulando o DELETE+INSERT

Regressão se: novas condições são adicionadas às existentes em vez de substituí-las; permitem-se updates parciais; updates só-de-array deixam linhas órfãs; algum caminho acessa a RPC sem o parâmetro ou usa operação aditiva.

Validação de tipo de pessoa (Física/Jurídica)

Enunciado: `cliente.ref_tipo_pessoa_id_FK` (camelCase no nome da coluna) é protegido por: (1) constraint FOREIGN KEY no schema (`schema_baseline.sql:881`) que rejeita IDs inválidos em INSERT e UPDATE; (2) validação PL/pgSQL explícita em `update_cliente_v2` (checa existência antes do update); (3) **nenhuma** validação explícita em `create_cliente_v2`, que depende só da constraint FK. NULL é permitido em todos os casos (tipo não determinado).

Tipo: business-rule

Evidência: - `supabase/migrations/104_unify_update_cliente_v2_with_status.sql:110-117` — IF `p_ref_tipo_pessoa_id_fk` IS NOT NULL THEN check SELECT 1 FROM `ref_tipo_pessoa` WHERE `id` = `p_ref_tipo_pessoa_id_fk` - `supabase/migrations/030_fix_create_cliente_v2_ambiguous_nome.sql:68` — INSERT usa nome citado "`ref_tipo_pessoa_id_FK`" (sufixo FK preservado) - `supabase/functions/clientes-v2/index.ts:359-372` — frontend aceita `tipoPessoa` como objeto/string/number, resolve para `p_ref_tipo_pessoa_id_fk` - Contraexemplo: `supabase/migrations/129_fix_create_cliente_missing_fields.sql:107` — `create_cliente_v2` aceita o parâmetro mas não valida; confia só na constraint FK

Regressão se: IDs inválidos de `ref_tipo_pessoa` são aceitos; a validação é removida; o tipo de pessoa é armazenado como string em vez de FK; a constraint FK não é imposta.

Empresa de faturamento opcional (CASE WHEN)

Enunciado: `cliente.empresaFaturamento` é nullable. No UPDATE, só é definido se `p_empresa_faturamento_id` IS NOT NULL; caso contrário preserva o valor atual (CASE WHEN). A integridade é imposta por constraint FOREIGN KEY (ON DELETE SET NULL, referencia `ref_empresas_subsidiarias`), mas — diferente de `ref_tipo_pessoa_id_fk` — **não** há validação explícita em nível de aplicação (IF EXISTS + RAISE EXCEPTION) em `update_cliente_v2`; apenas a validação em nível de banco (FK).

Tipo: business-rule

Evidência: - `supabase/migrations/104_unify_update_cliente_v2_with_status.sql:146` — "`empresaFaturamento`" = CASE WHEN `p_empresa_faturamento_id` IS NOT NULL THEN `p_empresa_faturamento_id` ELSE `c."empresaFaturamento"` END - `supabase/functions/clientes-v2/index.ts:374-386` — frontend extrai `empresaFaturamento` como objeto ou ID numérico, resolve para `p_empresa_faturamento_id` (ou NULL)

Regressão se: empresa vira NOT NULL; adiciona-se validação de integridade referencial em app quando não deveria mudar o mecanismo; CASE WHEN é trocado por COALESCE; uma empresa default é atribuída quando não especificada.

Campos numéricos (desconto/pedido mínimo) com default 0

Enunciado: Para `desconto_financeiro` e `pedido_minimo` há dois mecanismos complementares. No INSERT: o frontend converte para 0 (se null) antes do SQL, e o SQL ainda protege com `COALESCE(p_value, 0)`. No UPDATE: valores null são enviados ao backend e `COALESCE(p_value, valor_atual)` preserva o valor existente. Ou seja: INSERT sempre garante 0; UPDATE preserva.

Tipo: business-rule

Evidência: - supabase/migrations/030_fix_create_cliente_v2_ambiguous_nome.sql:95-96 — INSERT: COALESCE(p_desconto_financeiro, 0), COALESCE(p_pedido_minimo, 0) - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:139-140 — UPDATE: desconto_financeiro = COALESCE(p_desconto_financeiro, c.desconto_financeiro) - supabase/functions/clientes-v2/index.ts:410-411 — frontend (INSERT): p_desconto_financeiro: body.descontoFinanceiro ?? body.desconto_financeiro ?? 0

Regressão se: NULL é armazenado em vez de 0; COALESCE é removido do INSERT; UPDATE usa NULL direto em vez de COALESCE com o valor existente; comparações numéricas falham em campos NULL.

Invariants

Status de aprovação restrito a três valores

Enunciado: cliente.status_aprovacao deve ser um de: ‘aprovado’, ‘pendente’ ou ‘rejeitado’. A constraint CHECK impõe corretamente esses valores. **Débito conhecido:** a lógica de workflow está quebrada — create_cliente_v2 (migrations 129/131) sempre atribui ‘aprovado’ independentemente do tipo de usuário, quando deveria atribuir ‘pendente’ para vendedor e ‘aprovado’ para backoffice (como estava correto na migration 105/081).

Tipo: invariant

Evidência: - supabase/migrations/001_fix_campos_obrigatorios.sql:67 — CHECK (status_aprovacao IN ('aprovado', 'pendente', 'rejeitado')) DEFAULT 'pendente' - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:120-127 — IF v_status_aprovacao NOT IN ('aprovado', 'pendente', 'rejeitado') THEN RAISE EXCEPTION - Contraexemplo: supabase/migrations/131_session_20260601_cliente_fixes.sql:29-38 — lógica sempre resulta em ‘aprovado’, violando o controle de workflow por tipo de usuário

Regressão se: outros valores são inseridos em status_aprovacao; a constraint CHECK é removida; transições de status não impõem valores válidos do enum.

UPSERT em contato/endereço preserva dados existentes quando novo valor é NULL

Enunciado: No ON CONFLICT DO UPDATE de cliente_contato e cliente_endereço, cada campo usa COALESCE(NULLIF(new_value), tabela.campo_existente) (referência à tabela, **não** a EXCLUDED) para impedir que updates com NULL apaguem dados existentes. O padrão está imposto no HEAD: em create_cliente_v2 desde a migration 131 e em update_cliente_v2 desde a migration 140. **Histórico:** a migration 131 (backport de prod, 2026-06-22) reintroduziu o padrão bugado EXCLUDED em update_cliente_v2, causando perda de dados em ~89 clientes, até ser corrigido pela migration 140 (2026-06-25).

Tipo: invariant

Evidência: - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:133-138 — telefone = COALESCE(NULLIF(trim(p_telefone), ''), cliente_contato.telefone) (usa valor da tabela, não EXCLUDED) - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:1-8 — comentário de bug: migrations anteriores usavam EXCLUDED.<col> (== NULL quando campo não enviado), causando perda de dados - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:165-172 — cliente_endereço segue o mesmo padrão: cep = COALESCE(...cliente_endereço.cep)

Regressão se: o UPSERT volta ao fallback EXCLUDED.column; o COALESCE é removido; campos são definidos direto dos parâmetros sem NULL-safety; parâmetros NULL apagam valores existentes.

Arch decisions

Grupo/Rede usa grupo_id (UUID FK) e grupo_rede (nome texto)

Enunciado: A tabela cliente possui ambos os campos: grupo_id (UUID) e grupo_rede (TEXT legado). A implementação é **inconsistente**: (1) no UPDATE, o RPC prioriza grupo_id via CASE WHEN quando definido; (2) no CREATE, ambos são inseridos diretamente sem lógica de priorização; (3) não há constraint FOREIGN KEY em grupo_id nem validação de integridade referencial; (4) o frontend tenta anular grupo_rede ao enviar grupo_id, mas create_cliente_v2 não respeita esse padrão. get_cliente_completo_v2 inclui grupo_rede_nome via JOIN para display.

Tipo: arch-decision

Evidência: - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:136-137 — grupo_rede = COALESCE(NULLIF(TRIM(p_grupo_rede), ''), c_grupo_rede), grupo_id = CASE WHEN p_grupo_id IS NOT NULL THEN p_grupo_id ELSE c_grupo_id END - supabase/functions/clientes-v2/index.ts:334-357 — frontend resolve grupoRede/grupoId para UUID; verifica formato UUID; fallback por nome via grupos_redes; envia p_grupo_id (ou NULL) - supabase/migrations/141_get_cliente_completo_grupo_nome.sql:52 — get_cliente_completo_v2 inclui grupo_rede_nome via JOIN: SELECT gr.nome FROM grupos_redes gr WHERE gr.id = c_grupo_id - Contraexemplo: supabase/migrations/129_fix_create_cliente_missing_fields.sql:85-112 — create_cliente_v2 insere grupo_rede e grupo_id sem CASE WHEN ou validação

Regressão se: grupo_id é removido ou não populado; grupo_rede é tratado como fonte da verdade em vez de campo de lookup; o CASE WHEN vira COALESCE nos dois campos (apagaria grupo_id quando NULL é passado).

Contato e endereço como relacionamentos singleton opcionais

Enunciado: Cada cliente tem no máximo um cliente_contato e um cliente_endereço (identificados por cliente_id PK). São criados/atualizados via UPSERT na mesma RPC do update do registro principal. Se nenhum campo de contato/endereço é fornecido, as tabelas auxiliares não são tocadas (sem DELETE; NULLs preservados). Enforcement: PRIMARY KEY (cliente_id) força uma linha por cliente; INSERT ... ON CONFLICT DO UPDATE; condicional IF pula tabelas auxiliares se nada é fornecido; COALESCE com valores existentes (fix da migration 140); não existe rota de DELETE direto.

Tipo: arch-decision

Evidência: - supabase/migrations/030_fix_create_cliente_v2_ambiguous_nome.sql:109-119 — se campos de contato fornecidos: INSERT ... ON CONFLICT DO UPDATE; senão pula por completo - supabase/migrations/104_unify_update_cliente_v2_with_status.sql:151-177 — IF p_telefone OR p_email OR ... THEN INSERT ON CONFLICT; senão nenhuma operação em cliente_contato - supabase/functions/clientes-v2/index.ts:109-118 — frontend mapeia objetos aninhados contato/endereco em parâmetros flat passados à RPC

Regressão se: cliente passa a ter múltiplos registros de contato ou endereço; DELETE é chamado quando campos são NULL; tabelas auxiliares não são atualizadas na mesma transação; endpoints separados passam a gerenciar contato/endereço.

Segmento via FK opcional (nullable)

Enunciado: `cliente.segmento_id` é BIGINT FK nullable para `segmento_cliente.id`. Um cliente pode não ter segmento (NULL). `get_cliente_completo_v2` inclui `segmento_nome` via LEFT JOIN apenas para display (retorna NULL se não houver segmento).

Tipo: arch-decision

Evidência: - supabase/migrations/001_fix_campos_obrigatorios.sql:79 — campo `segmento_id` adicionado à tabela `cliente` - supabase/migrations/141_get_cliente_completo_grupo_nome.sql:99-101 — LEFT JOIN `segmento_cliente sc ON sc.id = c.segmento_id AND sc.deleted_at IS NULL` - supabase/functions/clientes-v2/index.ts:189-190 — `mapClienteCompleto` mapeia `segmento_nome` do resultado da RPC

Regressão se: `segmento_id` vira NOT NULL; constraint FK torna segmento obrigatório; `segmento_nome` não é incluído em `get_cliente_completo_v2`; segmento passa a ser armazenado como texto em vez de ID.

Mappers do frontend usam cadeias de fallback

Enunciado: `mapClienteCompleto` e `mapClienteListItem` usam cadeias ?? (nullish coalescing) para lidar com variações de nome de campo (snake_case do DB, camelCase da API, nomes legados). Ex.: `grupo_rede_nome ?? grupo_rede ?? grupoRede ?? ''`. Permite compatibilidade retrógrada, mas pode esconder mismatches (o bug histórico de `grupo_rede_nome` ausente fazia o campo aparecer vazio ao usuário até a migration 141).

Tipo: arch-decision

Evidência: - supabase/functions/clientes-v2/index.ts:103-127 — `mapClienteListItem`: `grupoRede: row.grupo_rede_nome ?? row.grupo_rede ?? row.grupoRede ?? ''` - supabase/functions/clientes-v2/index.ts:160-246 — `mapClienteCompleto`: cadeias de fallback extensas para todos os campos - supabase/migrations/141_get_cliente_completo_grupo_nome.sql:1-5 — bug: versão anterior não retornava `grupo_rede_nome`, mapper caía em `grupo_rede` (texto), campo aparecia vazio

Regressão se: cadeias de fallback são removidas; apenas um nome de campo é checado; variações de nome não são antecipadas; mappers não extraem objetos aninhados como `contato.email`.

Contrato — Vendas & Emissão ao Tiny (ERP)

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Invariantes

Pedido enviado deve ter ID Tiny

Enunciado: Quando um pedido é enviado ao Tiny via edge `tiny-enviar-pedido-venda-v1`, o campo `id_tiny` é persistido APENAS após receber um ID válido do Tiny. O status é alterado para ‘Em aberto’ apenas quando a resposta HTTP da edge function é recebida com sucesso no frontend. Porém, o webhook do Tiny pode alterar o status posteriormente sem validar se a confirmação HTTP inicial foi recebida, criando dessincronização possível entre confirmação do usuário e estado real do banco.

- **Tipo:** invariant
- **Evidência:**

- `src/components/SalesPage.tsx:1102-1107` — GUARD anti falso-sucesso: sem ID Tiny retornado, lança erro e não muda status
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:706-708` — Valida se `registro.id` existe; se não, lança erro ‘Tiny nao retornou id do pedido’
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:714-728` — Persiste `id_tiny` e `numero_pedido` apenas após confirmação do Tiny
 - **Regressão se:** O sistema aceita sucesso sem ID Tiny retornado, ou tenta reenviar um pedido já com `erpPedidoId` preenchido.
-

Vendedor deve ter idtiny cadastrado

Enunciado: Toda venda DEVE estar associada a um vendedor (`vendedor_uuid`). O vendedor DEVE ter registro em `dados_vendedor` com `idtiny` preenchido. Sem `idtiny`, envio ao Tiny é bloqueado com erro ‘Vendedor sem idtiny’.

- **Tipo:** invariant
 - **Evidência:**
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:233-266` — Valida `vendedor_uuid`; busca `dados_vendedor`; verifica `idtiny`
 - `src/components/SalesPage.tsx:306-308` — UI mostra erro: ‘Vendedor não configurado. Verifique o cadastro do vendedor.’
 - **Regressão se:** Sistema permite venda sem vendedor ou com vendedor sem `idtiny`; falha ao validar `vendedor_uuid`.
-

Regras de negócio

Natureza de operação obrigatória e mapeada

Enunciado: A `natureza_operacao_id` DO PEDIDO é validada e obrigatória apenas via edge function `tiny-enviar-pedido-venda-v1` (mapeamento 1:1 via `tiny_empresa_natureza_operacao` para uma natureza Tiny `tiny_valor` válida para a empresa). Contudo, outras rotas de emissão (`emitirpedido`, `emitir-pedido-sem-vendedor`, `TinyEmitirAPI`) NÃO validam o mapeamento em `tiny_empresa_natureza_operacao`, permitindo envio ao Tiny sem garantir que a natureza Tiny será corretamente informada. O Tiny acabaria usando sua operação padrão, não a natureza pretendida.

- **Tipo:** business-rule
 - **Evidência:**
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:300-320` — Resolve `natureza_operacao_id` do pedido; lança erro se vazio
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:323-340` — Busca mapeamento em `tiny_empresa_natureza_operacao`; erro se não encontrado
 - `src/components/SalesPage.tsx:300-302` — UI mostra erro resumido: ‘Natureza de operação não mapeada para esta empresa’
 - **Contraexemplo:** `supabase/functions/emitirpedido/index.ts:93-176` — busca `pedido.natureza_operacao`, consulta `natureza_operacao.tem_comissao` para comissão, monta payload Tiny (linhas 142-176) sem `id_natureza_operacao`, ignorando `tiny_empresa_natureza_operacao`. Permite envio sem mapeamento validado.
 - **Regressão se:** Permite envio sem natureza ou com natureza não-mapeada na empresa; muda formato ou localização do mapeamento sem atualizar edge function.
-

Deleção de pedido é soft delete (deleted_at)

Enunciado: Soft-delete com deleted_at está implementado (RPC delete_pedido_venda_v2). Leitura via RPC (list_pedido_venda_v2, get_pedido_venda_v2) filtra com deleted_at IS NULL. PORÉM a aplicação é inconsistente: (1) tiny-enviar valida deleted_at em código, não SQL; (2) tiny-verificar não valida deleted_at — pedidos deletados podem ser verificados no Tiny; (3) comissoes-v2 não filtra deleted_at — pedidos deletados com status Faturado podem gerar comissão. A regra é parcialmente imposta.

- **Tipo:** business-rule
 - **Evidência:**
 - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:224-226 — Valida if (!pedido || pedido.deleted_at) para rejeitar pedidos deletados
 - src/services/api.ts:4183-4192 — Edge function pedido-venda-v2 DELETE usa soft delete
 - **Contraexemplo:** supabase/functions/tiny-verificar-pedido-v1/index.ts:62 — query não filtra deleted_at; um pedido com deleted_at preenchido pode ser recuperado e verificado no Tiny, violando a regra “não aparecem em listas e não podem ser reenviados”.
 - **Regressão se:** Implementa hard delete ou permite reenviamento de pedidos com deleted_at preenchido; queries não filtram deleted_at.
-

Empresa deve ter chave API Tiny

Enunciado: A validação de empresa (ativo=true, deleted_at=null) e chave_api (token Tiny) é obrigatória APENAS no fluxo Tiny ERP v1 moderno. Funções Edge legadas (emitirpedido, TinyEmitirAPI) ainda existem e permitem envio ao Tiny sem validação de estado da empresa ou preenchimento da chave_api.

- **Tipo:** business-rule
 - **Evidência:**
 - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:273-292 — Busca empresa por ID; valida ativo=true, deleted_at=null; verifica chave_api não vazia
 - src/components/SalesPage.tsx:294-296 — UI mostra erro: ‘Empresa sem chave API do Tiny configurada’
 - **Contraexemplo:** supabase/functions/emitirpedido/index.ts:82-91 — função aceita API_Code direto do client e envia ao Tiny sem validar empresa em BD; supabase/functions/TinyEmitirAPI/index.ts:19-30 — função requer apenas token e pedido, sem validação de empresa ou chave_api no BD.
 - **Regressão se:** Usa empresa inativa ou com chave_api vazia; não valida status da empresa antes de enviar.
-

Cliente deve ter CPF/CNPJ válido

Enunciado: Cliente DEVE ter cpf_cnpj com exatamente 11 (CPF) ou 14 (CNPJ) dígitos após remoção de caracteres não-numéricos. Sem validação ou com tamanho inválido, envio ao Tiny é bloqueado.

- **Tipo:** business-rule
 - **Evidência:**
 - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:365-368 — Normaliza CPF/CNPJ, valida comprimento (11 ou 14)
 - src/components/SalesPage.tsx:297-299 — UI mostra erro: ‘CPF/CNPJ do cliente inválido’
 - **Regressão se:** Permite cliente com CPF/CNPJ vazio, inválido ou com tamanho errado; pula validação de dígitos.
-

Pedido deve ter itens

Enunciado: Pedido DEVE ter pelo menos 1 item (`pedido_venda_produtos`). No fluxo primário ativo (frontend `SalesPage` → `tiny-enviar-pedido-venda-v1`), a validação `itens.length > 0` é imposta em ambas as camadas. Existem funções alternativas (`emitir-pedido-sem-vendedor`, `emitirpedido`) que também validam, mas `TinyEmitirAPI` é um proxy genérico sem validação de negócio, não é chamado do frontend e não implementa essa regra.

- **Tipo:** business-rule
- **Evidência:**
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:496-498` — Valida `itensDb.length > 0`; erro se vazio
 - `src/components/SalesPage.tsx:1083-1086` — Frontend: verifica itens antes de chamar `enviarAoERP`
- **Contraexemplo:** `supabase/functions/TinyEmitirAPI/index.ts:19-29` — função aceita `{ token, pedido }` e passa direto ao Tiny sem validar se `itens.length > 0`; não há verificação de itens na função.
- **Regressão se:** Permite envio de pedido vazio; remove validação de itens; não sincroniza validação entre frontend e edge.

Número de pedido no Tiny é distinto do número local

Enunciado: `numero` (formato PV-YYYY-XXXX) é gerado no client/frontend durante criação. `numero_pedido` vem do Tiny ERP após envio bem-sucedido. Ambos são persistidos: `numero` identifica no sistema local, `numero_pedido` (Tiny) identifica no ERP. Duplicar pedido cria novo `numero` mas no estado Rascunho, sem `numero_pedido` (pois não foi enviado).

Nota: verificação desta regra ficou parcial (`verdict partial`; verificação de detalhes falhou). Tratar como não totalmente confirmada — cotejar com o código antes de depender dela.

- **Tipo:** business-rule
- **Evidência:**
 - `src/components/SalesPage.tsx:1160-1173` — `handleDuplicarPedido` copia pedido original mas descarta `id`, `numero_original`, `integracaoERP`, `createdAt`, `updatedAt`
 - `src/components/SalesPage.tsx:1908` — novo `numero` gerado: `PV-2025-${String(Date.now()).substring(7, 11)}`
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:717-718` — Persiste `numero_pedido` retornado pelo Tiny no banco
- **Regressão se:** Trata `numero` local como identificador único no Tiny; reutiliza `numero` em duplicação; não persiste `numero_pedido` do Tiny.

Contrato — Fiscal: Simples Nacional & Natureza de Operação

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Este contrato consolida as regras verificadas do domínio fiscal (Simples Nacional + Natureza de Operação). Quando uma regra foi verificada como `partial`, o enunciado abaixo reflete o `corrected_statement` — a versão fiel ao que o código realmente faz.

Business rules

Consulta ReceitaWS obrigatória apenas para PJ novo ou revalidação em envio

Enunciado (corrigido — verdict: partial): A regra é implementada parcialmente: (1) criar cliente com CNPJ via `create-cliente-v2` consulta ReceitaWS se a flag `FEATURE_SIMPLES_NACIONAL_LOOKUP` estiver ativada (best-effort, não bloqueia 201, apenas para PJ); (2) enviar pedido ao Tiny via `tiny-enviar-pedido-venda-v1` revalida Simples Nacional sempre que flag ligada e cliente é PJ, sem cache, independentemente do tempo desde a última consulta. **Porém**, a rota ativa de criação de cliente (`clientes-v2` POST, usada pelo frontend) **não** implementa revalidação de Simples Nacional. As funções legadas `emitirpedido`, `emitir-pedido-sem-vendedor` e `TinyEmitirAPI` também não implementam a revalidação, mas estão marcadas como código morto na auditoria de 2026-06-01.

Tipo: business-rule

Evidência: - docs/decisions/adr/ADR-004-revalidacao-simples-por-pedido.md:24-25 — decisão explícita de revalidar a cada envio sem cache (empresa pode ser excluída do Simples a qualquer momento; risco tributário real). - supabase/functions/create-cliente-v2/index.ts:329-362 — feature flag consultada na criação (best-effort, apenas PJ). - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:402-484 — revalidação incondicional antes de envio ao Tiny quando flag ligada e cliente é PJ. - supabase/functions/clientes-v2/index.ts:432 — contraexemplo: POST ativo do frontend chama RPC `create_cliente_v2` sem revalidação de Simples.

Regressão se: Sistema deixasse de revalidar ReceitaWS no envio de pedido (p.ex. usando cache de 30 dias) — aceitaria emitir NF com regime tributário incorreto até 30 dias após cliente sair do Simples (risco real conforme decisão do cliente em 2026-04-22).

Seleção de `tiny_valor` segue hierarquia determinística por `optante_simples_nacional`

Enunciado (verdict: confirmed): Função `resolveNaturezaTiny` aplica a regra: (1) empresa sem dual-mapping → `tiny_valor`, fallback `no_dual_company`; (2) mapeamento sem dual (`tinyValorSimples===null`) → `tiny_valor`, fallback `no_dual`; (3) `optante===null` → `tiny_valor`, fallback `null_optante`; (4) `optante===true` → `tiny_valor_simples` (ou `tiny_valor` se vazio), fallback `none`; (5) `optante===false` → `tiny_valor`, fallback `none`. A função é pura, determinística e testável; o resultado é aplicado sem modificação ao payload Tiny e auditado no evento `natureza.resolvida`.

Tipo: business-rule

Evidência: - supabase/functions/_shared/natureza-resolver.ts:52-83 — função pura com 5 casos e razões de fallback. - docs/specs/SPEC.md:207-244 — CA-007 detalha os 5 cenários (A-E), incluindo short-circuit DP-006. - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:646-661 — função chamada e resultado logado com evento `natureza.resolvida`.

Regressão se: A lógica de seleção for alterada (ex.: usar `tiny_valor_simples` quando `optante=false`, ou não respeitar `companyHasDualMapping`) — pedidos seriam enviados com natureza incorreta para Simples ou não-Simples conforme o desvio.

D3: Envio ao Tiny é BLOQUEADO se lookup ReceitaWS falha para empresa com dual-mapping

Enunciado (verdict: confirmed): Quando a empresa possui pelo menos um mapeamento com `tiny_valor_simples` preenchido (tem dual-mapping) e ReceitaWS falha (timeout, rate-limit, missing_field, network_error), o envio ao Tiny é interrompido com erro `REGIME_LOOKUP_FAILED`. A falha é registrada na tabela `regime_lookup_falha` (best-effort) para auditoria. Não há caminho de código que permita envio ao Tiny nessa condição (com flag ligada, cliente PJ, CNPJ válido).

Tipo: business-rule

Evidência: - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:452-481` — se `lookup.status != 'ok'`, insere em `regime_lookup_falha`, loga `regime_lookup.bloqueio` e lança `REGIME_LOOKUP_FAILED`. - `docs/specs/SPEC.md:112-130` — Fluxo F-2 descreve o bloqueio (RF-003). - `src/components/SalesPage.tsx:1124-1132` — frontend detecta `REGIME_LOOKUP_FAILED` e oferece botão “Tentar novamente”.

Regressão se: O bloqueio D3 fosse removido (e o fallback usasse `optante=null` quando lookup falha para empresa com dual) — pedidos seriam emitidos com natureza potencialmente incorreta quando ReceitaWS está indisponível, violando o risco central mitigado pela feature.

Persistência de `optante_simples_nacional` após consulta bem-sucedida

Enunciado (corrigido — verdict: partial): Quando ReceitaWS retorna `status='ok'` com `optante` booleano, o sistema persiste `cliente.optante_simples_nacional` e `cliente.optante_simples_nacional_consultado_em`. Em `create-cliente-v2` a persistência é sempre best-effort após lookup bem-sucedido, sem bloqueio (CA-004). Em `tiny-enviar-pedido-venda`, a persistência ocorre após sucesso de ReceitaWS **apenas quando a empresa tem dual-mapping habilitado** (DP-006); se a empresa não tem dual-mapping, o lookup é integralmente pulado, a persistência não ocorre e o valor persistido (ou null) é usado para resolver a natureza.

Tipo: business-rule

Evidência: - `supabase/functions/create-cliente-v2/index.ts:329-362` — após lookup ok, atualiza `optante_simples_nacional + consultado_em` (não bloqueante). - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:433-451` — se `lookup.status='ok'`, persiste `optante` e `consultado_em` antes de `resolveNaturezaTiny`. - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:408-432` — contraexemplo: com `companyHasDualMapping=false`, o bloco de lookup (433-482) nunca executa; `optante` não é atualizado antes de `resolveNaturezaTiny`. - `src/types/customer.ts:160-163` — tipo com `optanteSimplesNacional?: boolean | null` e `optanteSimplesNacionalConsultadoEm?: string | null`.

Regressão se: A persistência for contornada e o resultado do lookup não for salvo — lookups subsequentes se repetiriam a cada envio, sem fallback para quando ReceitaWS falha, removendo resiliência a falhas temporais.

Resposta bruta da ReceitaWS NUNCA é persistida; só `optante` boolean + timestamp

Enunciado (verdict: confirmed): As Edge Functions extraem apenas `simples.optante` da resposta ReceitaWS e persistem esse boolean em `cliente.optante_simples_nacional` mais o timestamp. A resposta completa (contendo PII de sócios, endereço completo, quadro societário) é descartada e nunca armazenada no banco. A tabela `regime_lookup_falha` loga apenas metadados de falha. Apenas `create-cliente-v2` e `tiny-enviar-pedido-venda-v1` importam `consultarSimplesNacional`, ambas seguindo o padrão.

Tipo: business-rule

Evidência: - supabase/functions/_shared/receitaws-client.ts:232-254 — body parseado; apenas simples.optante extraído; sem persistência do body completo. - supabase/functions/create-client-v2/index.ts:340-347 — apenas optante e consultadoEm escritos na tabela cliente. - docs/specs/SPEC.md:60,84 — Anti-SPEC §6 e RNF-003 proíbem persistir resposta bruta (PII de sócios).

Regressão se: A resposta completa da ReceitaWS fosse persistida — o banco conteria PII não solicitada (dados de sócios, endereços completos), violando minimização de dados e criando risco de compliance/responsabilidade sem valor de negócio.

Invariants

Mapeamento Natureza Operação possui dual-ID: tiny_valor + tiny_valor_simples

Enunciado (corrigido — verdict: partial): A tabela tiny_empresa_natureza_operacao possui exatamente duas colunas relacionadas à natureza: tiny_valor (obrigatório, pré-F-001) e tiny_valor_simples (nullable, pós-F-001). Quando tiny_valor_simples é preenchido, determina que a empresa emite com naturezas distintas para optantes do Simples Nacional. A lógica de seleção está corretamente implementada em resolveNaturezaTiny. **Entretanto**, a invariante CB-003 (rejeitar tinyValor vazio com tinyValorSimples preenchido) deveria ser enforçada com erro 400 no endpoint POST /tiny-empresa-natureza-operacao-v2, mas o código atual apenas faz soft-delete sem essa validação, permitindo contorno da regra via API.

Tipo: invariant

Evidência: - docs/decisions/adr/ADR-003-modelagem-dual-id-natureza-operacao.md:23-31 — decisão de adicionar coluna única nullable tiny_valor_simples (migration 108). - src/types/tinyNaturezaOperacao.ts:1-9 — contrato de tipo: tinyValor (string, required) e tinyValorSimples (string | null). - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:322-350 — query busca tiny_valor e tiny_valor_simples da tabela de mapeamento. - supabase/functions/tiny-empresa-natureza-operacao-v2/index.ts:160-178 — contraexemplo: tinyValor vazio faz soft-delete sem verificar tinyValorSimples; POST {tinyValor:'', tinyValorSimples:'2002'} aceito em vez de rejeitar com NATUREZA_MAPEAMENTO_INCOMPLETO (CONTRACTS.md §2).

Regressão se: tiny_valor_simples for removido ou o sistema deixar de consultar seu valor ao enviar pedido — empresas que precisam de naturezas distintas para Simples vs não-Simples emitiriam sempre a mesma natureza, gerando NF classificada erroneamente para um grupo de clientes.

Cliente PF (CPF) nunca é consultado em ReceitaWS; optante permanece null

Enunciado (verdict: confirmed): O sistema identifica PF por CNPJ/CPF com 11 dígitos (PJ = 14 dígitos). Para PF, o campo optante_simples_nacional permanece null sem tentar consulta, e a natureza_operacao é sempre tiny_valor (ignora dual-mapping). Confirmado em criação (isPJ = length === 14) e em envio (tipoPessoa === 'J' && cpfCnpj.length === 14); resolveNaturezaTiny trata null como fallback para tiny_valor.

Tipo: invariant

Evidência: - supabase/functions/create-client-v2/index.ts:333-335 — isPJ = cpfCnpjSanitized.length === 14; lookup só prossegue se isPJ. - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:408 — tipoPessoa === 'J' && cpfCnpj.length === 14 exigidos para entrar no bloco de revalidação. - docs/specs/SPEC.md:30-31 — RF-001: “Só se aplica a CNPJ” (PJ); para CPF o campo fica permanentemente null.

Regressão se: Clientes PF fossem consultados em ReceitaWS — o sistema tentaria classificá-los por status Simples (que PF não possui — só PJ pode ser optante), resultando em dados inválidos e possíveis escolhas de natureza incorretas.

Timeout da requisição ReceitaWS é estritamente 5 segundos (RNF-001)

Enunciado (verdict: confirmed): Todas as chamadas `consultarSimplesNacional` usam `AbortController` com timeout de 5 segundos. Se o fetch não completar em 5s, a requisição é abortada e retorna `status='failed'`, `reason='timeout'`. O timeout aplica-se por tentativa (não cumulativo entre retries) — cada `attemptLookup()` cria seu próprio controller e timer. Ambos os callers chamam sem especificar `timeoutMs`, usando o default de 5000ms.

Tipo: invariant

Evidência: - `supabase/functions/_shared/receitaws-client.ts:8-9` — `DEFAULT_TIMEOUT_MS = 5_000`. - `supabase/functions/_shared/receitaws-client.ts:124-126` — `AbortController` criado com timeout; `AbortError` capturado na linha 273. - `docs/specs/SPEC.md:73-76` — RNF-001 especifica timeout de 5s (Fluxos F-1 e F-2).

Regressão se: O timeout aumentasse para 30s, a criação de cliente bloquearia mais tempo e prejudicaria a UX; se removido, chamadas lentas ou travadas cascatariam em timeouts da Edge Function (60s total no Supabase). 5s é o compromisso acordado.

CNPJ sempre mascarado em logs (RNF-003)

Enunciado (corrigido — verdict: partial): A função `maskCnpj` mascara CNPJ para logs da ReceitaWS (preservando os 3 primeiros e 2 últimos dígitos; `'12345678901234' → '123*****91'`), e todos os eventos de log `receitaws.lookup` usam o campo `cnpjMasked`, nunca o CNPJ em texto claro. **Porém**, o mascaramento NÃO é enforcado em respostas HTTP da API: `/romaneio-logistica-v1`, `/empresas-v2`, `/get-cliente-v2` e `/transportador-logistica-v1` retornam CNPJ sem máscara em suas respostas JSON. O requisito RNF-003 aplica-se apenas a logs (“não aparece em logs”), não a respostas de API.

Tipo: invariant

Evidência: - `supabase/functions/_shared/receitaws-client.ts:40-44` — `maskCnpj` preserva 3 primeiros + 2 últimos; retorna `'***'` se `input < 5 chars`. - `supabase/functions/_shared/receitaws-client.ts:97,109,259` — `cnpjMasked` computado cedo e usado em todos os `emitLog`. - `docs/specs/SPEC.md:81-84` — RNF-003 Segurança: CNPJ não aparece em logs em texto claro. - `supabase/functions/romaneio-logistica-v1/index.ts:219,223` — contraexemplo: retorna `empresaCnpj` e `transportadorCnpj` em texto claro na resposta JSON.

Regressão se: O CNPJ bruto fosse logado — exposição de PII violaria regulações de proteção de dados (LGPD, compliance) e vazaria informação comercial sensível (quais empresas estão sendo consultadas quanto ao Simples).

Arch decisions

DP-006: Empresa sem nenhum dual-mapping pula consulta ReceitaWS (short-circuit)

Enunciado (corrigido — verdict: partial): Em RF-003 (envio de pedido ao Tiny), antes de chamar ReceitaWS, o sistema faz COUNT com head:true na tabela tiny_empresa_natureza_operacao filtrando empresa_id, ativo=true, deleted_at IS NULL e NOT tiny_valor_simples IS NULL. Se count=0 (empresa inteira sem dual-mapping), pula o lookup ReceitaWS e usa tiny_valor direto com fallback no_dual_company. Se o COUNT falha (dualCountError), o fail-safe mantém companyHasDualMapping=true (comportamento pré-DP-006). **Escopo:** DP-006 é uma otimização específica de RF-003 (envio de pedido), não de RF-001/RF-002 (criação de cliente) — create-cliente-v2 chama ReceitaWS sem COUNT probe por estar fora do escopo.

Tipo: arch-decision

Evidência: - supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:408-430 — COUNT EXACT com head:true; se dualCountError → companyHasDualMapping=true (fail-safe); senão baseado em count > 0. - docs/specs/SPEC.md:44-45 — RF-003 otimização (DP-006, 2026-04-24): pular ReceitaWS quando não há dual-mapping (resultado nunca muda a escolha de natureza; poupa latência e cota). - supabase/functions/_shared/natureza-resolver.ts:60-61 — trata companyHasDualMapping=false retornando fallback no_dual_company.

Regressão se: O short-circuit fosse contornado (sempre chamando ReceitaWS independentemente de companyHasDualMapping) — chamadas de API desnecessárias aumentariam latência e consumo de cota sem alterar nenhuma decisão, desperdiçando recursos e ampliando o risco de dependência externa.

ReceitaWS 429 rate-limit dispara retry único automático após 3s (INC-004)

Enunciado (corrigido — verdict: partial): Quando consultarSimplesNacional recebe 429 ou detecta padrão rate-limit no response body (INC-005), dorme RATE_LIMIT_RETRY_DELAY_MS (3s) e tenta o lookup mais uma vez. Máximo de 1 retry por chamada; se o retry também falha, retorna status='failed' com reason='rate_limited'. **Nota de observabilidade:** HTTP 429 na tentativa 1 loga com traceId (linha 146) em vez de traceIdAttempt, criando inconsistência de rastreabilidade, enquanto INC-005 loga corretamente com traceIdAttempt (linha 196).

Tipo: arch-decision

Evidência: - supabase/functions/_shared/receitaws-client.ts:55-85 — chama attemptLookup(params, 1); se rate_limited, dorme e tenta attemptLookup(params, 2). - supabase/functions/_shared/receitaws-client.ts:143-159, 179-209 — detecção HTTP 429 (linha 143) + padrão INC-005 (linha 179) para texto “too many requests” mesmo com HTTP 200.

Regressão se: A lógica de retry fosse removida e o primeiro rate-limit falhasse imediatamente — criação de cliente/envio de pedido cairiam em regime_lookup_falha com mais frequência em horários de pico, bloqueando pedidos desnecessariamente quando um simples retry teria sucesso.

Contrato — Condições de Pagamento

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Invariantes

intervalo_parcela é o source-of-truth para emissão de pedidos

- **Tipo:** invariant
- **Enunciado:** Ao emitir um pedido ao Tiny (ou qualquer processo de faturamento), o array `intervalo_parcela` é sempre a fonte de verdade para gerar as parcelas. NÃO use `Prazo_pagamento` (scalar) nem `Quantidade_parcelas` para construir intervalos em tempo de emissão — sempre leia `intervalo_parcela`. Quando `intervalo_parcela` está vazio, o fallback é `[0]` (à vista).
- **Evidência:**
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:551-580` — linhas 562-568 extraem e normalizam `intervalo_parcela` de `Condicao_De_Pagamento`, alimentando a construção de parcelas (linhas 582-591).
 - `supabase/functions/emitir-pedido-sem-vendedor/index.ts:132-159` — processa `intervalosRaw`, converte para array e mapeia para objeto `parcela` com dias e valor.
 - `supabase/functions/emitirpedido/index.ts:129-140` — extrai `intervalo_parcela`, mapeia cada dias para `parcela`, divide valor total igualmente entre as parcelas.
- **Regressão se:** o código usar `Prazo_pagamento` ou `Quantidade_parcelas` para gerar intervalos no Tiny — parcelamentos complexos (10/15/20) degenerarão em parcelas erradas; o Tiny receberia um único prazo final ou ordem incorreta de vencimentos.

Quantidade_parcelas é sempre o comprimento do intervalo_parcela (imposto apenas na API v2)

- **Tipo:** invariant
- **Enunciado:** A regra `Quantidade_parcelas == intervalo_parcela.length` é imposta apenas pela API TypeScript v2 (`condicoes-pagamento-v2/index.ts`), via `processarPrazoPagamento`, que sincroniza os campos em CREATE e UPDATE. Para '10/15/20', `Quantidade_parcelas = 3`; à vista = 1 (ou 0 se `intervalo` vazio). Existe uma rota SQL alternativa (`rpc_insert_condicao_pagamento`) que NÃO impõe a sincronização, permitindo violações (ex: `quantidade_parcelas=3` com `intervalo_parcela=[10,20]`). Recomendação: adicionar validação no banco (CHECK constraint ou trigger) ou deprecicar a RPC antiga em favor da API v2.
- **Evidência:**
 - `supabase/functions/condicoes-pagamento-v2/index.ts:86-111` — `processarPrazoPagamento()`: linha 106 `quantidadeParcelas = valores.length`.
 - `supabase/functions/condicoes-pagamento-v2/index.ts:332` — INSERT: `Quantidade_parcelas: quantidadeParcelas`.
 - `supabase/functions/condicoes-pagamento-v2/index.ts:389` — UPDATE: `Quantidade_parcelas = quantidadeParcelas`.
 - `supabase/migrations/122_baseline_prod_functions_20260601.sql` — `rpc_insert_condicao_pagamento` aceita `p_quantidade_parcelas` e `p_intervalo_parcela` como parâmetros independentes sem sincronização.
- **Regressão se:** `Quantidade_parcelas` divergir de `intervalo_parcela.length` — loops de geração de parcelas no Tiny receberão comprimento errado; divisão de valor por parcela será matematicamente errada.

Descrição contém TODAS as parcelas (imposto apenas no caminho automático)

- **Tipo:** invariant
- **Enunciado:** A regra é aplicada apenas no caminho automático de geração: quando `descricao` NÃO é fornecida, `gerarDescricao()` constrói o nome incluindo TODOS os intervalos de parcela separados por '/' (ex: 'PIX - 10/15/20 dias - desc extra 0%'), não apenas o último valor. Porém, em create (linha 319) e update (linha 407), se o cliente enviar `body.descricao`, a descrição é inserida/atualizada diretamente sem validação, permitindo contornar a regra.
- **Evidência:**
 - `supabase/functions/condicoes-pagamento-v2/index.ts:119-129` — comentário explícito: 'Mantém TODAS as parcelas no nome (ex.: "10/15/20 dias")'. Antes usava só o último prazo,

- gerando nomes errados (“20 dias”) para condições parceladas.’
- supabase/functions/condicoes-pagamento-v2/index.ts:126-129 — gerarDescricao(), linha 128 constrói `{{intervaloParcela.join('/')}}` dias — junta TODOS os intervalos com ‘/’.
- supabase/functions/condicoes-pagamento-v2/index.ts:319 (create) e :407-408 (update) — caminho que aceita body.descricao arbitrária sem validação (contorno da regra).
- **Regressão se:** apenas o último prazo for incluído no nome (ex: ‘PIX - 20 dias’ para 10/15/20) — usuários não saberão que existem parcelas intermediárias; auditoria do sistema ficará confusa sobre quais intervalos foram realmente utilizados.

Regras de Negócio

Parcelamento é TRUE se Quantidade_parcelas > 1 (condicional no UPDATE)

- **Tipo:** business-rule
- **Enunciado:** A regra Parcelamento = (Quantidade_parcelas > 1) é implementada apenas CONDICIONALMENTE em condicoes-pagamento-v2: é respeitada em CREATE sempre (linha 335), mas em UPDATE (linhas 386-392) apenas quando prazoPagamento é definido explicitamente. Se um UPDATE alterar apenas outros campos (ex: descontoExtra, valorMinimo, condicaoCredito) sem tocar prazoPagamento, o Parcelamento fica desatualizado. Não há constraint de banco que force a regra, e a RPC rpc_insert_condicao_pagamento aceita p_parcelamento sem validação.
- **Evidência:**
 - supabase/functions/condicoes-pagamento-v2/index.ts:335 — INSERT: Parcelamento: quantidadeParcelas > 1.
 - supabase/functions/condicoes-pagamento-v2/index.ts:390 — UPDATE: Parcelamento = quantidadeParcelas > 1 (só quando prazoPagamento fornecido).
 - supabase/functions/condicoes-pagamento-v2/index.ts:386-392 — UPDATE parcial sem prazoPagamento não recalcula Parcelamento.
 - supabase/migrations/122_baseline_prod_functions_20260601.sql:5824-5844 — rpc_insert_condicao_pagamento aceita p_parcelamento sem validação.
- **Regressão se:** Parcelamento ficar FALSE para uma condição com 3 parcelas — filtros e consultas que usam esse flag ignorarão errado a condição; análises de parcelamentos estarão incompletas.

intervalo_parcela é gerado ao informar prazoPagamento (mas pode ficar vazio)

- **Tipo:** business-rule
- **Enunciado:** Ao criar uma Condição de Pagamento com prazoPagamento (ex: ‘30/60/90’), o processamento TENTA gerar um intervalo_parcela correspondente, mas o intervalo_parcela pode ficar vazio (array []) se a string prazoPagamento não contiver valores numéricos válidos (ex: ‘abc/def/xyz’ passa na validação de não-vazio mas resulta em array vazio após parseFloat). A inserção sempre ocorre com intervalo_parcela, mas seu conteúdo não é garantido. A validação só checa se prazoPagamento é vazio/null — não valida se contém valores numéricos.
- **Evidência:**
 - supabase/functions/condicoes-pagamento-v2/index.ts:300-305 — validação: se prazoPagamento vazio, erro. Senão, processarPrazoPagamento() é chamado.
 - supabase/functions/condicoes-pagamento-v2/index.ts:95-103 — filter remove todos os NaN, podendo resultar em array vazio.
 - supabase/functions/condicoes-pagamento-v2/index.ts:337 — INSERT: intervalo_parcela: intervaloParcela.
 - supabase/functions/condicoes-pagamento-v2/index.ts:386 — UPDATE não valida que prazoPagamento seja não-vazio.
- **Regressão se:** intervalo_parcela for NULL/vazio apesar de Quantidade_parcelas > 1 — a emissão do pedido será impossível (length undefined); processarPrazoPagamento na emissão tentará iterar sobre

undefined/null e falhará.

Desconto é percentual simples do valor total

- **Tipo:** business-rule
- **Enunciado:** O campo Desconto é um percentual (0-100). Aplicado ao valor_total do pedido na emissão UMA ÚNICA VEZ: $\text{descontoExtra} = \text{valor_total} * (\text{Desconto}/100)$ e $\text{valor_com_desconto} = \text{valor_total} - \text{descontoExtra}$. Não é percentual cumulativo com outros descontos — descontos padrão e extra ficam em colunas separadas (percentual_desconto_padrao vs percentual_desconto_extra) e são aplicados independentemente.
- **Evidência:**
 - `supabase/functions/emitirpedido/index.ts:122-124` — $\text{descontoExtra} = \text{valorTotalPedido} * (\text{condicaoPagamento.Desconto} / 100)$.
 - `supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:597` — Desconto extraído do `pedido_venda` como `valor_desconto_extra` e enviado ao Tiny.
- **Regressão se:** Desconto for interpretado como desconto fixo (em R\$) em vez de percentual — valores serão severamente incorretos; um desconto de '10' (10%) seria aplicado como R\$10 fixo, quebrando pedidos acima de R\$100.

Descrição é regenerada automaticamente quando componentes mudam (com valores mistos)

- **Tipo:** business-rule
- **Enunciado:** Durante UPDATE, se algum de (`formaPagamentoId`, `prazoPagamento`, `descontoExtra`) for modificado mas `descricao` não for fornecido, a função chama `gerarDescricao()` usando: (a) a forma de pagamento fornecida ou atual no BD, (b) o prazo fornecido ou atual no BD, (c) o desconto fornecido ou atual no BD. Assim, se apenas `formaPagamentoId` for modificado, `descricao` será regenerada usando a NOVA forma de pagamento mas os ANTIGOS prazo e desconto (valores mistos, não necessariamente “os novos valores”).
- **Evidência:**
 - `supabase/functions/condicoes-pagamento-v2/index.ts:407-440` — UPDATE: linhas 407-409 checam se `descricao` foi fornecido; se não, mas algum componente foi atualizado, chama `gerarDescricao()`.
 - `supabase/functions/condicoes-pagamento-v2/index.ts:425-435` — quando `body.prazoPagamento` e `body.descontoExtra` são undefined, `prazoInput` e `desconto` são buscados do BD (atuais), não dos novos valores do request.
- **Regressão se:** Descrição não fosse regenerada após mudança de `prazoPagamento` — um admin alteraria '30 dias' para '30/60/90' mas o nome continuaria '30 dias', enganando usuários sobre o parcelamento real.

Decisões de Arquitetura

intervalo_parcela é um array bigint[] no banco

- **Tipo:** arch-decision
- **Enunciado:** O campo `intervalo_parcela` na tabela `Condicao_De_Pagamento` é do tipo `bigint[]` (array de inteiros). A aplicação processa valores como números (`parseFloat`, podendo incluir decimais), mas o PostgreSQL os converte implicitamente para `bigint[]` na inserção. Armazena [10, 15, 20] para parcelamento 10/15/20 dias. Pode ser NULL ou array vazio para condições à vista. Todas as operações SELECT, INSERT e UPDATE o tratam consistentemente como array.
- **Evidência:**

- supabase/functions/condicoes-pagamento-v2/index.ts:199,279 — SELECT e INSERT/UPDATE tratam intervalo_parcela como array; linhas 337, 391 inserem/atualizam array diretamente.
- supabase/functions/tiny-enviar-pedido-venda-v1/index.ts:562-568 — Array.isArray(cond.intervalo_parcela) check e map dos elementos.
- **Regressão se:** intervalo_parcela for armazenado como string '10/15/20' em vez de array [10, 15, 20] — parsing na emissão falharia; conversão de string para array seria necessária, quebrando o contrato da API.

Contrato — Comissões

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Cada bloco documenta uma regra verificada contra o código-fonte. Quando a verificação apontou imprecisão (verdict: partial), o enunciado abaixo já reflete o texto corrigido.

Invariantes (garantidas por constraint/schema)

Constraint: desconto_minimo <= desconto_maximo em listas_preco_comissionamento

Enunciado: Banco rejeita faixas onde desconto_minimo > desconto_maximo. CHECK (desconto_minimo <= desconto_maximo). A validação é reforçada em múltiplas camadas: a RPC upsert_price_list levanta exceção antes do insert e a REST API (listas-preco-v2) valida antes de inserir; a camada de aplicação é ainda mais estrita (rejeita min == max), enquanto o banco só exige min <= max. **Tipo:** invariant **Evidência:** - supabase/schema_baseline.sql (linha 861) — ADD CONSTRAINT lpc_min_le_max CHECK ((desconto_minimo <= desconto_maximo)) **Regressão se:** Se a constraint for removida, faixas invertidas podem ser inseridas, levando a comportamento imprevisível de comissão.

Período aberto por vendedor é único (controle_comissao_periodo)

Enunciado: Tabela controle_comissao_periodo tem UNIQUE(vendedor_uuid, periodo). A constraint garante que cada combinação (vendedor, período) é única (máximo 1 registro por período por vendedor), prevenindo duplicatas de período para um vendedor. **Porém, não há constraint, trigger ou validação que impeça múltiplos períodos com status='aberto' para o mesmo vendedor** (ex.: 2026-01 aberto + 2026-02 aberto é permitido). **Tipo:** invariant **Evidência:** - supabase/migrations/083_comissoes_gestao.sql (linha 114) — UNIQUE(vendedor_uuid, periodo) **Regressão se:** Se a constraint for removida, múltiplos registros de período poderiam existir, causando ambiguidade no saldo e status.

Período armazenado como 'YYYY-MM' (ex: '2025-10')

Enunciado: Campo periodo em lancamentos_comissao, pagamentos_comissao, controle_comissao_periodo e vendedor_comissao é TEXT. O formato YYYY-MM é documentado como esperado e gerado automaticamente para novos períodos, **mas não há constraint CHECK ou validação que impeça outros formatos TEXT**. A interface RelatorioPeriodoComissoes menciona suporte a formato anual (YYYY) além do mensal (YYYY-MM). **Tipo:** invariant **Evidência:** - supabase/migrations/083_comissoes_gestao.sql (linhas 18, 62, 107) — periodo TEXT NOT NULL -- Formato YYYY-MM - src/types/comissao.ts (linhas 10, 39, 58, 81) — periodo: string // "2025-10" (linha 81 documenta também formato anual "2025") **Regressão se:** Se o formato mudar (ex: YYYYMM ou YYYY/MM), queries de filtro por período quebrarão silenciosamente.

Business rules

Operações de Bonificação não geram comissão

Enunciado: Pedidos cuja natureza_operacao é exatamente 'Bonificação' (case-sensitive, com til) retornam status 'bonificacao_sem_comissao' e não geram registro em vendedor_comissao. **Atenção (pitfall):** a verificação é comparação exata sem normalização — variações de caso ou caracteres (ex: 'bonificação', 'BONIFICACAO') contornam a regra. Uma implementação robusta usaria `LOWER(natureza_operacao) = LOWER('Bonificação')`. **Tipo:** business-rule **Evidência:** -
 supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 53-57, esp. linha 54: `if v_pedido.natureza_operacao = 'Bonificação' then sem normalização`) -
 supabase/migrations/122_baseline_prod_functions_20260601.sql — `get_preview_comissoes` usa `natureza_operacao IS DISTINCT FROM 'Bonificação'` **Regressão se:** Se uma bonificação começar a gerar comissão (função remove verificação ou tipo de operação for alterado), comissões indevidas serão calculadas.

Vendedor possui exatamente dois modelos de comissão possíveis

Enunciado: Todo vendedor (dados_vendedor.Comissão) usa modelo 1 (conforme lista de preço e desconto do cliente) ou modelo 2 (alíquota fixa). O sistema rejeita outros valores via `ELSE RAISE EXCEPTION` em `generate_vendedor_comissao`, além de FK para dados_comissao, RLS restringindo modificações a backoffice e validação TypeScript ('aliquota_fixa' → 2 / 'lista_preco' → 1). **Tipo:** business-rule **Evidência:** -
 supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 69-98) — `IF v_tipo_comissao = 2 ... ELSIF = 1 ... ELSE RAISE EXCEPTION` -
 supabase/migrations/122_baseline_prod_functions_20260601.sql — `get_preview_comissoes` `CASE WHEN dv.Comissão = 2 ... WHEN dv.Comissão = 1` **Regressão se:** Se vendedor tiver Comissão != 1 AND != 2, o sistema lança exceção 'Tipo de comissão inválido'.

Tipo 1 (por lista): comissão derivada de lista_de_preco + desconto do cliente

Enunciado: Para vendedores com modelo 1, a comissão é buscada em listas_preco_comissionamento pelo lista_de_preco do cliente (se definido) e seu desconto (desconto entre desconto_minimo e desconto_maximo). Se nenhuma faixa for encontrada, **OU se lista_de_preco for NULL, OU se não houver faixas na tabela, a comissão = 0** (fallback silencioso). **Tipo:** business-rule **Evidência:** -
 supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 72-93) — `SELECT lpc.comissao WHERE lista_preco_id = ... AND desconto BETWEEN min AND max; fallback NULL -> 0` -
 supabase/schema_baseline.sql (linhas 339-345) — `listas_preco_comissionamento(id, lista_preco_id, desconto_minimo, desconto_maximo, comissao)`; FK cliente com `ON DELETE SET NULL` (linha 875) **Regressão se:** Se cliente.desconto ficar NULL ou listas_preco_comissionamento for deletada, a comissão cai para 0 silenciosamente.

Tipo 2 (alíquota fixa): comissão é percentual fixo do vendedor

Enunciado: Para vendedores com modelo 2, `comissao_percentual = dados_vendedor.aliquotafixa` (constante por vendedor). Não há rota alternativa: em todos os caminhos de cálculo o percentual vem exclusivamente de `coalesce(aliquotafixa, 0)`. **Tipo:** business-rule **Evidência:** -
 supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 69-70) — `IF v_tipo_comissao = 2 THEN v_percentual := coalesce(v_aliquota_fixa, 0)` -
 supabase/schema_baseline.sql (linhas 176-212) — `dados_vendedor.aliquotafixa NUMERIC` **Regressão se:** Se aliquotafixa for NULL, o sistema defaulta para 0 (sem comissão).

Valor de comissão = valor_total * percentual / 100, arredondado a 2 casas

Enunciado: Cálculo: `ROUND((valor_total_pedido::numeric * percentual / 100), 2)`. Acumulação, saldo final e pagamentos usam `NUMERIC(15,2)`. **Tipo:** business-rule **Evidência:** -
`supabase/migrations/132_fix_generate_comissao_cliente_oc.sql` (linhas 100-101) —
`v_valor_comissao := round((v_pedido.valor_total::numeric * v_percentual / 100), 2)` -
`supabase/schema_baseline.sql` (linhas 320-329, 339-345, 412-422) — tabelas de comissão em `NUMERIC(15, 2)` **Regressão se:** Se o arredondamento mudar (ex: 3 casas decimais), saldos desviarão centavo a centavo.

Pedido sem vendedor_uuid não gera comissão

Enunciado: Se `pedido.vendedor_uuid IS NULL`, a geração de comissão falha com exceção '`Pedido % sem vendedor_uuid`'. Todos os pontos de entrada (trigger `trigger_comissao_faturado`, edge function `comissoes-v2`, RPC) passam obrigatoriamente por `generate_vendedor_comissao` (`SECURITY DEFINER`), que valida antes de qualquer insert. **Tipo:** business-rule **Evidência:** -
`supabase/migrations/132_fix_generate_comissao_cliente_oc.sql` (linhas 45-46) — IF
`v_pedido.vendedor_uuid IS NULL THEN RAISE EXCEPTION` **Regressão se:** Se a verificação de NULL for removida, pedidos órfãos gerarão comissão sem dono (FK falhará ou criará inconsistência).

Cliente (tipo 1) ausente ou com lista_de_preco NULL

Enunciado: Se o cliente **não existir**, o sistema **lança exceção 'Cliente % não encontrado'** e não processa comissão (não defaulta para 0). Se o cliente existir MAS `lista_de_preco` for NULL, ou nenhuma faixa de comissionamento corresponder ao desconto, o sistema defaulta percentual para 0, resultando em `valor_comissao = 0`. **Tipo:** business-rule **Evidência:** -
`supabase/migrations/132_fix_generate_comissao_cliente_oc.sql` (linhas 79-81) — if not found then
raise exception '`Cliente % não encontrado.`'; (linhas 82-93) `v_lista_preco_id NULL` ⇒ query sem
match ⇒ `v_percentual` forçado a 0 na linha 92 **Regressão se:** Se o cliente for deletado após o pedido, a comissão será recalculada para 0 ao atualizar o pedido.

Faixa de desconto: primeiro match é usado

Enunciado: Busca em `generate_vendedor_comissao`: `ORDER BY desconto_minimo ASC, id ASC LIMIT 1`. Se cliente.desconto se encaixa em múltiplas faixas com `desconto_minimo` idênticos, a de menor id é usada como desempate. **Nota (pitfall):** a leitura em TypeScript (`listas-preco-v2`) omite o `id ASC`, afetando apenas exibição, não o cálculo de comissão. **Tipo:** business-rule **Evidência:** -
`supabase/migrations/132_fix_generate_comissao_cliente_oc.sql` (linhas 83-89) — ... ORDER BY
`lpc.desconto_minimo ASC, lpc.id ASC LIMIT 1` - `supabase/functions/listas-preco-v2/index.ts`
(linhas 211, 349) — `.order('desconto_minimo', { ascending: true })` sem critério de desempate `id`
Regressão se: Se o `ORDER BY` for alterado (ex: DESC), a faixa de maior desconto seria usada, invertendo a comissão.

Faixa com (min=0, max=0, comissao=0) é automaticamente deletada

Enunciado: O trigger `after_insert_delete_zero_discount` existe e remove linhas com `desconto_minimo=0`, `desconto_maximo=0` e `comissao=0` imediatamente após INSERT. **Porém**, a API TypeScript (`listas-preco-v2`) defaulta `desconto_maximo` para 100 quando não fornecido, então em fluxos normais faixas são inseridas como `{0, 100, 0}` e o trigger NÃO é disparado. O trigger é efetivo apenas para inserções diretas (RPC/SQL) que especifiquem os três campos como 0. **Tipo:** business-rule **Evidência:** -
`supabase/migrations/122_baseline_prod_functions_20260601.sql` — `delete_zero_discount_rows`: IF
NEW.`desconto_minimo` = 0 AND NEW.`desconto_maximo` = 0 AND NEW.`comissao` = 0 THEN DELETE -
`supabase/schema_baseline.sql` — CREATE TRIGGER `after_insert_delete_zero_discount` ... EXECUTE
FUNCTION `delete_zero_discount_rows()` - `supabase/functions/listas-preco-v2/index.ts` (linhas 312,

435) — `max == null ? 100 : Number(max)` **Regressão se:** Se o trigger for desabilitado, faixas nulas se acumularão e o primeiro match (min=0) absorverá desconto=0, zerando comissões inesperadamente.

Status de período é um de: ‘aberto’, ‘fechado’, ‘pago’

Enunciado: Coluna status em controle_comissao_periodo tem CHECK (status IN ('aberto', 'fechado', 'pago')). Estado inicial DEFAULT 'aberto'. As RPCs (084, 122) inserem apenas valores válidos.

Tipo: business-rule **Evidência:** - supabase/migrations/083_comissoes_gestao.sql (linha 108) — CHECK (status IN ('aberto', 'fechado', 'pago')) e DEFAULT 'aberto' - supabase/schema_baseline.sql (linhas 164, 849) — confirmam a definição **Regressão se:** Se um status diferente (ex: 'aguardando') for inserido, a constraint falhará; se removida, inconsistências aparecem.

Não é permitido adicionar lançamentos em período fechado ou pago

Enunciado: A função create_lancamento_comissao_v2 verifica IF status IN ('fechado', 'pago') THEN RAISE EXCEPTION. **Atenção (pitfall):** a proteção é incompleta — (a) se não existir registro em controle_comissao_periodo, status retorna NULL e NULL IN (...) é FALSE, permitindo inserção; (b) PUT/DELETE de lançamentos em /comissoes-v2 não validam o status do período, contornando a RPC inteiramente. **Tipo:** business-rule **Evidência:** - supabase/migrations/084_comissoes_rpc.sql (linhas 117-124) — SELECT status INTO v_status_periodo; IF v_status_periodo IN ('fechado', 'pago') THEN RAISE EXCEPTION - supabase/functions/comissoes-v2/index.ts (linhas 228-244 PUT sem validação; 249-262 DELETE sem validação) **Regressão se:** Se a verificação for removida, lançamentos retroativos poderiam ser adicionados após fechamento, alterando saldos fechados.

Saldo final = saldo_anterior + comissão + créditos – débitos – pagos

Enunciado: Em fechar_periodo_comissao_v2: `v_saldo_final := v_saldo_anterior + v_total_comissao + v_total_creditos - v_total_debitos - v_total_pagos`. **Escopo:** no relatório (get_relatorio_comissoes_v2), períodos com status 'fechado' ou 'pago' retornam cp.saldo_final armazenado; outros períodos usam a fórmula recalculada. **Tipo:** business-rule **Evidência:** - supabase/migrations/084_comissoes_rpc.sql (linha 244) — fórmula do saldo final - supabase/migrations/084_comissoes_rpc.sql (linhas 85-88) — CASE do relatório: se status = 'fechado' OR 'pago' retorna saldo_final, senão recalcula **Regressão se:** Se o operador de débito for + ao invés de -, saldos crescerão ao invés de diminuir com devoluções.

Novo período herda saldo_anterior = saldo_final do período anterior

Enunciado: Ao fechar período, fechar_periodo_comissao_v2 cria automaticamente o próximo mês com saldo_anterior = saldo_final deste mês (via ON CONFLICT DO UPDATE). **Atenção (pitfall):** essa relação não é protegida por constraint ou trigger — usuários backoffice podem contornar via UPDATE direto na tabela (permitido pela RLS policy “Backoffice pode gerenciar controle de periodos”). **Tipo:** business-rule **Evidência:** - supabase/migrations/084_comissoes_rpc.sql (linhas 269-293) — calcula próximo período; INSERT controle_comissao_periodo com saldo_anterior = v_saldo_final (linha 291) - supabase/migrations/083_comissoes_gestao.sql — RLS policy “Backoffice pode gerenciar controle de periodos” (FOR ALL) **Regressão se:** Se a lógica de herança for removida, saldo_anterior sempre será 0, perdendo carryover e saldos acumulados.

Preview de comissões exclui pedidos que já têm comissão gerada

Enunciado: get_preview_comissoes retorna apenas pedidos com status IN ('Em aberto', 'Aprovado', 'Preparando envio', 'Pronto para envio', 'Enviado'), COM natureza_operacao IS DISTINCT FROM 'Bonificação', E sem registro em vendedor_comissão (NOT EXISTS). **Tipo:** business-rule **Evidência:** - supabase/migrations/122_baseline_prod_functions_20260601.sql (linhas 148-163) — WHERE clause

com 3 condições (status, natureza_operacao, NOT EXISTS) **Regressão se:** Se a verificação NOT EXISTS for removida, o preview contará comissões já geradas duas vezes.

Comissão é recalculada quando valor_total do pedido é atualizado

Enunciado: O trigger pedido_venda_au_generate_comissao gera/atualiza comissão APENAS se (1) valor_total mudar E for > 0, E (2) natureza_operacao <> 'Bonificação'. Comissões também são geradas/atualizadas por: trigger trg_comissao_on_faturado (quando status='Faturado') e pela rota API POST /calcular-pendentes (sem validação de valor_total > 0). **Não há** mecanismo automático de cancelamento de comissões quando valor_total é reduzido para 0 ou quando pedidos são cancelados. **Tipo:** business-rule **Evidência:** - supabase/schema_baseline.sql — trigger pedido_venda_au_generate_comissao: AFTER UPDATE OF valor_total ... WHEN (new.valor_total IS DISTINCT FROM old.valor_total AND new.valor_total > 0) - supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 110-122) — IF v_existente_id IS NOT NULL THEN UPDATE; (linhas 53-57) bloqueio de Bonificação **Regressão se:** Se o trigger for desabilitado, alterações de valor não atualizarão a comissão existente, deixando dados stale.

Comissão é gerada quando status do pedido muda para 'Faturado'

Enunciado: A geração de comissão é disparada por DOIS mecanismos independentes: (1) trigger trg_comissao_on_faturado detecta mudança de status para 'Faturado' (OLD.status <> NEW.status) e chama generate_vendedor_comissao; (2) trigger pedido_venda_au_generate_comissao detecta mudanças em valor_total (distinct AND > 0) e chama generate_vendedor_comissao independente de status. Adicionalmente, generate_vendedor_comissao pode ser chamada manualmente via POST /calcular-pendentes. **Tipo:** business-rule **Evidência:** - supabase/schema_baseline.sql — trigger trg_comissao_on_faturado: AFTER UPDATE OF status ... WHEN (new.status = 'Faturado'); (linha 1286) trigger pedido_venda_au_generate_comissao - supabase/migrations/122_baseline_prod_functions_20260601.sql — trigger_comissao_faturado: IF NEW.status = 'Faturado' AND (OLD.status IS NULL OR OLD.status <> 'Faturado') **Regressão se:** Se o trigger for desabilitado, pedidos faturados não geram comissão automaticamente; devem ser processados manualmente.

Apenas backoffice cria/edita/deleta lançamentos, pagamentos e fecha períodos

Enunciado: Endpoints POST/PUT/DELETE em comissoes-v2 verificam IF user.tipo !== 'backoffice' THEN THROW ERROR. Todos os 10 endpoints de escrita são cobertos e a autenticação JWT é mandatória antes de qualquer processamento. **Tipo:** business-rule **Evidência:** - supabase/functions/comissoes-v2/index.ts (linhas 201, 229, 250, 289, 318, 339, 357, 382; e 481, 523 para /vendas) — IF user.tipo !== 'backoffice' THEN THROW 'Apenas backoffice...' **Regressão se:** Se os checks de role forem removidos, vendedores poderiam manipular comissões próprias.

Vendedor pode visualizar apenas suas próprias comissões (RLS)

Enunciado: RLS policies em lancamentos_comissao, pagamentos_comissao, controle_comissao_periodo têm DUAS camadas: (1) vendedores veem apenas suas próprias linhas (USING auth.uid() = vendedor_uuid); (2) backoffice vê TODAS as linhas de TODOS os vendedores (USING EXISTS(... tipo='backoffice')). A restrição por vendedor_uuid aplica-se apenas a vendedores, não globalmente. **Tipo:** business-rule **Evidência:** - supabase/migrations/083_comissoes_gestao.sql (linhas 31-33, 80-81, 127-128) — USING (auth.uid() = vendedor_uuid); (linhas 82-90) policy “Backoffice pode ver todos pagamentos” sem restrição a vendedor_uuid - supabase/functions/comissoes-v2/index.ts (linhas 135-139, 159-163, 188-190, 276-278, 468-470) — filtro na aplicação: IF user.tipo === 'vendedor' THEN restrict by user.id **Regressão se:** Se a RLS for desabilitada ou filtros removidos, vendedores verão comissões de colegas.

Campos oc_cliente e cliente_nome são gravados e auditados em cada comissão

Enunciado: A função generate_vendedor_comissao foi corrigida em migration 132 para carregar pedido.ordem_cliente e pedido.nome_cliente e gravá-los em vendedor_comissao.oc_cliente e .cliente_nome em ambos INSERT e UPDATE. Esta correção era necessária porque as migrações 082 e 122 não o faziam. **Atenção (pitfall):** o trigger preencher_cliente_nome_vendedor_comissao preenche apenas cliente_nome a partir de cliente_id, deixando oc_cliente sem preenchimento automático em INSERTs diretos. **Tipo:** business-rule **Evidência:** - supabase/migrations/132_fix_generate_comissao_cliente_oc.sql (linhas 28-36 carrega; 115-118 UPDATE; 136-137 INSERT) - supabase/schema_baseline.sql — vendedor_comissao: oc_cliente TEXT, cliente_id BIGINT, cliente_nome TEXT (desnormalizado para auditoria) - Contraexemplo histórico: supabase/migrations/082_commission_logic_update.sql (linhas 119-139 INSERT / 109-114 UPDATE omitem os campos) **Regressão se:** Se a gravação for removida, o relatório não conseguirá mostrar OC e Cliente (ficará em branco, como no bug de abr/2026).

Contrato — Logística & Rastreio SSW

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Cada regra abaixo foi verificada contra o código. Quando o veredito foi *partial*, o enunciado apresentado é o **enunciado corrigido** (corrected_statement) e um contraexemplo é anexado como *pitfall*/nota de regressão.

Invariantes

Enum de tipo de ocorrência (5 valores em Zod, 5 no banco)

Tipo: invariant

Enunciado: O campo tipo de OcorrenciaSSW é restrito a exatamente 5 valores: Cliente, Informativo, Entrega, Sistema, Operacional. O ENUM do banco inclui esses mesmos 5 (migration 119 definiu 4; migration 120 adicionou Entrega). Schema TypeScript e banco devem permanecer sincronizados.

Evidência: - packages/shared/types/frete-logistica.ts:25-31 — enum TipoOcorrenciaSSW com 5 valores. - supabase/migrations/120_ssw_tracking_adjustments.sql:19 — migration 120 adiciona Entrega ao ENUM tipo_ocorrencia_ssw. - supabase/migrations/119_frete_logistica_base.sql:35-41 — migration 119 original definiu 4 valores (antes de Entrega).

Regressão se: adicionar ou remover valores do enum sem coordenar schema TypeScript e migrations do banco; usar tipo de ocorrência fora dos 5 valores definidos.

Idempotência da chave de acesso NFe (índice único global) — PARCIAL

Tipo: invariant

Enunciado (corrigido): A chave de acesso NFe (código de 44 dígitos) deve ser globalmente única entre todos os fretes ativos. O índice único uq_frete_logistica_chave_acesso previne duplicatas no nível do banco. Porém, o tratamento idempotente é INCOMPLETO: enquanto autoCreateFreteLogistica trata o erro 23505 (violação de unicidade) como skip idempotente, o endpoint PUT em frete-logistica-v1/index.ts e o handler de webhook em webhook-tiny-atualizacao/index.ts não possuem tratamento específico para violações de

constraint UNIQUE ao atualizar nfe_chave_acesso, podendo resultar em erros não-tratados ou respostas 404 falsas em vez de comportamento idempotente gracioso.

Evidência: - supabase/migrations/119_frete_logistica_base.sql:273-276 — índice UNIQUE em nfe_chave_acesso WHERE deleted_at IS NULL. - supabase/migrations/119_frete_logistica_base.sql:168 — CHECK constraint nfe_chave_44 valida formato 44 dígitos. - supabase/functions/frete-logistica-v1/index.ts:357 — handler de erro verifica violações de nfe_chave_44.

Regressão se: remover o índice único; permitir múltiplos fretes com a mesma chave NFe; aceitar chave sem validação de 44 dígitos; soft-delete sem revalidar unicidade.

Pitfall (contraexemplo): supabase/functions/webhook-tiny-atualizacao/index.ts:246-258 — UPDATE não trata o código 23505. Dois webhooks concorrentes tentando associar a mesma nfe_chave_acesso a fretes distintos: um falha com erro PostgreSQL não-tratado em vez de conflito gracioso.

Idempotência de Pedido Venda + NFe Numero (índice composto) — PARCIAL

Tipo: invariant

Enunciado (corrigido): A combinação (empresa_id, nfe_numero) deve ser única entre fretes ativos. Essa constraint garante que o R-LOG-3 auto-create do Tiny não duplique fretes quando o MESMO PEDIDO é reenviado (idempotência de INSERT). Porém, a idempotência de UPDATE não é garantida: se dois updates de webhook concorrentes tentam setar nfe_numero em fretes diferentes com (empresa_id, nfe_numero) idêntico, o segundo falha com erro 23505 e não é tratado como sucesso idempotente.

Evidência: - supabase/migrations/119_frete_logistica_base.sql:267-271 — índice UNIQUE uq_frete_logistica_empresa_nfe em (empresa_id, nfe_numero) WHERE nfe_numero IS NOT NULL. - supabase/migrations/121_frete_logistica_uq_pedido_venda.sql — migration adicional para constraints de unicidade de pedido_venda. - supabase/functions/_shared/frete-auto-create.ts:42-47 — R-LOG-3 detecta 23505 (duplicate key) e trata como sucesso idempotente (skipped: true).

Regressão se: remover o índice composto; permitir pares (empresa_id, nfe_numero) duplicados; não tratar erros 23505 de duplicate key no auto-create.

Pitfall (contraexemplo): supabase/functions/webhook-tiny-atualizacao/index.ts:245-258 — UPDATE de nfe_numero sem detecção/recuperação de duplicata concorrente; erro 23505 é logado mas não tratado como idempotente. Segundo webhook concorrente setando o mesmo nfe_numero em frete diferente falha.

Regras de negócio

Aderência de status terminal (2 definições) — PARCIAL

Tipo: business-rule

Enunciado (corrigido): A regra é aplicada nos fluxos principais (on-demand via frete-logistica-v1 e cron via ssw-sweep-v1), mas há um ponto de entrada alternativo (ssw-tracking-v1) que permite polling SSW sem respeitar status terminal — descrito como função de “debug e teste manual”, mas exposta na API de produção. Existem duas definições consistentes de status terminal: (1) helpers.ts define TERMINAL_STATUSES como {Entregue, Devolvido - Entregue}, e (2) ssw-refresh.ts define TERMINAL_FILTER como filtro SQL para os mesmos status. Fretes não-terminais são elegíveis para polling SSW; uma vez em Entregue ou Devolvido - Entregue, não há mais polling.

Evidência: - supabase/functions/_shared/frete-logistica-helpers.ts:65-72 — definição do set `TERMINAL_STATUSES`: apenas Entregue e Devolvido - Entregue. - supabase/functions/_shared/ssw-refresh.ts:11-14 — comentário reconhece explicitamente “duas definições divergentes de status” como padrão robusto intencional. - supabase/functions/_shared/ssw-refresh.ts:110-115 — query do sweep exclui status terminais: `.not('status_entrega', 'in', TERMINAL_FILTER)`.

Regressão se: adicionar Recusado ou qualquer outro status ao set terminal sem atualizar tanto `TERMINAL_STATUSES` quanto `TERMINAL_FILTER`; fazer polling de fretes já em Entregue/Devolvido - Entregue.

Pitfall (contraexemplo): supabase/functions/ssw-tracking-v1/index.ts:70 — chamada a `fetchSswTracking(chaveNfe)` sem validação de `isTerminalStatus()`, permitindo polling SSW para qualquer chave NFe independentemente do `status_entrega` do frete associado.

Override de status “sticky” (regra do último evento administrativo)

Tipo: business-rule

Enunciado: Ao resolver status a partir da timeline de rastreo SSW, se o último evento mapeia para Em Trânsito (mapeamento genérico default), o sistema busca para trás por um status “sticky” (Entregue, Devolvido - Entregue, Devolvido - Trânsito ou Recusado). Isso impede que eventos administrativos como ANEXADO COMPROVANTE (70) revertam uma entrega concluída. Agendado e Em Trânsito - Reentrega NÃO são sticky, para preservar a progressão.

Evidência: - supabase/functions/_shared/frete-logistica-helpers.ts:99-120 — `resolveFreteStatusFromTracking` implementa a lógica sticky com o set `STICKY`. - tests/edge/ssw-tracking-helpers.test.ts:90-97 — teste verifica que comprovante complementar (70) após entrega não reverte status para Em Trânsito. - tests/edge/ssw-tracking-helpers.test.ts:99-105 — teste confirma que saída para entrega após agendamento reverte para Em Trânsito (Agendado não é sticky).

Regressão se: remover qualquer status do set `STICKY`; adicionar Agendado ou Reentrega ao set `STICKY`; processar a timeline linearmente sem busca reversa por status sticky.

Mapeamento de tipo e código de ocorrência SSW — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Códigos de ocorrência SSW mapeiam para status distintos na função `mapOcorrenciaToStatus` APENAS para fluxos guiados por SSW: `tipo=Entrega + (01) → Entregue`; `tipo=Cliente + (02) → Agendado`; `AGENDAD[A0] + (08) → Agendado`; `RECUSAD[A0] → Recusado`; variantes `DEVOLU[CÇ][AÃ]0` (checadas ANTES de RECUSADO) → variantes Devolvido; `REENTREGA → Em Trânsito - Reentrega`; default → Em Trânsito. Padrões regex são case-insensitive e checados em ordem específica. Porém, o status também é atribuído diretamente (contornando este mapper) nos endpoints `romaneio-logistica-v1` e `entrega-publica-v1`, e a ordem de precedência (DEVOLUÇÃO antes de RECUSADO) não está explicitamente documentada na regra original.

Evidência: - supabase/functions/_shared/frete-logistica-helpers.ts:74-97 — `mapOcorrenciaToStatus` implementa as regras de mapeamento com padrões regex. - tests/unit/ssw-status-mapper.test.ts:35-75 — cobertura abrangente de todos os caminhos de mapeamento. - tests/edge/ssw-tracking-helpers.test.ts:38-73 — testes de edge case confirmam ordem de mapeamento e casamento de padrões.

Regressão se: alterar padrões regex, ordem de mapeamento ou associações código-para-status; aceitar novos códigos de ocorrência sem atualizar o mapper.

Pitfall (contraexemplo): supabase/functions/romaneio-logistica-v1/index.ts:323 atribui status_entrega: 'Em Trânsito' diretamente sem chamar mapOcorrenciaToStatus. Da mesma forma, supabase/functions/entrega-publica-v1/index.ts:143 e :166 atribuem Entregue e Agendado diretamente sem validação pelo mapper. Nota: AGENDAD[A0]\s*(08\) exige AMBOS os padrões juntos, não apenas (08).

Cache de polling SSW (TTL de 30 minutos)

Tipo: business-rule

Enunciado: Dados de rastreio SSW são cacheados por frete com base em frete_logistica_ocorrencia.created_at. O cache é válido por 30 minutos. O polling é pulado se a ocorrência mais recente for mais nova que 30 minutos atrás, a menos que force=true seja passado. Aplica-se tanto a polls on-demand (get_with_ocorrencias?force=true) quanto a sweeps agendados (ssw-sweep-v1 cron com force=false).

Evidência: - supabase/functions/_shared/ssw-client.ts:122-132 — isCacheStale define TTL de 30 minutos na constante CACHE_TTL_MS. - supabase/functions/_shared/ssw-refresh.ts:45-57 — refreshSswForFrete checa staleness do cache antes de fazer polling, exceto se force=true. - supabase/functions/ssw-sweep-v1/index.ts:44-46 — sweep cron horário usa force=false, respeitando o cache.

Regressão se: alterar a constante TTL sem atualizar todos os callers; fazer polling com force=false quando a ocorrência está fresca; ignorar a checagem de staleness do cache.

Controle de concorrência do sweep SSW

Tipo: business-rule

Enunciado: O polling SSW em lote processa fretes com concorrência de 8 (CONCURRENCIA=8) para respeitar o rate limit SSW de 20 req/s. Lotes do sweep processam sequencialmente a 8 requisições concorrentes por lote, para evitar sobrecarregar a API externa. Limite default do sweep é 500 fretes por execução; 300 para o sweep via cron.

Evidência: - supabase/functions/_shared/ssw-refresh.ts:100-136 — sweepSsw implementa batching CONCURRENCIA=8 e o parâmetro limit. - supabase/functions/ssw-sweep-v1/index.ts:44-46 — cron horário usa limit: 500 para sweep geral.

Regressão se: aumentar a concorrência além de 8 sem coordenação com o rate limit SSW; remover o parâmetro limit causando queries ilimitadas; mudar o processamento em lote para sequencial.

Status inicial de novo frete — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Todos os fretes recém-criados via POST para frete-logistica-v1 ou via R-LOG-3 auto-create do Tiny têm status default Em Separação. Porém, entrega-publica-v1 (endpoint público, não-autenticado) pode criar/atualizar fretes com valores de status terminal (Entregue, Agendado) diretamente, contornando o default Em Separação.

Evidência: - packages/shared/types/frete-logistica.ts:116 — schema FreteLogisticaCreate faz default de statusEntrega para Em Separação. - supabase/functions/frete-logistica-v1/index.ts:139 — handler POST faz default de status_entrega para Em Separação se não fornecido. - supabase/functions/_shared/frete-auto-create.ts:34 — R-LOG-3 auto-create fixa status_entrega: 'Em Separação'.

Regressão se: mudar o status default para algo diferente de Em Separação; permitir criação de novos fretes sem status explícito; auto-create usando status inicial diferente.

Pitfall (contraexemplo): supabase/functions/entrega-publica-v1/index.ts:143 e :166 — POST action=confirmar_entrega seta status_entrega para Entregue; POST action=reportar_agendamento seta status_entrega para Agendado, ambos contornando o default Em Separação.

Criação de romaneio dispara transição para Em Trânsito — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Ao criar um romaneio via POST action=create, o sistema LISTA fretes disponíveis filtrados por status válido (Em Separação/Aguardando Coleta) no GET listar_disponiveis, mas o endpoint POST create NÃO valida se os freteIds submetidos estão de fato em status válido antes de inserir em romaneio_frete. A atualização de status para Em Trânsito é tentada APÓS o insert na junção e é não-crítica (logada, mas não faz rollback). Não há constraint no banco impedindo fretes em status inválido de serem incluídos em um romaneio.

Evidência: - supabase/functions/romaneio-logistica-v1/index.ts:88 — listar disponíveis filtra fretes com .in('status_entrega', ['Em Separação', 'Aguardando Coleta']). - supabase/functions/romaneio-logistica-v1/index.ts:320-329 — POST create atualiza status do frete para Em Trânsito; falha é logada mas não falha a criação do romaneio.

Regressão se: alterar os status pré-romaneio elegíveis; atualizar status de forma síncrona como parte do commit do romaneio; impedir atualização de status se o insert do romaneio falhar.

Pitfall (contraexemplo): supabase/functions/romaneio-logistica-v1/index.ts:265-318 — POST create aceita array freteIds sem consultar/validar status_entrega antes de inserir em romaneio_frete (linha 313). Um frete em Entregue ou Recusado pode ser incluído em um romaneio. A atualização de status (linhas 321-329) ocorre após o insert da junção e falha silenciosamente sem rollback.

Cálculo de dias em trânsito (apenas fretes ativos) — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Dias em trânsito (diasEmTransito) é calculado como dias decorridos de data_saida até agora, mas SOMENTE para operações GET de lista (list, list_by_status, get_with_ocorrencias). Quando calculado, a lógica retorna null se: data_entrega não é nulo, data_saida está ausente/inválido, ou o tempo atual é anterior a data_saida. Recuperações de registro único (GET por id), POST (create) e PUT (update) não incluem diasEmTransito nas respostas.

Evidência: - supabase/functions/_shared/frete-logistica-helpers.ts:127-136 — diasEmTransito retorna null se dataEntrega existe ou dataSaida inválido.

Regressão se: calcular dias de trânsito para fretes entregues; remover a checagem de nulo para dataEntrega; usar data atual em vez de cálculo preciso via Date.now().

Pitfall (contraexemplo): supabase/functions/frete-logistica-v1/index.ts:279-284 — GET por id retorna formatFrete() sem enriquecimento de diasEmTransito, contradizendo a aplicação universal implícita na regra original.

Soft delete (campo deleted_at) — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Apenas algumas entidades logísticas impõem soft delete via deleted_at: frete_logistica, transportador_logistica e fatura_transportadora têm coluna deleted_at e queries filtram deleted_at IS NULL. Porém, hard delete é ativamente usado em ssw-refresh.ts linha 66 (frete_logistica_ocorrencia.delete()) e romaneio-logistica-v1/index.ts linha 316 (romaneio_expedicao.delete() em rollback de erro). Adicionalmente, as tabelas origem_frete e regio_destino foram criadas sem coluna deleted_at e não implementam soft delete — origem_frete usa apenas flag ativo, e regio_destino não tem handler de delete.

Evidência: - supabase/migrations/119_frete_logistica_base.sql:162, 81, 213, 231 — colunas deleted_at adicionadas a tabelas de logística. - supabase/functions/frete-logistica-v1/index.ts:302, 407-408 — queries filtram .is('deleted_at', null); DELETE usa soft-delete via update. - supabase/functions/_shared/ssw-refresh.ts:108 — queries do sweep excluem linhas soft-deleted.

Regressão se: hard-delete de qualquer registro logístico; falhar em incluir deleted_at IS NULL nas queries; usar statement DELETE em vez de UPDATE com deleted_at.

Pitfall (contraexemplo): supabase/functions/_shared/ssw-refresh.ts:66 — await supabase.from('frete_logistica_ocorrencia').delete().eq('frete_id', id) executa deleção física sem soft-delete; também supabase/functions/romaneio-logistica-v1/index.ts:316 — await admin.from('romaneio_expedicao').delete().eq('id', romaneioId).

Buckets de status do Dashboard com consolidação e decomposição — PARCIAL

Tipo: business-rule

Enunciado (corrigido): O Dashboard/Kanban implementa 5 buckets com a consolidação de Em Trânsito – Reentrega no bucket Em Trânsito, limite 20 por bucket e ordenação correta (data_emissao desc, id desc), servidos pela action list_by_status. Porém: (1) o status Em Trânsito – Reentrega permanece mapeado em DASHBOARD_BUCKETS apesar de ter sido marcado para remoção no Changelog V1.56 — conflita com a declaração de tipo Zod que não o inclui; (2) o status Aguardando Agendamento (adicionado no Changelog V1.56) não tem bucket mapeado em DASHBOARD_BUCKETS, permanecendo invisível no Dashboard/Kanban apesar de estar registrado no banco.

Evidência: - supabase/functions/_shared/frete-logistica-helpers.ts:44-50 — mapeamento DASHBOARD_BUCKETS: bucket Em Trânsito inclui Em Trânsito e Em Trânsito – Reentrega. - packages/shared/types/frete-logistica.ts:164-172 — tipo DashboardBucketLabel define os nomes exatos dos buckets. - supabase/functions/frete-logistica-v1/index.ts:169-184 — handleListByStatus consulta 5 buckets com limite 20 cada, ordenação consolidada.

Regressão se: alterar definições de bucket; remover a consolidação Em Trânsito; adicionar novos status sem atualizar os buckets; modificar a ordenação dentro dos buckets.

Pitfall (contraexemplo): packages/shared/types/frete-logistica.ts:16 — o tipo Zod StatusEntregaFrete não inclui Em Trânsito – Reentrega (contradiz o mapeamento DASHBOARD_BUCKETS), e

não há bucket para Aguardando Agendamento apesar de estar no Zod.

Feature flags: FEATURE_LOG_CRM (master) e FEATURE_LOG_CRM_SSW (sub-flag) — PARCIAL

Tipo: business-rule

Enunciado (corrigido): Nem todas as funções logísticas gatean em FEATURE_LOG_CRM. Enquanto frete-logística-v1, ssw-sweep-v1, ssw-tracking-v1, origem-frete-v1, regioao-destino-v1 e transportador-logística-v1 implementam gating com 503 quando desabilitado, romaneio-logística-v1 não possui nenhuma verificação de feature flag e processa requisições independentemente do estado da flag. Funções específicas de SSW gatean adicionalmente em FEATURE_LOG_CRM_SSW. Os requisitos de case-sensitivity (exigindo true literal, minúsculo, sem trim; rejeitando TRUE e true) estão corretamente implementados em todos os helpers de flag.

Evidência: - supabase/functions/frete-logística-v1/index.ts:31, 243-244 — frete-logística-v1 checa isLogCrMEnabledFromEnv() e retorna 503 se desabilitado. - supabase/functions/ssw-sweep-v1/index.ts:29-34 — ssw-sweep-v1 gatea em FEATURE_LOG_CRM e FEATURE_LOG_CRM_SSW. - tests/edge/frete-logística-v1.test.ts:20-45 — testes verificam o requisito case-sensitive de true (TRUE maiúsculo é falso).

Regressão se: tornar as flags case-insensitive; aplicar trim nos valores; retornar códigos de status diferentes para o estado desabilitado; remover a checagem da sub-flag SSW; mudar o default para habilitado.

Pitfall (contraexemplo): supabase/functions/romaneio-logística-v1/index.ts:1-335 — Edge Function logística que processa GET e POST sem qualquer verificação de FEATURE_LOG_CRM ou isLogCrMEnabledFromEnv(), violando o gating universal.

Contrato — Permissões, Usuários & RLS

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Business Rules

Apenas backoffice cria/atualiza/deleta usuários

Apenas usuários com tipo='backoffice' podem criar, atualizar ou deletar outros usuários. Tentar criar/atualizar/deletar como vendedor retorna 403 “Insufficient permissions”. A regra é imposta em três camadas: (1) Edge Functions checam user.tipo !== 'backoffice' antes da operação; (2) RPCs revalidam p_created_by/p_updated_by/p_deleted_by = backoffice com RAISE EXCEPTION; (3) RLS policies restringem INSERT a is_user_backoffice(auth.uid()).

- **Tipo:** business-rule
- **Evidência:**
 - supabase/functions/create-user-v2/index.ts:290-298 — if (user.tipo !== 'backoffice') → createPermissionErrorResponse 403
 - supabase/functions/update-user-v2/index.ts:219-220 — check de tipo aplicado via RPC (linha 249)

- supabase/functions/delete-user-v2/index.ts:120-124 — if (user.tipo !== 'backoffice') → 403 “Apenas usuários backoffice podem excluir usuários”
- supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:80-90 — RPC create_user_v2 valida p_created_by backoffice
- **Regressão se:** um vendedor conseguir chamar os endpoints create/update/delete e atingir a RPC sem erro — quebra integridade de dados e o modelo de autorização.

Apenas backoffice altera permissões

Em update-user-v2, apenas usuários backoffice (user.tipo === 'backoffice') podem modificar o campo permissões. Vendedor tentando alterar permissões dispara 403 “Apenas backoffice pode alterar permissões”. A validação é incondicional: se sanitizedData.permissões está definido, o check de backoffice é obrigatório e bloqueante antes de qualquer modificação no banco. A RPC update_user_v2 sequer aceita parâmetro permissões.

- **Tipo:** business-rule
- **Evidência:**
 - supabase/functions/update-user-v2/index.ts:271-277 — if (sanitizedData.permissões !== undefined) { if (user.tipo !== 'backoffice') throw ValidationError('Apenas backoffice...') }
- **Regressão se:** o check de tipo for removido — vendedores poderiam escalar as próprias permissões ou conceder permissões a outros usuários.

Recriar usuário com mesmo email reativa linha soft-deleted (dois caminhos)

create-user-v2 reativa soft-deleted users por dois caminhos: (1) Se auth_user_id é fornecido, a RPC usa INSERT ... ON CONFLICT (user_pkey) DO UPDATE para reativar (ativo=true, deleted_at=NULL, preservando user_id); (2) Se auth_user_id NÃO é fornecido, a Edge Function detecta o soft-deleted user e chama supabaseAdmin.from('user').update(...) com deleted_at=NULL e ativo=true, retornando sem invocar a RPC. Ambos os mecanismos existem e funcionam, mas só o primeiro passa pela RPC.

- **Tipo:** business-rule
- **Evidência:**
 - supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:4-22 — INC-012: ON CONFLICT reativa a linha, preservando user_id → FKs
 - supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:121-131 — ON CONFLICT ON CONSTRAINT user_pkey DO UPDATE seta ativo=TRUE, deleted_at=NULL
 - supabase/functions/create-user-v2/index.ts:362-410 — caminho da Edge Function: detecta usuarioSoftDeletado e chama .update() direto, retornando na linha 410 sem invocar a RPC
- **Regressão se:** a cláusula ON CONFLICT for removida e um INSERT direto for usado — recriar soft-deleted user falharia com “duplicate key” e a reativação seria impossível.

Convite por email quando não há auth_user_id

Se create-user-v2 recebe um usuário NOVO sem auth_user_id, envia convite via inviteUserByEmail, popula auth_user_id da resposta (linha 429) e chama a RPC (linha 441). Porém, ao reativar um soft-deleted user (mesmo email já existe com deleted_at não-nulo), chama resetPasswordForEmail no caminho de reativação, atualiza a tabela user diretamente SEM invocar a RPC, e NÃO popula auth_user_id — retorna cedo na linha 410 após o UPDATE, nunca chegando à RPC da linha 441.

- **Tipo:** business-rule
- **Evidência:**

- supabase/functions/create-user-v2/index.ts:359-431 — Step 4.5: if (!sanitizedData.auth_user_id) chama inviteUserByEmail/resetPasswordForEmail, popula auth_user_id
- supabase/functions/create-user-v2/index.ts:393-410 — caminho de reativação: chama resetPasswordForEmail(email) com redirectTo e retorna na linha 410 sem chamar a RPC (linha 441)
- **Regressão se:** o convite não for enviado ou auth_user_id não for extraído da resposta — a criação completaria mas o usuário não teria como se autenticar.

Invariants

tipo restrito a backoffice ou vendedor

O campo tipo na tabela user deve ser 'backoffice' ou 'vendedor'. Todas as operações create/update validam: p_tipo NOT IN ('backoffice', 'vendedor') → erro. Nenhum outro tipo é permitido. Imposto em múltiplas camadas: CHECK constraint na tabela, validação nas RPCs (RAISE EXCEPTION) e validação nas Edge Functions.

- **Tipo:** invariant
- **Evidência:**
 - supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:59-61 — RPC: IF p_tipo IS NULL OR p_tipo NOT IN ('backoffice', 'vendedor') THEN RAISE EXCEPTION
 - supabase/functions/create-user-v2/index.ts:321-327 — if (!body.tipo || ! [...].includes(body.tipo)) throw ValidationError
 - supabase/functions/update-user-v2/index.ts:219-220 — check de tipo garante apenas backoffice/vendedor
- **Regressão se:** a constraint de tipo for removida — novos tipos poderiam ser inseridos, quebrando lógica de negócio que depende de tipo='backoffice' vs tipo='vendedor'.

permissões é array JSONB

A coluna permissões na tabela user tem CHECK constraint jsonb_typeof(permissões) = 'array'. permissões deve sempre ser armazenado como array JSON de strings, nunca objeto ou null (default '[]'::jsonb). A constraint no banco é a última linha de defesa; a validação na aplicação é defensiva.

- **Tipo:** invariant
- **Evidência:**
 - supabase/migrations/089_user_permissions.sql:35-49 — ADD COLUMN permissões JSONB NOT NULL DEFAULT '[]'; ADD CONSTRAINT user_permissions_array_chk CHECK (jsonb_typeof(permissões) = 'array')
 - supabase/functions/create-user-v2/index.ts:332-335 — if (!Array.isArray(body.permissões)) throw ValidationError('permissões deve ser um array de strings')
- **Regressão se:** a constraint de array for removida e permissões for armazenado como objeto ou null — RPCs e código cliente que esperam iterar sobre array falhariam.

nome com mínimo de 2 caracteres

O campo nome tem validação de comprimento mínimo (≥ 2 após trim) em: RPC create_user_v2 (SQL), RPC update_user_v2 (SQL), Edge Function create-user-v2 (linha 317) e Edge Function update-user-v2 (linha 216). ENTRETANTO, há um caminho alternativo no código de reativação (create-user-v2, linhas 374-389)

que faz UPDATE direto na tabela sem passar pela RPC — nesse caso a validação SQL nunca é aplicada pelo banco (não há CHECK constraint), e a segurança depende apenas da validação da Edge Function, que pode ser contornada via acesso direto (há policy `allow_all` permitindo UPDATE autenticado).

- **Tipo:** invariant
- **Evidência:**
 - `supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:55-57` — RPC: `IF p_nome IS NULL OR LENGTH(TRIM(p_nome)) < 2 THEN RAISE EXCEPTION`
 - `supabase/functions/create-user-v2/index.ts:317-320` — `if (!validateMinLength(body.nome, 2)) throw ValidationError('Nome deve ter pelo menos 2 caracteres')`
 - `supabase/functions/update-user-v2/index.ts:216-218` — mesmo check `validateMinLength`
 - Contraexemplo: `supabase/functions/create-user-v2/index.ts:384-389` (reativação bypassa RPC); `supabase/schema_baseline.sql:1266` (policy `allow_all` permite UPDATE direto sem constraints)
- **Regressão se:** a constraint de comprimento mínimo for removida — nomes de um caractere poderiam ser armazenados, quebrando lógica que assume nome significativo.

Validação de JWT obrigatória nas Edge Functions

Todas as edge functions de usuário (`create/update/list/delete/get-user-v2`) devem validar tokens JWT via `validateJWT()` antes de processar qualquer request. O usuário deve estar ativo (`ativo=true`) e não `soft-deleted` (`deleted_at IS NULL`).

- **Tipo:** invariant
- **Evidência:**
 - `supabase/functions/_shared/auth.ts:25-145` — `validateJWT()` verifica `auth.getUser(token)`, depois consulta a tabela `user` com `.eq('ativo', true).is('deleted_at', null)`
 - `supabase/functions/create-user-v2/index.ts:268-284` — Step 1: AUTENTICAÇÃO — valida JWT antes de prosseguir
 - `supabase/functions/list-users-v2/index.ts:92-101` — validação de JWT; retorna 401 se `authError` ou nenhum user
- **Regressão se:** uma edge function pular `validateJWT()` ou remover o filtro `.eq('ativo', true)` — usuários inativos contornariam a autenticação. > Nota: verdict registrado como `partial` (“verificação falhou”); tratar esta regra como não plenamente confirmada até nova verificação.

Email único entre usuários ativos

Email deve ser único entre usuários ativos (`deleted_at IS NULL`). Múltiplos `soft-deleted` users com o mesmo email são permitidos, mas apenas um usuário ativo por email. `create-user-v2` checa isso antes do INSERT.

- **Tipo:** invariant
- **Evidência:**
 - `supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:67-77` — `SELECT EXISTS(...WHERE LOWER(u2.email) = LOWER(p_email) AND u2.deleted_at IS NULL)` — só usuários ativos bloqueiam criação
 - `supabase/functions/create-user-v2/index.ts:365-373` — distingue `usuarioAtivo` vs `usuarioSoftDeletado`, lança erro se ativo existe
- **Regressão se:** o filtro `deleted_at` for removido do check de unicidade — recriar um usuário com email `soft-deleted` falharia mesmo sendo intencionado permitir o reuso. > Nota: verdict registrado como `partial` (“verificação falhou”); tratar esta regra como não plenamente confirmada até nova verificação.

Arch Decisions

Deleção de usuário é soft delete, não hard delete (não garantido por constraints)

`delete_user_v2` faz soft delete em `public.user` (`UPDATE ativo=false, deleted_at=NOW()`), mas isso NÃO é garantido por constraints do banco. As Foreign Keys em `dados_vendedor(user_id)` e `notificacao(usuario_id)` usam `ON DELETE CASCADE`, o que deletaria fisicamente as linhas dependentes caso um `DELETE` direto fosse executado em `public.user`, violando a preservação do audit trail. A promessa de soft-delete depende inteiramente do código de aplicação nunca usar `DELETE` — o schema em si não a impõe.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/migrations/114_create_user_v2_upsert_reactivate_soft_deletes.sql:1-22` — BUG-006/007: `delete_user_v2` faz só soft-delete (`UPDATE ... SET ativo=false, deleted_at=NOW()`), NÃO toca em `auth.users`
 - `supabase/functions/delete-user-v2/index.ts:137-161` — edge function chama RPC `delete_user_v2`, retorna `deleted_at` (indicador de soft delete)
 - Contraexemplo: `supabase/migrations/002_fix_relacoes_fks.sql:22` — `fk_dados_vendedor_user_id` com `ON DELETE CASCADE`; `supabase/migrations/090_notificacoes_v2.sql:11` — `notificacao.usuario_id` com `ON DELETE CASCADE`
- **Regressão se:** a RPC `delete_user_v2` fizer hard delete em vez de soft update — constraints FK quebrariam e o audit trail seria perdido.

RLS de conta corrente usa allow-all para autenticados (antipadrão)

As tabelas `conta_corrente_cliente` e `pagamento_acordo_cliente` usam policies RLS que permitem acesso indiscriminado com `USING (true)`, `WITH CHECK (true)` para usuários autenticados. A filtragem DEVERIA estar implementada em RLS policies (ex.: `USING (auth.uid() = vendedor_id OR is_backoffice())`), não delegada a RPCs. O padrão atual viola segurança de banco de dados: qualquer autenticado pode ler todas as linhas via acesso direto, ignorando as RPCs que implementam controle de acesso na application layer. Agravante: a Edge Function usa `SERVICE_ROLE_KEY`, que combinado com policies permissivas permite bypass total se acessar as tabelas base em vez das RPCs.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/migrations/068_rls_conta_corrente.sql:13-56` — `conta_corrente_cliente`: `SELECT/INSERT/UPDATE/DELETE` todos usam `USING (true)` para `authenticated`; lógica delegada à RPC
 - `supabase/migrations/068_rls_conta_corrente.sql:15-23` — comentário: “A lógica de permissão é implementada nas funções RPC, então aqui permitimos acesso geral for `authenticated`”
 - Contraexemplo: `supabase/migrations/068_rls_conta_corrente.sql:19-23` (`SELECT` policy `USING (true)` permite qualquer autenticado ler qualquer linha); `supabase/functions/conta-corrente-v2/index.ts:169-171` (Edge Function usa `SERVICE_ROLE_KEY`, contornando RLS se acessar tabelas base diretamente)
- **Regressão se:** a RLS for endurecida para negar acesso autenticado e a filtragem nas RPCs não for atualizada — usuários perdem acesso aos dados de `conta_corrente`.

Sanitização de input previne XSS/injection (aplicada de forma inconsistente)

Inputs de string SÃO sanitizados via `sanitizeInput()` em `create-user-v2` (`email`, `nome`, `user_login`, `permissoes`): remove `<>`, `javascript:`, padrões `on*=`; emails são lowercased e trimmed. PORÉM, em `update-user-v2`, campos de objeto complexos (`dadosBancarios`, `contatosAdicionais`) são armazenados como

JSONB sem sanitização, permitindo persistir conteúdo potencialmente malicioso se os objetos contiverem strings com scripts. A regra de sanitização é imposta de forma inconsistente entre os caminhos de update da mesma entidade.

- **Tipo:** arch-decision
- **Evidência:**
 - supabase/functions/create-user-v2/index.ts:225-229 — `sanitizeInput(input): replace(/[<>]/g, '').replace(/javascript:gi, '').replace(/on\w+=/gi, '')`
 - supabase/functions/create-user-v2/index.ts:338-348 — email lowercased+trimmed, nome trimmed, permissões filtrado para itens não-string
 - Contraexemplo: supabase/functions/update-user-v2/index.ts:340-341 — `dadosBancarios` e `contatosAdicionais` atribuídos direto sem sanitização, ao contrário de `observacoesInternas` (linha 339) e campos `endereco.*` (linhas 329-337)
- **Regressão se:** a sanitização for removida — usuários poderiam injetar tags `<script>` ou event handlers, causando XSS na UI do backoffice.

Contrato — Produtos & Listas de Preço

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Invariantes

Produto.situacao restrita a três valores

A `situacao` do Produto é restrita aos valores 'Ativo', 'Inativo' ou 'Excluído' via CHECK constraint no banco de dados. O DELETE soft-deleta corretamente definindo `situacao='Excluído'`. O CREATE no UI padrão e o import validam os valores. Porém, CREATE e UPDATE na função Edge (`produtos-v2`) **não** validam `body.situacao` antes de enviá-lo ao banco, dependendo apenas do constraint de database para rejeitar valores inválidos.

- **Tipo:** invariant
- **Evidência:**
 - `src/types/produto.ts:27` — `SituacaoProduto = 'Ativo' | 'Inativo' | 'Excluído'`
 - `supabase/schema_baseline.sql` (constraint `produto_situacao_check`) — `situacao IN ('Ativo', 'Inativo', 'Excluído')`
 - `supabase/functions/produtos-v2/index.ts:516-517, 722-723` — create seta `situacao='Ativo'`, delete seta `situacao='Excluído'` e `ativo=false`
 - Contraexemplo: `supabase/functions/produtos-v2/index.ts:516` (create sem validação) e `:649` (update sem validação) aceitam `body.situacao` sem verificar se está em `['Ativo', 'Inativo', 'Excluído']`
- **Regressão se:** o constraint for removido ou um quarto valor (ex.: 'Cancelado') for inserido. Produtos com `situacao` inválida ficam não consultáveis ou causam erro de parsing.

Lista de preço usa exatamente um tipo de comissão: fixa ou conforme_desconto

Cada `ListaPreco.tipoComissao` é 'fixa' (percentual fixo) ou 'conforme_desconto' (faixas por desconto). Se `tipoComissao='fixa'`, usa-se `percentualFixo`. Se `tipoComissao='conforme_desconto'`, o array `faixasDesconto` define a comissão por faixa de desconto. Uma lista não pode ter os dois.

- **Tipo:** invariant
- **Evidência:**
 - `src/types/listaPreco.ts:23-31` — `TipoComissao = 'fixa' | 'conforme_desconto'`; campos condicionais `percentualFixo` (fixa) ou `faixasDesconto` (`conforme_desconto`)
 - `supabase/functions/listas-preco-v2/index.ts:264-276` — validação POST: se `tipoComissao='conforme_desconto'`, `faixasInput.length` deve ser `> 0`
 - `supabase/functions/listas-preco-v2/index.ts:108-109` — `formatListaPreco` determina `tipoComissao` pela presença de linhas de faixas; se existem faixas, o tipo é `'conforme_desconto'`
- **Regressão se:** uma lista for criada com `percentualFixo` e `faixasDesconto` não-vazio simultaneamente. Cálculo de comissão retorna resultados inconsistentes ou quebra por campos ausentes.

Faixas de desconto devem ter `descontoMin < descontoMax` (ou `descontoMax=null` aberta)

Para `tipoComissao='conforme_desconto'`, a validação de faixa é imposta **apenas** pela API REST TypeScript (`index.ts:268-275`): rejeita quando `max != null AND max <= min`. Porém, funções SQL alternativas (`create_lista_preco_v2`, `update_lista_preco_v2`) existem em migrations e permitem inserts diretos **sem validação**. O constraint de banco `lpc_min_le_max` permite `desconto_minimo = desconto_maximo` (`<=` não-estrito), contradizendo o requisito de `<` estrito. A normalização `null→100` está corretamente implementada em TypeScript, mas `update_lista_preco_v2` usa `999.99` em vez de `100`.

- **Tipo:** invariant
- **Evidência:**
 - `supabase/functions/listas-preco-v2/index.ts:268-275` — validação POST: se `max != null && max <= min`, lança erro 'Faixas de desconto inválidas'
 - `supabase/functions/listas-preco-v2/index.ts:379-386` — validação PUT: mesmo check
 - `supabase/functions/listas-preco-v2/index.ts:306-316`, `429-439` — no insert/update, `max == null ? 100 : Number(max)` normaliza faixas abertas para 100
 - `supabase/schema_baseline.sql` (CHECK `lpc_min_le_max`) — `desconto_minimo <= desconto_maximo`
 - Contraexemplo: `supabase/migrations/122_baseline_prod_functions_20260601.sql:6887-6897` — `update_lista_preco_v2` insere faixas com `COALESCE(..., 999.99)` sem validar `desconto_minimo < desconto_maximo`
- **Regressão se:** a validação for removida. Faixas inválidas com min/max invertidos são persistidas; lookups de comissão retornam percentuais errados ou falham silenciosamente.

Cada par cliente-produto em `status_mix` é único

A tabela `status_mix` tem constraint UNIQUE em (`cliente_id`, `produto_id`). Só existe um registro de status por combinação cliente-produto. Upserts usam esse constraint para atualizar registros existentes.

- **Tipo:** invariant
- **Evidência:**
 - `supabase/migrations/137_status_mix_table.sql:13` — UNIQUE (`cliente_id`, `produto_id`)
 - `src/components/CustomerMixTab.tsx:149-152` — upsert com `onConflict: 'cliente_id,produto_id'`
 - `supabase/functions/status-mix-v2/index.ts:98-101` — endpoint PUT faz upsert com `onConflict: 'cliente_id,produto_id'`
- **Regressão se:** linhas (`cliente_id`, `produto_id`) duplicadas forem inseridas. UI mostra status conflitantes ou erro de integridade no upsert.

Denormalização de Marca, Tipo e Unidade no produto (parcial no UPDATE)

O produto armazena cópias denormalizadas de `marca.nome`, `ref_tipo_produto.nome` e `ref_unidade_medida.sigla` em `nome_marca`, `nome_tipo_produto` e `sigla_unidade`. A action de update em `produtos-v2` **tenta** buscar e atualizar esses campos, mas só o faz se a query de lookup tiver sucesso e encontrar o registro. Se o ID for inválido ou o lookup falhar, os campos denormalizados são silenciosamente não atualizados, podendo deixar dados obsoletos no banco (embora constraints FK eventualmente rejeitem a operação inteira). O CREATE valida a existência antes de prosseguir; o UPDATE não valida.

- **Tipo:** invariant
- **Evidência:**
 - `supabase/functions/produtos-v2/index.ts:593-630` — update: se `marcaId/tipoProdutoId/unidadeId` muda, busca da tabela de lookup e atualiza `nome_marca/nome_tipo_produto/sigla_unidade`
 - `supabase/functions/produtos-v2/index.ts:519-521` — create: busca nomes das tabelas de lookup e armazena nos campos denormalizados
 - `supabase/schema_baseline.sql` (tabela `produto`) — `marca (FK)/nome_marca (text); tipo_id (FK)/nome_tipo_produto (text); unidade_id (FK)/sigla_unidade (text)`
 - Contraexemplo: `supabase/functions/produtos-v2/index.ts:593-604` — ao atualizar `marcaId`, faz `if (marca) { updateData.nome_marca = marca.nome }` sem `else/erro` se `marca` for NULL; compare com CREATE (:483-485) que faz `if (!marca) throw Error()`
- **Regressão se:** a lógica de update pular a atualização dos campos denormalizados. Produtos mostram `marca/tipo/unidade` desatualizados após a tabela de lookup ser modificada.

Regras de negócio

Produto elegível para venda: `preco > 0 AND disponivel=true AND ativo=true`

Ao selecionar produtos para um pedido de venda (`SaleFormPage`), só produtos com `preco > 0 AND disponivel !== false AND ativo !== false` aparecem no combobox seletor. O filtro aplica-se aos produtos vindos de `lista_preco_produtos` (registros master produto juntados via RPC). Porém: (1) o filtro é **frontend-only**, aplicado ao renderizar `produtosDisponiveisParaPedido`; (2) `ativo` e `disponivel` são flags globais da tabela `produto`, não específicas por lista; (3) itens de vendas existentes **não** são refiltrados retroativamente; (4) um produto pode ser `ativo=true` mas indisponível a todos os vendedores via `disponivel=false` — não é mecanismo por-vendedor ou por-lista.

- **Tipo:** business-rule
- **Evidência:**
 - `src/components/SaleFormPage.tsx:317` — `.filter(pp => pp.preco > 0 && pp.disponivel !== false && pp.ativo !== false)`
 - `src/types/listaPreco.ts:1-14` — `ProdutoPreco` inclui `preco`, `ativo`, `disponivel`
 - `src/data/mockProdutos.ts` (dados de exemplo) — `ativo=true/disponivel=true` (normal); `ativo=true/disponivel=false` (oculto); `situacao='Excluído'` (soft-deletado)
 - Contraexemplo: `src/components/SaleFormPage.tsx:642-645` — carrega e exibe itens de venda existentes sem reaplicar o filtro, permitindo produtos com `disponivel=false` ou `ativo=false` se adicionados antes da flag mudar
- **Regressão se:** um produto com `preco=0` ou `disponivel=false` for vendido; ou um produto for removido de uma lista sem removê-lo de pedidos já criados com essa lista.

Aba Mix só mostra produtos Ativos e de tipo não-promocional

Ao carregar produtos para a aba Mix do cliente (CustomerMixTab), só produtos com `situacao='Ativo'` E cujo nome de tipo **não** contém 'promo' nem 'brinde' são incluídos. Tipo vazio ou tipo contendo 'revenda' são permitidos. Este é um filtro client-side, não imposto na API.

- **Tipo:** business-rule
- **Evidência:**
 - `src/components/CustomerMixTab.tsx:36-49` — filtro: `tipoNome.includes('revenda') || (!includes('promo') && !includes('brinde'))`; aplica-se a produtos com `situacao='Ativo'`
 - `src/types/statusMix.ts:1-17` — StatusMix liga um produto a um cliente com status 'ativo' ou 'inativo'
- **Regressão se:** o filtro de tipo for removido. Produtos promocionais e brindes aparecem no mix do cliente, causando classificação errada das preferências.

Auto-ativação de produto no mix por pedido (detecção invertida)

O sistema tenta proteger produtos desativados manualmente contra reativação automática ao carregar o mix do cliente (CustomerMixTab), MAS a detecção de “manualmente desativado” está **invertida**: procura por `ativadoManualmente=false & status='inativo'` em vez de `ativadoManualmente=true & status='inativo'`. Quando um usuário reativa um produto (toggle para ativo), o campo `ativado_manualmente` deveria ser resetado para false no mesmo upsert, mas não é. Assim, um produto pode ser reativado pelo usuário mas ficar marcado com `ativado_manualmente=true` — inversão semântica da intenção.

- **Tipo:** business-rule (pitfall)
- **Evidência:**
 - `src/components/CustomerMixTab.tsx:114-146` — lógica de auto-ativação: verifica se produto aparece em vendas, pula se foiManualmenteDesativado
 - `src/components/CustomerMixTab.tsx:191-207` — toggle manual: seta `ativado_manualmente=true` ao desativar, false ao ativar
 - `src/types/statusMix.ts:12` — StatusMix.ativadoManualmente: flag booleana de override manual
 - Contraexemplo: `src/components/CustomerMixTab.tsx:125-126` — condição procura `!sm.ativadoManualmente && sm.status === 'inativo'` (`ativadoManualmente=FALSE`), oposto da regra descrita; e `:191-207` não reseta `ativado_manualmente` para false no toggle para ‘ativo’
- **Regressão se:** a lógica da flag `ativadoManualmente` for revertida ou removida. Produtos desativados manualmente reativam no próximo carregamento de pedido; preferências do cliente são sobrescritas.

Somente usuários backoffice criam, atualizam ou deletam listas de preço

A função `edge listas-preco-v2` verifica `user.tipo === 'backoffice'` para POST, PUT e DELETE. Vendedores não podem executar essas operações. GET é permitido para todos os usuários autenticados.

- **Tipo:** business-rule
- **Evidência:**
 - `supabase/functions/listas-preco-v2/index.ts:255, 367, 474` — POST/PUT/DELETE: `if (user.tipo !== 'backoffice') throw error`; GET (`:173-251`) sem check de tipo
- **Regressão se:** o check de role for removido. Vendedores modificam listas de preço, causando corrupção de dados e cálculos de comissão inconsistentes.

Decisões de arquitetura

Fotos de produto fora do endpoint de lista, carregadas sob demanda com cache em memória

O endpoint de lista (`action=list`) omite a coluna `foto` do `SELECT` no banco e explicitamente seta `foto: undefined` na resposta. Fotos são carregadas sob demanda apenas quando o componente `ProductThumbnail` entra no viewport (via `IntersectionObserver`), chamando `api.getById('produtos', id)`. O `fotoCache` (`Map`) evita re-fetch. Porém, `ProductsListPage.tsx:498-506` contém código morto que testa `produto.foto`, que nunca será verdadeiro, pois o campo é sempre `undefined` na resposta da lista.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/functions/produtos-v2/index.ts:167-171` — `SELECT` omite a coluna `foto`; comentário: ‘fotos são base64 inline e o payload chegava a ~31MB / ~19s’
 - `src/components/ProductsListPage.tsx:36-73` — `ProductThumbnail`: `fotoCache` `Map`, `IntersectionObserver` com `rootMargin 150px`, chama `api.getById('produtos', id)` na interseção
 - `src/components/ProductsListPage.tsx:304` — `action` de `list` explicitamente seta `foto: undefined` na resposta
 - Contraexemplo (código morto): `src/components/ProductsListPage.tsx:498-506` — testa `{produto.foto ? ...}` como se `foto` pudesse ser `truthy` na resposta de lista; sempre cai para `ProductThumbnail`
- **Regressão se:** a coluna `foto` for incluída na query de lista ou o `IntersectionObserver` removido. Resposta da lista explode para 31MB+, causando timeouts de browser e fallback a mock.

Deleção de produto é soft-delete (parcial no domínio)

A `action` de `delete` em `produtos-v2` (tabela `produto`) corretamente **não** remove linhas; usa `soft-delete` com `deleted_at` timestamp, `situacao='Excluído'` e `ativo=false`, filtrando com `.is('deleted_at', null)`. Porém, a regra **não** é universalmente aplicada no domínio: `listas-preco-v2` usa `hard-delete` `.delete()` em `listas_preco`, `produtos_listas_precos` e `listas_preco_comissionamento` (nas operações `POST`, `PUT` e `DELETE`), pois essas tabelas não possuem coluna `deleted_at`.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/functions/produtos-v2/index.ts:712-737` — `delete`: `.update({ deleted_at, situacao: 'Excluído', ativo: false })`; sem `.delete()`
 - `supabase/functions/produtos-v2/index.ts:196, 368` — queries de `list/get`: `.is('deleted_at', null)` filtra `soft-deletados`
 - `supabase/functions/status-mix-v2/index.ts:37-38` — tabela de usuário também usa `soft-delete`: `.is('deleted_at', null)`
 - Contraexemplo: `supabase/functions/listas-preco-v2/index.ts:479-481 (DELETE)` — usa `.delete()` direto em `listas_preco`, não `soft-delete`
- **Regressão se:** `hard-delete` for usado no lugar. Dados históricos de pedido referenciam IDs de produto inexistentes; trilha de auditoria é perdida.

Deleção de lista de preço faz cascade para produtos e faixas de comissão

Ao deletar uma lista via a função `edge` `listas-preco-v2`, as linhas em `produtos_listas_precos` e `listas_preco_comissionamento` são deletadas manualmente pela `edge function` (`:479-480`) antes de deletar o registro master, e constraints FK com `ON DELETE CASCADE` dão proteção de fallback. Porém, RLS policies (`listas_preco allow_all`, `listas_preco_comissionamento allow_all`, `produtos_listas_precos`

`allow_all`) permitem que qualquer usuário autenticado faça DELETE dessas tabelas diretamente, contornando a `edge function` e sua ordem de deleção manual.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/schema_baseline.sql` (FK constraints) — `fk_listas_preco: ON DELETE CASCADE;`
`produtos_listas_precos_lista_preco_id_fkey: ON DELETE CASCADE`
 - `supabase/functions/listas-preco-v2/index.ts:479-481` — DELETE: deleta manualmente `produtos_listas_precos`, `listas_preco_comissionamento`, depois `listas_preco`
 - Contraexemplo: `supabase/schema_baseline.sql:1183, 1184, 1235` — RLS policies `allow_all` permitem DELETE direto contornando a `edge function`
- **Regressão se:** o cascade for desabilitado ou os deletes manuais forem pulados. Registros órfãos de produto-lista e faixas permanecem, consumindo storage e quebrando integridade referencial.

Query de lista de produtos limitada a 2000 linhas (para evitar `statement_timeout`)

O endpoint de lista de produtos (`action='list'`) aplica `LIMIT 2000` para evitar `statement_timeout` (60s) do Postgres em produção. O endpoint retorna até 2000 produtos ordenados por `descricao` (ascendente). Esta ordenação é alfabética, não cronológica (não por `created_at`). A UI assume que todos os produtos cabem em 2000 ou implementa filtragem client-side. A query original usava `ORDER BY created_at DESC` e causava timeouts; foi substituída por `ORDER BY descricao` ascendente.

- **Tipo:** arch-decision
- **Evidência:**
 - `supabase/functions/produtos-v2/index.ts:167-198` — comentário: ‘LIMIT 2000 + ORDER BY descricao: a query original (SELECT * sem LIMIT + ORDER BY created_at DESC) estava estourando o `statement_timeout` de 60s’; aplica `.limit(2000)`
 - `supabase/functions/produtos-v2/index.ts:166-171` — comentário do sintoma: ‘dropdown de produtos no `PriceListFormPage` caía em fallback mock silencioso’
 - Contraexemplo: `supabase/functions/produtos-v2/index.ts:776-818` (`action list_import_logs`) usa `limit` variável (1-1000) de query params, provando que o padrão `LIMIT 2000` fixo não é uniforme entre todas as actions do endpoint
- **Regressão se:** o LIMIT for removido. Query retorna >2000 linhas, excede o `statement_timeout`, e cai em mock silenciosamente, ocultando produtos reais.

Contrato — Relatórios

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Business rules

Classificação ABC baseada em acumulado anterior

A classificação de clientes/produtos em Curva A/B/C é determinada pelo percentual acumulado **ANTERIOR** à entrada do cliente, não posterior. Curva A: `acumulado anterior < 80%`; Curva B: `80% ≤ acumulado anterior < 95%`; Curva C: `acumulado anterior ≥ 95%`.

- **Tipo:** business-rule
- **Evidência:**

- `/src/components/ABCCurveCard.tsx:117-132` — calcula `acumuladoAnterior` ANTES de acumular; classifica pelo `acumuladoAnterior`, não pelo final.
- `/src/components/CustomerABCReportPage.tsx:296-310` — classifica ANTES de acumular; comentário explícito “CORREÇÃO: Classificar ANTES de acumular o percentual atual”.
- **Regressão se:** a lógica mudar para classificar pelo `acumulado DEPOIS` de somar o percentual do cliente atual — clientes borderline (perto de 80% e 95%) mudarão de curva.

Parâmetro `include_itens` ativa retorno de itens do pedido na API (modo LIST)

Quando `api.get('vendas', { params: { include_itens: true } })` é chamado (modo LIST), a Edge Function `pedido-venda-v2` retorna cada pedido com um array `produtos`. Porém, quando `api.getById('vendas', id)` é chamado (modo GET by ID), a Edge Function **SEMPRE** retorna os produtos, independente de qualquer parâmetro. Sem o parâmetro `include_itens` no modo LIST, o array `produtos` não é incluído, e componentes como `ProductABCReportPage` buscam itens diretamente do Supabase como alternativa.

- **Tipo:** business-rule
- **Evidência:**
 - `/src/services/api.ts:1953-1955` — mapeamento `params.include_itens=true` → `baseParams.include_itens='true'` passado à Edge Function.
 - `/supabase/functions/pedido-venda-v2/index.ts:156-228` — GET by ID sempre retorna produtos.
 - `/supabase/functions/pedido-venda-v2/index.ts:287-319` — GET LIST só retorna produtos com a flag `include_itens`.
- **Regressão se:** remover `include_itens` da Edge Function — relatórios que dependem de itens detalhados (Mix, Solicitado/Faturado) ficarão vazios.

Status Mix determina ativação de produto para cliente

Cada combinação (cliente, produto) pode ter um `status_mix` armazenado no banco com valores ‘ativo’ ou ‘inativo’ somente (imposto por CHECK constraint). Ao exibir o status do mix, o sistema trata registros ausentes na tabela `status_mix` como ‘sem_cadastro’ (estado virtual, não armazenado). Essa lógica de três estados (ativo/inativo/sem_cadastro) indica se um produto está ativo no mix do cliente, previamente marcado inativo, ou nunca cadastrado. A camada de banco impede estritamente qualquer valor fora de ‘ativo’ e ‘inativo’ via constraint SQL.

- **Tipo:** business-rule
- **Evidência:**
 - `/supabase/migrations/137_status_mix_table.sql:3-14` — tabela `status_mix` com `UNIQUE(cliente_id, produto_id)` e `CHECK status IN ('ativo', 'inativo')`.
 - `/src/components/RelatorioMixCliente.tsx:270-291` — busca `status_mix` e classifica como ‘ativo’/‘inativo’, ou ‘sem_cadastro’ (virtual) se não encontrado.
- **Regressão se:** retirar a tabela `status_mix` — o sistema não conseguirá rastrear quais produtos estão ativos para cada cliente.

Relatórios paginam automaticamente até carregar TODOS os dados

Em vez de limitar a resultados de uma página, os relatórios (clientes, vendas, produtos) fazem loop pelas páginas retornadas pela Edge Function até `totalPages`, concatenando resultados. Máximo 100 itens/página na API, mas o relatório coleta tudo.

- **Tipo:** business-rule
- **Evidência:**
 - `/src/components/CustomerABCReportPage.tsx:68–78` — carregamento com paginação: loop com `pagina <= totalPaginas`; concatena todos em `todosClientes`.
 - `/src/services/api.ts:1962–1973` — mesmo padrão para vendas: “percorremos TODAS as páginas e juntamos tudo”, em vez de usar `limit=1000` que retornava apenas 100 mais recentes.
- **Regressão se:** remover paginação — relatórios exibirão apenas os primeiros 100 registros; em bases com 960+ clientes, a análise ABC ficará incompleta.

ROI é calculado para os últimos 365 dias (filtro client-side)

Relatório ROI por cliente executa um filtro **client-side** para limitar vendas aos últimos 365 dias, sem UI para período customizado. Contudo, a restrição **não é enforçada server-side**: a API é chamada sem parâmetros de data, recebendo todos os pedidos do cliente, e apenas depois faz o filtro em memória. A API suporta `dataInicio/dataFim` mas não os utiliza aqui. Há risco de exposição se o filtro client-side falhar ou for contornado.

- **Tipo:** business-rule
- **Evidência:**
 - `/src/components/RelatorioROICliente.tsx:116–123` — `dataInicio = 365` dias atrás; filtra vendas `>= dataInicio` em memória.
 - `/src/components/RelatorioROICliente.tsx:120` — `api.get("vendas", { params: { clienteId: clienteSelecioneado.id } })` não passa `dataInicio/dataFim`.
- **Regressão se:** deixar período customizado afetar o cálculo de ROI — o indicador ficará inconsistente (mesmo cliente com ROI diferente em períodos distintos).

Exportação CSV inclui BOM UTF-8 para compatibilidade Excel (com uma exceção)

A maioria dos exports para CSV (12 de 13 casos) adiciona BOM (Byte Order Mark) UTF-8 no início do arquivo e usa `charset=utf-8` para garantir leitura correta de caracteres acentuados no Excel e LibreOffice. **Exceção conhecida:** `SellerCommissionsPage.tsx` viola esse padrão ao exportar CSV sem BOM, criando inconsistência na aplicação da regra.

- **Tipo:** business-rule
- **Evidência:**
 - `/src/components/CustomerABCReportPage.tsx:414` — `new Blob(['' + csvContent], { type: 'text/csv;charset=utf-8;' })`.
 - `/src/components/RelatorioMixCliente.tsx:389` — mesmo padrão de BOM na exportação Mix.
 - `/src/components/SellerCommissionsPage.tsx:221` — `new Blob([csvContent], { type: "text/csv;charset=utf-8;" })` — SEM BOM (contraexemplo).
- **Regressão se:** remover o BOM — a acentuação será corrompida no Excel (“São Paulo” vira gibberish em algumas máquinas).

Architecture decisions

ABC de Produtos carrega itens direto de `pedido_venda_produtos` via Supabase (RLS)

Para `ProductABCReportPage` **especificamente**, os itens SÃO buscados diretamente da tabela `pedido_venda_produtos` via cliente Supabase (com JWT injetado), em lotes de 200, respeitando RLS — em vez de contar com `include_itens` da API. Porém, outros relatórios (ex.: `SolicitadoFaturadoReportPage`) ainda

usam `include_itens` da API, indicando que essa mudança arquitetural **não foi universalmente aplicada** a todos os relatórios de produto.

- **Tipo:** arch-decision
- **Evidência:**
 - `/src/components/ProductABCReportPage.tsx:55-76` — query direta: `supabase.from('pedido_venda_produtos').select(...).in('pedido_venda_id', batch)` em lotes de 200.
 - `/src/components/ChangelogPage.tsx:75` — nota de correção: “dados agora carregam corretamente — client Supabase injeta token JWT, respeitando RLS”.
 - `/src/components/SolicitadoFaturadoReportPage.tsx:70` — usa `api.get('vendas', { params: { include_itens: true } })` (padrão divergente).
- **Regressão se:** RLS policies forem removidas ou o token não for injetado — a query retornará itens de pedidos de outros usuários ou nenhum resultado.

Contrato — Conta Corrente

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Este contrato cobre o domínio **Conta Corrente** (compromissos de `conta_corrente_cliente`, pagamentos de `pagamento_acordo_cliente`, categorias de `categorias_conta_corrente`) e os artefatos correlatos de **Condições/Formas de Pagamento** que alimentam este domínio.

Notação de verdict: - Regras **confirmed** estão impostas de ponta a ponta. - Regras **partial** têm o enunciado ajustado para refletir a imposição real (limites/brechas conhecidas). Leia o enunciado corrigido e a nota de brecha antes de assumir garantia total.

Invariants

Tipo de Compromisso deve ser ‘Investimento’ ou ‘Ressarcimento’

Tipo: invariant · **Verdict:** confirmed

Compromissos (`conta_corrente_cliente`) aceitam apenas dois tipos: `'investimento'` ou `'ressarcimento'`. O valor é normalizado para lowercase na criação e validado em tempo de operação. Imposto em múltiplas camadas: RPC (create/update), handler HTTP e CHECK constraint no banco (defesa em profundidade — rejeita valor inválido mesmo com RLS permissiva).

Evidência: - `supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:200-202` — `create_conta_corrente_v2: IF p_tipo_compromisso IS NULL OR LOWER(p_tipo_compromisso) NOT IN ('investimento', 'ressarcimento')` - `supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:390-392` — `update_conta_corrente_v2: mesmo padrão` - `supabase/functions/conta-corrente-v2/index.ts:595-598` — validação no handler HTTP com mensagem específica - CHECK constraint `conta_corrente_cliente_tipo_compromisso_check (schema_baseline)` força `tipo_compromisso = ANY (ARRAY['investimento', 'ressarcimento'])`

Regressão se: o sistema aceitar um tipo diferente (ex.: ‘Empréstimo’, ‘Serviço’) ou deixar o tipo NULL em novo compromisso.

Valor do Compromisso deve ser maior que zero

Tipo: invariant · **Verdict:** partial

Enunciado corrigido: A validação é imposta na CRIAÇÃO de compromissos (SQL rejeita valor ≤ 0). Porém, na ATUALIZAÇÃO, há falha no handler TypeScript: valores iguais a zero são silenciosamente convertidos para NULL ao serem enviados à RPC, contornando a validação. A regra é parcialmente implementada: **criar protegido, atualizar vulnerável a bypass com valor = 0**.

Evidência: - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:192-194 — create_conta_corrente_v2: IF p_valor IS NULL OR p_valor ≤ 0 THEN RAISE EXCEPTION - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:386-388 — update_conta_corrente_v2: IF p_valor IS NOT NULL AND p_valor ≤ 0 THEN (NULL passa) - supabase/functions/conta-corrente-v2/index.ts:585-587 — handler valida valor > 0 na criação

Brecha (contraexemplo): supabase/functions/conta-corrente-v2/index.ts:695 — PUT com valor=0 usa body.valor ? Number(body.valor) : null; 0 é falsy → converte para null → contorna a validação da RPC.

Regressão se: o sistema permitir criar compromisso com valor=0, negativo ou NULL sem rejeição.

Status de Compromisso é calculado dinamicamente a partir de pagamentos

Tipo: invariant · **Verdict:** partial

Enunciado (usar com ressalva — verificação inconclusiva): O status de um compromisso ('Pendente', 'Pago Parcialmente', 'Pago Integralmente') é calculado dinamicamente comparando o valor total com o valor_pago atual, não sendo armazenado diretamente na tabela; muda conforme novos pagamentos são adicionados. A verificação automática deste ponto foi inconclusiva — tratar como observado no código, não como garantia end-to-end.

Evidência: - supabase/migrations/067_rpc_conta_corrente_v2.sql:309-313 — get_conta_corrente_v2: CASE WHEN SUM(pac.valor_pago)=0 THEN 'Pendente' WHEN $\geq$ valor THEN 'Pago Integralmente' ELSE 'Pago Parcialmente' - supabase/migrations/067_rpc_conta_corrente_v2.sql:113-117 — list_conta_corrente_v2 usa a mesma lógica de cálculo dinâmico - src/types/contaCorrente.ts:47-52 — interface StatusCompromisso define os três valores possíveis

Regressão se: o status for armazenado em coluna separada sem sincronização com pagamentos, ou o cálculo não comparar corretamente valor vs valor_pago.

Soma de pagamentos não pode exceder valor do compromisso

Tipo: invariant · **Verdict:** partial

Enunciado corrigido: A regra é imposta APENAS via funções RPC e código de aplicação, não via constraints/triggers de banco. A validação (soma de todos os pagamentos + novo não pode ultrapassar o valor do compromisso) pode ser contornada com INSERT direto na tabela. Uma implementação robusta exigiria CONSTRAINT CHECK ou TRIGGER BEFORE INSERT/UPDATE replicando a lógica.

Evidência: - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:631-639 — create_pagamento_conta_corrente_v2: SELECT SUM(valor_pago) e IF (total + novo) $>$ valor THEN

RAISE - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:781-791 — update_pagamento_conta_corrente_v2: mesmo comportamento, excluindo o pagamento atual do cálculo

Brecha (contraexemplo): supabase/migrations/068_rls_conta_corrente.sql:77-81 — policy pagamento_acordo_cliente_insert_policy permite INSERT direto via Supabase SDK sem passar pela RPC; não há trigger que reforce a validação no banco.

Regressão se: o sistema criar pagamento cuja soma ultrapasse o valor do compromisso (overpayment).

Nomes de categorias devem ser únicos (case-insensitive) entre categorias ativas

Tipo: invariant · **Verdict:** partial

Enunciado corrigido: O índice UNIQUE existe e força a unicidade em nível de banco (`LOWER(TRIM(nome))` WHERE `deleted_at IS NULL`), e as RPCs validam antes de inserir. Porém, a RLS policy de INSERT usa `WITH CHECK (true)`, permitindo inserções diretas que contornam a validação de negócio da RPC — embora o índice ainda impeça duplicatas de fato, a camada de aplicação não garante que toda inserção passe pela validação.

Evidência: - supabase/migrations/021_create_categorias_conta_corrente.sql:21-23 — `CREATE UNIQUE INDEX idx_categorias_conta_corrente_nome_unique ON LOWER(TRIM(nome)) WHERE deleted_at IS NULL`

Brecha (contraexemplo): supabase/migrations/021_create_categorias_conta_corrente.sql:55-59 — RLS de INSERT com `WITH CHECK (true)` permite `supabase.from('categorias_conta_corrente').insert({...})` direto, contornando `create_categorias_conta_corrente_v2`.

Regressão se: o sistema criar duas categorias ativas com o mesmo nome (ou diferindo só em case) sem erro de constraint.

Flag Parcelamento é 'true' se quantidade de parcelas > 1

Tipo: invariant · **Verdict:** partial

Enunciado corrigido: Na tabela `Condicao_De_Pagamento`, o campo `Parcelamento` é `NOT NULL BOOLEAN`: `false` para à vista (1 parcela, `prazo=0`) e `true` quando há 2+ parcelas. É calculado automaticamente APENAS em create e quando `prazoPagamento` é fornecido em update. Updates que modificam outros campos (`condicaoCredito`, `descontoExtra`, `valorMinimo`) sem fornecer `prazoPagamento` deixam `Parcelamento` desatualizado.

Evidência: - supabase/functions/condicoes-pagamento-v2/index.ts:304-336 — create: `Parcelamento = quantidadeParcelas > 1` - supabase/functions/condicoes-pagamento-v2/index.ts:387-391 — update: recalcula após parse

Brecha (contraexemplo): supabase/functions/condicoes-pagamento-v2/index.ts:386-392 — `Parcelamento` só recalcula se `body.prazoPagamento !== undefined`; update de apenas `{id, condicaoCredito}` não recalcula.

Regressão se: `Parcelamento` for setado manualmente para `false` com 2 parcelas, ou vice-versa.

Todo compromisso deve estar vinculado a um cliente válido

Tipo: invariant · **Verdict:** partial

Enunciado corrigido: cliente_id é obrigatório em conta_corrente_cliente e, QUANDO CRIADO VIA RPC (create_conta_corrente_v2), deve referenciar um cliente existente com deleted_at IS NULL. Porém, a validação de deleted_at IS NULL está APENAS na RPC, não na tabela. Inserts diretos via RLS podem passar com cliente deleted_at IS NOT NULL desde que o cliente_id exista (a FK só valida existência).

Evidência: - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:184-214 — create_conta_corrente_v2: IF p_cliente_id IS NULL + SELECT ... WHERE cliente_id = p_cliente_id + IF NOT FOUND RAISE - supabase/functions/conta-corrente-v2/index.ts:579-581 — handler: IF !clienteId RAISE

Brecha (contraexemplo): supabase/migrations/068_qls_conta_corrente.sql:28-33 — RLS de INSERT com WITH CHECK (true); INSERT direto com cliente_id de cliente deletado é aceito pela FK.

Regressão se: o sistema criar compromisso sem cliente_id ou com cliente_id inexistente em cliente.

Business rules

Título do Compromisso é obrigatório e tem mínimo de 2 caracteres

Tipo: business-rule · **Verdict:** partial

Enunciado corrigido: A regra é imposta apenas via camada de aplicação (RPC e handler HTTP), mas não está protegida a nível de banco. Pode ser contornada por INSERT/UPDATE direto na tabela, pois as RLS usam WITH CHECK (true) e a tabela conta_corrente_cliente não tem CHECK constraint sobre título.

Evidência: - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:196-198 — create_conta_corrente_v2: IF p_titulo IS NULL OR LENGTH(TRIM(p_titulo)) < 2 - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:382-384 — update_conta_corrente_v2: mesma validação - supabase/functions/conta-corrente-v2/index.ts:588-590 — handler: titulo obrigatório

Brecha (contraexemplo): supabase/migrations/068_qls_conta_corrente.sql:29-33 — INSERT com WITH CHECK (true); tabela sem CHECK em título.

Regressão se: o sistema permitir criar compromisso sem título ou com título vazio/único caractere.

Forma de Pagamento é obrigatória para pagamentos

Tipo: business-rule · **Verdict:** partial

Enunciado corrigido: Pagamentos (pagamento_acordo_cliente) requerem, na CRIAÇÃO: forma_pagamento (texto com mín. 2 chars) OU forma_pagamento_id (ID válido em ref_forma_pagamento) — pelo menos um, validado na RPC. Na ATUALIZAÇÃO, ambos podem ser omitidos, mantendo valores anteriores sem revalidação obrigatória de tamanho. A coluna forma_pagamento é NOT NULL na tabela; forma_pagamento_id é nullable.

Evidência: - supabase/migrations/075_add_forma_pagamento_id_to_pagamentos_rpc.sql:57-81 — create_pagamento_conta_corrente_v2: se forma_pagamento_id, busca nome; senão forma_pagamento obrigatório - supabase/functions/conta-corrente-v2/index.ts:247-248 — handler rejeita se ambos vazios

Brecha (contraexemplo):

supabase/migrations/075_add_forma_pagamento_id_to_pagamentos_rpc.sql:268-289 — update_pagamento_conta_corrente_v2 permite UPDATE sem forçar validação se ambos forem null/undefined.

Regressão se: o sistema criar pagamento sem forma_pagamento e sem forma_pagamento_id válido.

Categoria é campo opcional em compromissos e pagamentos

Tipo: business-rule · **Verdict:** confirmed

Tanto compromissos quanto pagamentos podem ser criados/atualizados sem categoria (categoria_id pode ser NULL). Quando fornecida, deve ser UUID válido referenciando categorias_conta_corrente — validado por FK (ON DELETE SET NULL) no banco. Colunas nullable em ambas as tabelas; RPCs declaram p_categoria_id UUID DEFAULT NULL.

Evidência: - supabase/migrations/075_add_forma_pagamento_id_to_pagamentos_rpc.sql:20-21 — p_categoria_id UUID DEFAULT NULL em create_pagamento_conta_corrente_v2 - src/types/contaCorrente.ts:44,63 — categoriaId opcional em Compromisso e Pagamento - supabase/migrations/073_fix_categoria_id_type_to_uuid.sql:1-10 — categoria migrada para UUID (não BIGINT)

Regressão se: o sistema forçar categoria obrigatória ou rejeitar NULL, ou aceitar UUID inexistente sem validação de FK.

Nome de categoria deve ter no mínimo 2 caracteres

Tipo: business-rule · **Verdict:** partial

Enunciado corrigido: Ao criar categoria (categorias_conta_corrente), o nome deve ter no mínimo 2 caracteres não-vazios, garantido por: validação HTTP (POST), validação RPC (SECURITY DEFINER) e CHECK constraint no PostgreSQL. Porém, a RLS não valida (WITH CHECK (true)) — apenas o CHECK constraint protege contra INSERT/UPDATE diretos via cliente autenticado.

Evidência: - supabase/migrations/021_create_categorias_conta_corrente.sql:18-19 — CONSTRAINT categorias_conta_corrente_nome_check CHECK (LENGTH(TRIM(nome)) >= 2) - supabase/functions/categorias-conta-corrente-v2/index.ts:171-173 — handler: IF (!body.nome || body.nome.trim().length < 2)

Brecha (contraexemplo): supabase/migrations/021_create_categorias_conta_corrente.sql:55-59 — RLS de INSERT com WITH CHECK (true); proteção real depende do CHECK constraint.

Regressão se: o sistema criar categoria com nome vazio ou único caractere.

Intervalo de parcelas é gerado dinamicamente a partir de string de prazos

Tipo: business-rule · **Verdict:** partial

Enunciado corrigido: Em condições de pagamento, o intervalo_parcela é calculado parseando a string prazoPagamento (formato '10/20/30'): quantidade de parcelas = quantidade de números; prazo_pagamento = último número. A descrição regenera automaticamente quando parcelas/prazos mudam, EXCETO se um campo

descricao for explicitamente fornecido no payload de update (override manual tem prioridade). No frontend padrão, descricao não é enviada em updates, então a regeneração funciona na prática.

Evidência: - supabase/functions/condicoes-pagamento-v2/index.ts:86-111 — processarPrazoPagamento: split por '/', parse floats, quantidadeParcelas=length, prazo=último - supabase/functions/condicoes-pagamento-v2/index.ts:114-133 — gerarDescricao: usa intervaloParcela.join('/'), ex.: '10/15/20 dias' - supabase/functions/condicoes-pagamento-v2/index.ts:406-440 — update: regenera descrição se prazoPagamento muda

Brecha (contraexemplo): supabase/functions/condicoes-pagamento-v2/index.ts:407-408 — PUT com {id, prazoPagamento, descricao: "override"} congela a descrição; updates subsequentes sem reenviar descricao não regeneram.

Regressão se: o sistema não regenerar a descrição quando o intervalo_parcela muda, gerando descrições desincronizadas.

Data do compromisso é obrigatória

Tipo: business-rule · **Verdict:** confirmed

Ao criar compromisso, data (DATE) é obrigatória e não pode ser NULL. Imposto em 3 camadas: handler HTTP, RPC create_conta_corrente_v2 e coluna data date NOT NULL na tabela. Única via de criação é POST /conta-corrente-v2, que passa por todas as validações.

Evidência: - supabase/migrations/072_add_categoria_id_to_all_conta_corrente_rpc.sql:188-190 — create_conta_corrente_v2: IF p_data IS NULL THEN RAISE - supabase/functions/conta-corrente-v2/index.ts:582-584 — handler: data/dataCompromisso/data_compromisso obrigatória

Regressão se: o sistema criar compromisso sem data.

Data de pagamento é obrigatória

Tipo: business-rule · **Verdict:** partial

Enunciado corrigido: Para pagamentos de CONTA CORRENTE (pagamento_acordo_cliente): data_pagamento é obrigatória e validada na RPC e no handler. Para pagamentos de COMISSÃO (pagamentos_comissao): data_pagamento é preenchida automaticamente com CURRENT_DATE, não sendo obrigatório fornecê-la na criação. A regra vale para conta corrente, não para pagamentos em geral.

Evidência: - supabase/migrations/075_add_forma_pagamento_id_to_pagamentos_rpc.sql:53-55 — create_pagamento: IF p_data_pagamento IS NULL THEN RAISE - supabase/functions/conta-corrente-v2/index.ts:244-246 — handler: dataPagamento obrigatória

Brecha (contraexemplo): supabase/migrations/084_comissoes_rpc.sql — create_pagamento_comissao_v2 não recebe data_pagamento; usa CURRENT_DATE.

Regressão se: o sistema criar pagamento de conta corrente sem data_pagamento.

Arch decisions

Forma de pagamento não pode ser deletada se tem condições vinculadas

Tipo: arch-decision · **Verdict:** confirmed

Ao deletar uma forma_pagamento (ref_forma_pagamento), o sistema verifica se há Condicao_De_Pagamento usando essa forma; havendo qualquer vínculo, a deleção é rejeitada. Proteção dupla: (1) SELECT no código antes do DELETE; (2) FK sem ON DELETE CASCADE (padrão RESTRICT) no banco.

Evidência: - supabase/functions/formas-pagamento-v2/index.ts:271-285 — delete: SELECT Condicao_De_Pagamento WHERE forma_pagamento_id = id LIMIT 1; se encontrar, nega - FK Condicao_De_Pagamento_forma_pagamento_id_fkey (schema) sem ON DELETE CASCADE → banco rejeita o DELETE

Regressão se: o sistema deletar forma_pagamento em uso em alguma Condicao_De_Pagamento.

Categorias usam soft delete (deleted_at)

Tipo: arch-decision · **Verdict:** partial

Enunciado corrigido: Categorias em categorias_conta_corrente são soft-deleted via UPDATE (SET deleted_at = NOW()) por RPCs SECURITY DEFINER. Não há policy RLS explícita para DELETE — operações DELETE são bloqueadas por padrão (deny implícito). Índices e queries filtram por deleted_at IS NULL. A proteção não vem de uma DELETE policy com deleted_at, mas da AUSÊNCIA de DELETE policy combinada ao uso exclusivo de UPDATE para soft-delete.

Evidência: - supabase/migrations/021_create_categorias_conta_corrente.sql:16,22,25,27 — coluna deleted_at TIMESTAMPTZ; índices filtram WHERE deleted_at IS NULL - supabase/migrations/021_create_categorias_conta_corrente.sql:53 — SELECT policy USING (deleted_at IS NULL) - Soft-delete via UPDATE em 022_rpc_categorias_conta_corrente_v2.sql (SET deleted_at = NOW())

Ressalva (contraexemplo): supabase/migrations/021_create_categorias_conta_corrente.sql:44-73 — não há CREATE POLICY FOR DELETE; a ausência de policy é a proteção.

Regressão se: categorias forem fisicamente deletadas em vez de soft delete, perdendo histórico.

Apenas backoffice pode criar/atualizar/deletar categorias

Tipo: arch-decision · **Verdict:** confirmed

Operações de escrita (POST/PUT/DELETE) em categorias-conta-corrente-v2 rejeitam usuários com tipo='vendedor'; apenas backoffice gerencia categorias. Validação if (user.tipo !== 'backoffice') throw ocorre ANTES de qualquer RPC; user.tipo vem de validateJWT() (consulta tabela user), sem bypass. GET não é restrito por tipo (coerente — regra é sobre escrita).

Evidência: - supabase/functions/categorias-conta-corrente-v2/index.ts:168-170 — POST create: IF user.tipo !== 'backoffice' THEN RAISE - supabase/functions/categorias-conta-corrente-v2/index.ts:193-195 — PUT update: mesma checagem - supabase/functions/categorias-conta-corrente-v2/index.ts:216-218 — DELETE: mesma checagem

Regressão se: um vendedor conseguir criar/editar/deletar categorias.

Contrato — Convenções de Dados & Padrões

Regras e invariantes verificadas no código. Não editar sem cotejar impacto.

Este contrato documenta convenções de modelagem, RPC e SQL do domínio de dados. Cada regra reflete o estado **verificado** do código (com correções aplicadas onde o veredito foi parcial).

Invariants

Padrão Soft-Delete (deleted_at) em Tabelas Principais — com violações conhecidas

Enunciado. Soft-delete com deleted_at TIMESTAMPTZ está implementado para tabelas principais (cliente, pedido_venda, natureza_operacao, produto, dados_vendedor, frete_logistica, grupos_redes, user, categorias_conta_corrente etc.). Regras verificadas:

1. Queries **não devem** retornar/juntar com registros onde deleted_at IS NOT NULL.
2. Delete de entidade principal é feito por UPDATE ... SET deleted_at = NOW(), nunca DELETE FROM.
3. Tabelas de associação **não** têm deleted_at (hard-delete permitido).

Violações reais existentes (não conformidade parcial): - list_conta_corrente_v2 / get_conta_corrente_v2 / update_conta_corrente_v2 fazem INNER/LEFT JOIN com cliente **sem** filtrar c.deleted_at → clientes soft-deletados aparecem em conta corrente. - list_pedido_venda_v2 filtra pv.deleted_at IS NULL mas **não** filtra cliente/natureza_operacao deletados. - Múltiplos return statements em 007_rpc_clientes_v2.sql (linhas 227, 359, 440, 742, 835) não filtram deleted_at (risco semântico baixo: o registro foi validado antes da operação).

Tipo. invariant

Evidência. - supabase/schema_baseline.sql:41,76,210,290,318,374,517 (colunas deleted_at em categorias_conta_corrente, cliente, dados_vendedor, frete_logistica, grupos_redes, natureza_operacao, produto) - supabase/migrations/025_fix_delete_cliente_v2_ambiguous_cliente_id.sql:52-61 (UPDATE c SET deleted_at = NOW()) - supabase/migrations/044_rpc_pedido_venda_v2.sql:903-908 (soft-delete de pedido) - supabase/migrations/007_rpc_clientes_v2.sql:590-591,630,814-815 (filtro WHERE c.deleted_at IS NULL) - Contraexemplo: supabase/migrations/067_rpc_conta_corrente_v2.sql:74 (INNER JOIN public.cliente c ON c.cliente_id = ccc.cliente_id sem c.deleted_at IS NULL)

Regressão se. Uma query não filtrar WHERE deleted_at IS NULL ou uma entidade principal sofrer hard-delete sem preparação prévia — quebra soft-delete auditável e reativação de clientes.

Padrão UPSERT com COALESCE(NULLIF(TRIM(param), ''), tabela.col) — não uniformemente aplicado

Enunciado. INSERT ... ON CONFLICT DO UPDATE deve usar COALESCE(NULLIF(TRIM(param), ''), <alias_tabela>.<coluna>) para campos nullable/string vazia, preservando valores existentes quando o parâmetro é NULL/vazio. **Nunca** usar EXCLUDED.<coluna> como fallback — usar o alias da tabela. Isso evita apagar dados quando um campo não é reenviado.

A regra é válida e comprovada (migration 140 corrigiu perda de dados em ~89 clientes causada por EXCLUDED.telefone), **porém a aplicação é incompleta**: 14 arquivos de migration ainda contêm o padrão violador EXCLUDED.<col>, alguns criados **depois** da migration 140.

Tipo. invariant

Evidência. - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:1-8,132-172 (fix do bug EXCLUDED.telefone; padrão correto telefone = COALESCE(NULLIF(TRIM(p_telefone), ''), cliente_contato.telefone)) - supabase/migrations/007_rpc_clientes_v2.sql:450-460 (NULLIF(TRIM(...), '') antes de INSERT) - supabase/migrations/009_rpc_natureza_operacao_v2.sql:17-20 (nome = COALESCE(NULLIF(TRIM(p_nome), ''), n.nome)) - Contraexemplo: supabase/migrations/130_fix_update_cliente_missing_fields.sql:183-188 (usa fallback EXCLUDED.telefone — viola a regra, criado APÓS a migration 140)

Regressão se. EXCLUDED.<col> for usado no ON CONFLICT DO UPDATE → campos omitidos na requisição são apagados. Se TRIM não for aplicado, espaços em branco são salvos como dados válidos.

Business Rules

Hard-Delete Permitido APENAS em Tabelas de Associação / Componentes de Pedido

Enunciado. Tabelas como pedido_venda_produtos (componentes de pedido), pagamento_acordo_cliente (pagamentos de acordo) e condições_cliente (mapeamento de condições) usam DELETE FROM sem deleted_at. São tabelas de associação, não entidades principais, e são recriadas integralmente ao atualizar o registro pai.

Tipo. business-rule

Evidência. - supabase/migrations/044_rpc_pedido_venda_v2.sql:792-793 (DELETE FROM public_pedido_venda_produtos pvp WHERE pvp_pedido_venda_id = p_pedido_id — substituição integral) - supabase/migrations/067_rpc_conta_corrente_v2.sql:1138-1139 (hard-delete de pagamento_acordo_cliente) - supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:176-177 (DELETE FROM condições_cliente para limpar e recriar)

Regressão se. Hard-delete for estendido para entidades principais (cliente, pedido_venda), ou se pedido_venda_produtos ganhar deleted_at sem atualizar a lógica de UPDATE — quebra histórico de pedidos antigos.

Nomes de Coluna com Acentos e Maiúsculas Mistas exigem Quoted Identifiers

Enunciado. Identificadores com acentos, caracteres especiais (ã, ç etc.) ou padrão maiúsculas/minúsculas mistas **devem** ser entre aspas duplas em queries. Exemplos: "Condição_ID", "Descrição", "cliente_endereço", "ref_tipo_endereco_id_FK". Isso evita o *identifier folding* do PostgreSQL e garante geração de SQL confiável.

Identificadores lowercase sem caracteres especiais (cliente_id, cep, rua, numero) **não** exigem aspas e funcionam corretamente; aspá-los por consistência é opcional. (A regra original dizia “SEMPRE” para todos — impreciso: aplica-se especificamente a identificadores acentuados/especiais/mixed-case.)

Tipo. business-rule

Evidência. - supabase/schema_baseline.sql:20-27 (tabela Condicao_De_Pagamento: "Condição_ID", "Parcelamento", "Condição_de_crédito", "Descrição") - supabase/schema_baseline.sql:106-116 (tabela "cliente_endereço" e coluna "ref_tipo_endereco_id_FK") -

supabase/migrations/140_fix_update_cliente_v2_excluded_upsert.sql:164–172
 ("cliente_endereço".cep, "cliente_endereço".rua quoted em ON CONFLICT DO UPDATE) -
 Contraexemplo: supabase/schema_baseline.sql:888 ("cliente_endereço_pkey" PRIMARY KEY
 (cliente_id) com cliente_id não-quoted funcionando — contradiz o “SEMPRE” universal)

Regressão se. Aspas forem omitidas em identificadores acentuados/mixed-case → PostgreSQL faz fold para lowercase e causa erro ou acesso à coluna errada. Se o app construir queries sem respeitar case desses identificadores, falha ao localizar colunas.

numero_pedido é TEXT, não IDENTITY — Geração Externalizada

Enunciado. A coluna numero_pedido em pedido_venda é TEXT, não BIGINT IDENTITY. O sistema gera números externamente (integração Tiny ERP / sequência customizada), não via AUTO INCREMENT. Valores são strings para permitir prefixos, formatos customizados e integração ERP. O parâmetro p_numero_pedido TEXT DEFAULT NULL é inserido diretamente, sem transformação. Nenhum trigger gera numero_pedido automaticamente. (Veredito: confirmado.)

Tipo. business-rule

Evidência. - supabase/schema_baseline.sql:429 (numero_pedido é TEXT, não IDENTITY) -
 supabase/migrations/044_rpc_pedido_venda_v2.sql:35–36, 83–84, 97 (p_numero_pedido TEXT, atribuído diretamente sem AUTO INCREMENT)

Regressão se. A aplicação usar AUTO INCREMENT/SEQUENCE para numero_pedido → quebra integração ERP que fornece números externamente. Se queries assumirem numero_pedido numérico (::BIGINT), falham em valores prefixados.

Numeração de Migrations com Gaps Permitidos (não sequencial)

Enunciado. Migrations usam tipicamente o padrão NNN (ex.: 001, 007, 044, 067), permitindo gaps históricos (não há exigência de sequência estrita). Gaps podem indicar migrations descartadas ou reserva de blocos. **Duas anomalias violam a uniformidade da regra:** - Prefixo 025 aparece duplicado em dois arquivos distintos. - Arquivos com sufixo alfabético (117a_backup, 118a_backup) existem apenas como referência/rollback manual — **não** são migrations aplicadas pelo sistema.

A ordem de execução pelo Supabase CLI **não está documentada** no projeto como numérica vs. por filename (é suposição, não validada).

Tipo. business-rule

Evidência. - supabase/migrations/ (sequência real com gaps: 001–005, pula 006, 007–030, 041–045, 067–087, 089+; gaps em 031–040, 046–066, 088) - Contraexemplo:
 supabase/migrations/025_cliente_completo_condicoes_conta.sql e
 025_fix_delete_cliente_v2_ambiguous_cliente_id.sql (prefixo 025 duplicado);
 117a_backup_create_grupos_redes_v2_before_vendedor.sql com comentário “NÃO APLICAR EM CONDIÇÕES NORMAIS”

Regressão se. O sistema assumir migrations estritamente sequenciais (001, 002, 003...) → tool de migrate pode parar ao ver 006 faltando. Se a aplicação depender de ordem assumida vs. numérica → inversão de ordem de execução.

Arch Decisions

Colunas Mapper com Sufixo '_nome' — desnormalização (padrão não uniforme)

Enunciado. Colunas de referência denormalizadas (nome_cliente, nome_vendedor, nome_natureza_operacao, nome_empresa_faturamento, nome_condicao_pagamento, nome_marca, nome_tipo_produto) são read-only para performance de SELECT. A desnormalização segue **múltiplos padrões, não uniformemente imposta**:

1. **pedido_venda:** via RPCs (create_pedido_venda_v2, update_pedido_venda_v2) que buscam nomes por LEFT JOIN **antes** de INSERT/UPDATE (segue a intenção da regra).
2. **produto:** INSERT/UPDATE direto na tabela com nomes buscados em SELECTs separados via TypeScript (**viola** o padrão — LEFT JOIN fora do banco).
3. **vendedor/comissão:** via trigger BEFORE INSERT.

Tipo. arch-decision

Evidência. - supabase/schema_baseline.sql:442-451 (pedido_venda: nome_cliente, nome_vendedor, nome_natureza_operacao, nome_empresa_faturamento, nome_condicao_pagamento) - supabase/migrations/044_rpc_pedido_venda_v2.sql:457-462, 480-498 (create_pedido_venda_v2 busca v_vendedor_nome via LEFT JOIN antes do INSERT) - supabase/migrations/025_cliente_completo_condicoes_conta.sql:56-60 (get_cliente_completo_v2 retorna vendedores via JSON_AGG de LEFT JOIN) - supabase/schema_baseline.sql:518-521 (produto: nome_marca, nome_tipo_produto) - Contraexemplo: supabase/functions/produtos-v2/index.ts:519-520 (INSERT com nome_marca/nome_tipo_produto via LEFT JOIN em TypeScript), linhas 602, 615 (UPDATE idem)

Regressão se. A aplicação tentar atualizar nome_cliente diretamente em vez de cliente.nome → mudança ignorada (read-only no RPC). Se a RPC não preencher _nome → NULL em pedidos antigos.

Índices Partial sobre deleted_at IS NULL para Performance — aplicação inconsistente

Enunciado. Índices em colunas frequently-searched incluem WHERE deleted_at IS NULL para criar índices partial (reduz tamanho e melhora query planning para soft-deletes). O projeto implementa isso em ~58% dos índices (57 de 99), especialmente em colunas core (cliente, pedido_venda, produto, dados_vendedor). **A aplicação é inconsistente:** índices em colunas frequently-searched nem sempre incluem a cláusula. A regra não é universalmente imposta.

Tipo. arch-decision

Evidência. - supabase/migrations/003_add_indices.sql:1-30 (CREATE UNIQUE INDEX idx_grupos_redes_nome_unique ... WHERE (deleted_at IS NULL)) - supabase/schema_baseline.sql:1061+ (índices com WHERE deleted_at IS NULL em todo o baseline) - Contraexemplo: supabase/schema_baseline.sql:1032 (CREATE INDEX idx_cliente_situacao ON public.cliente USING btree (ref_situacao_id) — sem WHERE deleted_at IS NULL, violando a convenção numa coluna frequently-searched)

Regressão se. Índices forem dropados/recriados sem a cláusula WHERE → queries que filtram deleted_at IS NULL passam a full table scan; performance de listagem de clientes ativos degrada.

Regression Checklist — Revisão de feature antes do merge

Checklist prático derivado das invariantes mais críticas do [system-contract](#). Rode-o contra QUALQUER feature nova (ou fix) antes de mergear. Cada pergunta aponta para o domínio de onde a invariante vem. Se uma resposta for “não sei”, pare e verifique — não presuma. Ver [known-incidents.md](#) para o que já quebrou.

Como usar: pule as seções que a feature não toca. Para as que toca, TODAS as caixas precisam estar marcadas (ou justificadas em PR) antes do merge.

Clientes / persistência ([clientes.md](#))

- ☐ A feature toca `update_cliente_v2` ou qualquer UPSERT ON CONFLICT? Se sim: o fallback do COALESCE referencia o **valor existente** (`cliente_contato.<col>`), **nunca** EXCLUDED.<col>? (INC de ~89 clientes com observação apagada — migration 140.)
- ☐ Salvar parcialmente (só alguns campos) **preserva** os campos não enviados em vez de sobrescrever com NULL/vazio?
- ☐ A feature mexe em algum mapper de detalhe (`get_cliente_completo_v2` e similares)? Se sim: o mapper retorna os campos `*_nome` (ex.: `grupo_rede_nome`) via join, igual à lista? (migration 141.)
- ☐ Nenhum campo existente foi **omitido** no mapper de detalhe ao adicionar campo novo?

Vendas & emissão Tiny ([vendas-emissao-tiny.md](#))

- ☐ A feature muda o fluxo de emissão? Se sim: sucesso só é declarado e status só muda para “Em aberto” quando o Tiny retorna um `tiny.pedido_id` **real**? (Falso sucesso — V1.71.)
- ☐ Em caso de falha/timeout do Tiny, o status permanece pendente e o erro é propagado ao usuário (sem “falso sucesso”)?
- ☐ A rastreabilidade não depende do número PV-2025-XXXX gerado no navegador (`Date.now`)? Ele é sobrescrito pelo número do Tiny no envio — não usar como chave de busca posterior.

Condições de pagamento ([condicoes-pagamento.md](#))

- ☐ O nome/descrição de condição parcelada usa **todas** as parcelas (ex.: “10/15/20”), não só a última? (V1.70.)
- ☐ O faturamento/emissão gera parcelas a partir de `intervalo_parcela` (source-of-truth), **não** a partir do nome nem de `Prazo_pagamento` scalar?
- ☐ `Quantidade_parcelas == intervalo_parcela.length` continua verdadeiro após CREATE e UPDATE?

Fiscal / Simples / Natureza ([fiscal-simples-natureza.md](#))

- ☐ Se a feature toca detalhe de cliente PJ: `tipoPessoa` é normalizado (acentos, “Pessoa Jurídica”) e o flag Optante Simples **não é omitido** no mapper? (V1.66.)
- ☐ Emissão com regime Simples **não confirmado** (ex.: ReceitaWS instável) é **bloqueada** (D3), em vez de emitir com natureza incorreta? (V1.67.)

Logística SSW ([logistica-ssw.md](#))

- ☐ A feature muda o resolver de status de frete? Se sim: os estados terminais (Entregue / Devolvido / Recusado) continuam **sticky** e não são rebaixados por evento administrativo posterior (ex.: “ANEXADO COMPROVANTE 70”)? (fix frete-logistica-helpers.)
- ☐ O resolver considera o histórico completo de eventos, **não** apenas o último?
- ☐ Alterações não quebram o cron ssw-sweep-hourly / edge ssw-sweep-v1?

Permissões & RLS ([permissoes-rls.md](#))

- ☐ A feature adiciona nova ação de backoffice? Se sim: ela é coberta por uma permissão e respeita o permissionamento existente?
- ☐ A feature toca update-user-v2 / list-users-v2? Se sim: permissoes não voltam a vir null e o redeploy em prod está previsto?
- ☐ A feature **não** introduz uma nova política allow_all que neutralize o RLS (débito já existente — não expandir)?

Edges & backend ([edges-catalog.md](#))

- ☐ Novo edge segue o padrão -v2 e tem verify_jwt configurado corretamente (público vs autenticado conforme o contrato do domínio)?
- ☐ O payload de API consumido pelo frontend em prod **não** mudou sem feature flag?
- ☐ Deploy de edge/RPC parte do main (fonte de verdade) e o commit foi feito **antes** do deploy manual?

Migrations ([arquitetura-stack.md](#), [convencoes-dados.md](#))

- ☐ Precisa de migration? Escreveu o brief em docs/plans/cursor-brief.md com rollback e aguardou confirmação humana?
- ☐ A migration usa a **próxima sequência livre** e **não** preenche gaps existentes?
- ☐ A migration **não** altera arquivos supabase/migrations/*.sql já aplicados?

Convenções de dados ([convencoes-dados.md](#))

- ☐ Feature usa **somente Supabase real** (src/services/*), não src/data/mock*?
- ☐ Campos derivados/*_nome vêm de join, consistentes entre lista e detalhe?
- ☐ Formatos BR (data, moeda, tipoPessoa) normalizados de forma consistente?

Versionamento visível ([arquitetura-stack.md](#))

- ☐ A PR dispara deploy em produção mexendo em src/? Se sim: systemVersion foi bumpado no SidebarUserInfo e o changelog atualizado no tooltip ✨ ?

Regra de ouro

Se a mudança colide com uma invariante registrada e você acredita que a invariante está errada: **corrija o SPEC/contrato primeiro**, com evidência no código, e só então implemente. Wiki e contrato são espelho — não force uma feature a “caber”.

Known Incidents — Mapa incidente → invariante violada

Cada incidente registrado abaixo aponta para a invariante que ele violou (no [system-contract](#)) e mostra como o [regression-checklist](#) o teria pego. Serve para não repetir a mesma classe de erro. Ao investigar um bug novo, confira antes se ele é reincidência de algum destes — reincidência (≥ 2 ocorrências) escala para Deep Work com refator, não fast-fix.

INC — UPSERT com EXCLUDED apagava observação de contato

- **O que aconteceu:** `update_cliente_v2` usava `EXCLUDED.<col>` no fallback do COALESCE durante o UPSERT. Ao salvar um cliente enviando payload parcial, o EXCLUDED (o valor novo/vazio) sobrescrevia a observação de contato existente. Resultado: observação de contato apagada em ~89 clientes ao salvar.
- **Invariante violada:** [clientes.md](#) — o fallback do COALESCE em UPSERT deve referenciar o **valor existente** (`cliente_contato.<col>`), **nunca** `EXCLUDED.<col>`.
- **Correção:** migration 140.
- **Como o checklist pegaria:** seção *Clientes / persistência*, primeira pergunta (“o COALESCE referencia o valor existente, não EXCLUDED?”) — resposta seria “não”, bloqueando o merge.

INC — Falso sucesso na emissão ao ERP

- **O que aconteceu:** a SalesPage (front) declarava “enviado com sucesso” e mudava o status para “Em aberto” mesmo quando o Tiny **não** retornava `tiny.pedido_id`. Pedidos apareciam como emitidos sem existir no ERP.
- **Invariante violada:** [vendas-emissao-tiny.md](#) — só marcar sucesso / mudar status quando o Tiny retorna um ID real (`tiny.pedido_id`).
- **Correção:** V1.71.
- **Como o checklist pegaria:** seção *Vendas & emissão Tiny*, pergunta “sucesso só com `tiny.pedido_id` real?” — reprovaria a lógica que muda status sem checar o ID.

INC — Entrega rebaixada por evento administrativo posterior (SSW)

- **O que aconteceu:** o resolver de status SSW usava apenas o **último** evento. Um evento administrativo tardio (ex.: “ANEXADO COMPROVANTE 70”) chegava depois da entrega e rebaixava o status de Entregue para outro estado.
- **Invariante violada:** [logistica-ssw.md](#) — estados terminais (Entregue / Devolvido / Recusado) são **sticky**; evento administrativo posterior não rebaixa.
- **Correção:** frete-logistica-helpers.
- **Como o checklist pegaria:** seção *Logística SSW*, perguntas “entrega continua sticky?” e “resolver considera o histórico completo, não só o último evento?” — ambas reprovariam o resolver por último-evento.

INC — Nome de condição parcelada mostrava só a última parcela

- **O que aconteceu:** o nome da condição parcelada era gerado com apenas a última parcela (“20 dias”) em vez de todas (“10/15/20”). Usuários não viam as parcelas intermediárias.
- **Invariante violada:** [condicoes-pagamento.md](#) — o nome usa **todas** as parcelas; o faturamento usa `intervalo_parcela` (não o nome).

- **Correção:** V1.70.
- **Como o checklist pegaria:** seção *Condições de pagamento*, pergunta “o nome usa todas as parcelas, não só a última?” — reprovava o gerarDescricao antigo.

INC — Grupo/Rede não exibido no detalhe do cliente

- **O que aconteceu:** get_cliente_completo_v2 não retornava grupo_rede_nome, então o campo Grupo/Rede deixava de aparecer no detalhe do cliente (aparecia na lista, sumia no detalhe).
- **Invariante violada:** [clientes.md](#) / [convencoes-dados.md](#) — o mapper de detalhe deve trazer os campos *_nome via join, igual à lista.
- **Correção:** migration 141.
- **Como o checklist pegaria:** seção *Clientes / persistência*, pergunta “o mapper de detalhe retorna os *_nome via join, igual à lista?” — reprovava o mapper incompleto.

INC/PITFALL — Número PV-2025-XXXX não é rastreável

- **O que aconteceu:** o número PV-2025-XXXX é gerado no navegador (Date.now) e é **sobrescrito** pelo número do Tiny no envio. Depois de emitido, o pedido não é localizável por esse número original.
- **Invariante violada:** [vendas-emissao-tiny.md](#) — pitfall de rastreabilidade: a chave de busca pós-emissão é o número do Tiny, não o PV- local.
- **Correção:** não é bug de código a corrigir; é um pitfall documentado. Não construir features de rastreio em cima do PV- local.
- **Como o checklist pegaria:** seção *Vendas & emissão Tiny*, pergunta “a rastreabilidade não depende do número PV- gerado no navegador?” — sinalizaria qualquer feature que use o PV- como chave de busca posterior.

INC — Optante Simples sumia no detalhe (acento + omissão)

- **O que aconteceu:** o flag Optante Simples desaparecia no detalhe por dois motivos: (a) acento/variação em tipoPessoa (“Pessoa Jurídica”) não era normalizado; (b) o campo era omitido no mapper de detalhe.
- **Invariante violada:** [fiscal-simples-natureza.md](#) — normalizar tipoPessoa e não omitir campos no mapper de detalhe.
- **Correção:** V1.66.
- **Como o checklist pegaria:** seção *Fiscal / Simples / Natureza*, pergunta “tipoPessoa normalizado e Optante Simples não omitido no mapper?” — reprovava as duas causas.

INC — Emissão com regime Simples não confirmado

- **O que aconteceu:** com a ReceitaWS instável, o regime Simples não era confirmado, mas a emissão prosseguia — com risco de natureza fiscal incorreta.
 - **Invariante violada:** [fiscal-simples-natureza.md](#) — regime Simples não confirmado deve **BLOQUEAR** a emissão (D3), nunca emitir com natureza incorreta.
 - **Correção:** V1.67.
 - **Como o checklist pegaria:** seção *Fiscal / Simples / Natureza*, pergunta “emissão com regime não confirmado é bloqueada (D3)?” — reprovava qualquer caminho que emita sem confirmação.
-